Atlas — Complete Documentation

Multi-tenant SaaS for travel agencies

Generated 2026-06-29 · 47 documents

- Live: https://agencetool.vercel.app
- Repo: github.com/ouks1993/agence_tool

Contents

- docs/cto-review.md

Part 4 — Architecture Decision Records

- docs/decisions/0001-navigation-ia-restructure.md
- docs/decisions/0002-agency-network-stage3.md
- docs/decisions/0003-booking-architecture-sprint1.md
- docs/decisions/0004-travel-platform-facade.md

Part 5 — Specs

- specs/phase-1/PLAN.md
- specs/ui-polish-responsive/README.md
- specs/ui-polish-responsive/action-required.md
- specs/ui-polish-responsive/requirements.md
- specs/ui-polish-responsive/tasks/task-01-globals-css.md
- specs/ui-polish-responsive/tasks/task-02-layout.md
- specs/ui-polish-responsive/tasks/task-03-site-header.md
- specs/ui-polish-responsive/tasks/task-04-site-footer.md
- specs/ui-polish-responsive/tasks/task-05-home-page.md
- specs/ui-polish-responsive/tasks/task-06-dashboard.md
- specs/ui-polish-responsive/tasks/task-07-chat-page.md
- specs/ui-polish-responsive/tasks/task-08-profile-page.md
- specs/ui-polish-responsive/tasks/task-09-auth-pages.md
- specs/ui-polish-responsive/tasks/task-10-setup-checklist.md
- specs/ui-polish-responsive/tasks/task-11-starter-prompt-modal.md

Atlas — Documentation Index

> **Atlas is the operating system for modern travel agencies.** Every decision should support that sentence.

Atlas is a **multi-tenant SaaS for travel agencies**. Each agency runs its clients, sales pipeline, proposals, bookings and finance in a fully isolated workspace; the vendor manages every agency from a platform console. Multilingual (EN/FR/AR with RTL), deployed on Vercel with GitHub auto-deploy.

- **Live:** https://agencetool.vercel.app
- **Repo:** github.com/ouks1993/agence_tool
- **Vendor console:** https://agencetool.vercel.app/platform

This is the reference index. The detailed reference is split into focused docs under `docs/`. See also `DESIGN.md` (design system) and `AGENTS.md`/`CLAUDE.md` (working conventions) — those remain the source of truth for UI and coding conventions respectively.

Documentation map

Doc	Covers
Vision	What Atlas is, who it's for, operating model, features at a glance
Architecture	Multi-tenancy · auth · platform admin · impersonation · roles · route map · module layout · i18n
Tech stack	Framework, libraries, services
Domain model	Core entities, the Lead→…→Feedback chain, aggregates & ownership
Database	Schema tables, tenancy columns, migrations, ID convention
API integrations	Duffel · Hotelbeds · Stripe (billing + Connect) · Resend · OpenRouter · Blob
Development guide	Setup, env vars, commands, scripts, DB workflow
Coding standards	→ points to `AGENTS.md`; conventions summary
UI / UX	→ points to `DESIGN.md`; product UX patterns
Business rules	RBAC, currency, booking lifecycle, commissions, vocabularies
Deployment	Vercel, env, prod migrations, demo accounts
Roadmap & changelog	Phase status, open items, commit changelog
Security	Tenant isolation, auth, guards, webhooks, DB safety, known risks
Analytics & BI	Dashboards, currency-safe metrics, CSV/Excel export

AI	Assistant + inline AI features

At a glance

- **Stack:** Next.js 16 (App Router) · React 19 · TypeScript · Tailwind v4 + shadcn/ui · PostgreSQL (Neon) + Drizzle · Better Auth · next-intl · Stripe + Stripe Connect · Resend · Duffel (flights) · Hotelbeds (hotels) · Vercel AI SDK + OpenRouter · Vercel.
- **Tenancy:** every business row is scoped to an `agencyId`; enforced via `requireAgencyUser()`. Vendor platform admin sits above all tenants.
- **Roles:** admin · manager · finance · support · agent — each with a tailored landing and nav.
- **Deploy:** push to `main` → Vercel auto-deploys production.
- **DBs:** dev `ep-wandering-sunset-aitlty78` · prod `ep-misty-thunder-aixz34vy`. Run `POSTGRES_URL=<prod-url> npx drizzle-kit migrate` after each schema change.

Phases 1–3 + UX + Data-quality/BI + Sprint 1 (booking) + Sprint 2 (Travel Platform) complete. Multi-tenant SaaS with live travel sourcing, billing, client portal, supplier/commission management, and inline AI features. Real supplier search and booking (Duffel + Hotelbeds) fully wired behind a provider-agnostic Travel Platform facade — activate with production credentials. Adding a new provider requires zero consumer changes. Migrations: 19 (latest `0019`).

Vision

> ## Atlas is the operating system for modern travel agencies.
>
> **Every decision should support that sentence.**

It is the north star: a feature, a design choice, or a piece of scope earns its place only if it makes Atlas more of an agency's single operating system. If it doesn't support the sentence, it doesn't ship.

> **No OTA knows agency behaviour. No agency knows market behaviour. Only Atlas sees both.**
>
> That information asymmetry — across thousands of agencies, in real time — is the business. Not the software. Not the AI features. The information.

Today an agency runs on a patchwork of tools:

- Excel
- WhatsApp
- Email
- Booking portals
- Accounting software

Everything should happen inside Atlas instead — **one workspace**.

Long-term goals

- CRM
- Sales
- Quotations
- Flight booking
- Hotel booking
- Supplier management
- Accounting
- Customer portal
- AI assistant
- BI
- Automation

Operating model

- **Per-agency isolation.** Every business record is scoped to an `agencyId`; agencies never see each other's data. Reference numbers (`BKG-…`, `PRD-…`) are unique per agency. See architecture.md and security.md.

- **Vendor + tenants.** The vendor (platform operator) sells Atlas to many agencies and bills them by subscription, managing all of them from a platform console. Each agency is a fully isolated tenant.
- **DZD-first, no FX.** The agency operates in Algerian Dinar. EUR/USD are supported for source data but Atlas never converts between currencies — analytics group by currency rather than summing across. See business-rules.md.
- **Graceful degradation.** Every external integration (flights, hotels, email, payments, AI) falls back to sample/logged behaviour when its keys are unset, so the app runs end-to-end with only a database and an auth secret. See api-integrations.md.

Where it is today

Atlas is a deployed, multi-tenant SaaS — multilingual (EN/FR/AR with RTL), live on Vercel with GitHub auto-deploy.

- **Live:** https://agencetool.vercel.app
- **Repo:** github.com/ouks1993/agence_tool
- **Vendor console:** https://agencetool.vercel.app/platform

Shipped against the long-term goals:

- **CRM** — clients with contacts and a unified funnel timeline.
- **Sales** — opportunities, pipeline stages, conversion analytics.
- **Quotations** — proposals with PDF + e-signature, one-click convert→booking, AI quote builder.
- **Flight booking** — live Duffel search (real order placement is pending).
- **Hotel booking** — live Hotelbeds search + content cache (real booking pending).
- **Supplier management** — directory, contracts & rates, supplier picker.
- **Customer portal** — magic-link login, trip view, online payments, proposal signing.
- **AI assistant** — agency-scoped chat + inline itinerary/quote/email/visa features. See ai.md.
- **BI** — decision dashboards + standardized CSV/Excel export. See analytics.md.
- **Accounting** — partial: payments, two-ledger commissions, AR aging. Full accounting is a long-term goal.
- **Automation** — early: auto-generated commissions, lifecycle guards. Broader automation (e.g. quote-on-stage-change, WhatsApp) is on the roadmap.

Atlas principles

The ten founding principles. Everything else — the design system, the never-rules, the golden workflow — is an expression of these.

1. **Client first.** The client record is the spine; every workflow starts there.
2. **Every entity belongs to an agency.** Multi-tenancy is non-negotiable — `agencyId` on every business row, enforced on every read/write.
3. **No duplicated data.** One source of truth; canonical values over free text.
4. **Automation over manual work.** Generate, don't ask — quotes, itineraries, commissions, documents.

5. **Every workflow has one happy path.** A single, obvious golden route through each flow (business-rules.md).
6. **Managers need insight.** Decision-oriented analytics where managers land.
7. **Agents need speed.** Fast, scoped surfaces — minimal clicks, minimal waiting.
8. **Data quality > flexibility.** Constrain inputs to keep reporting clean.
9. **Everything is reversible.** Prefer soft state and undo over destructive operations.
10. **Every page answers "What do I do next?"** Always surface the next action; never a dead end.

These are operationalized as UI guardrails in ui-ux.md, hard constraints in business-rules.md, and tracked against the code in the gap tracker.

See roadmap.md for phase status and open items, and the index for the full documentation map.

Tech Stack

Layer	Choice
Framework	Next.js 16 (App Router), React 19, TypeScript
Styling	Tailwind CSS v4 (`@theme inline`), shadcn/ui (new-york), Lucide icons
Fonts	Geist (sans/mono), IBM Plex Sans Arabic (RTL)
Auth	Better Auth (email/password, invitation-gated)
Database	PostgreSQL (Neon) + Drizzle ORM
i18n	next-intl 4 (cookie-based, no URL routing)
Charts	recharts 3 (tokenized)
AI	Vercel AI SDK + OpenRouter
Email	Resend (transactional: invites, password reset, proposals)
Billing	Stripe subscriptions (vendor → agency) + webhook
Payments	Stripe Connect (traveler → agency, destination charges)
Flights	Duffel (Amadeus self-service kept only as legacy fallback)
Hotels	Hotelbeds (APITUDE: availability + content)
PDF	`@react-pdf/renderer` (server-rendered proposals)
Data export	`exceljs` (XLSX) + built-in CSV (UTF-8 BOM) for BI/Power BI
Storage	Vercel Blob (optional)
Hosting	Vercel + GitHub auto-deploy

Conventions tied to the stack

- **Styling** follows the design system in `DESIGN.md`; see ui-ux.md.
- **Working conventions** (sub-agents, UUID ids, Drizzle workflow, testing) live in `AGENTS.md`; see coding-standards.md.
- **External integrations** and their env vars are documented in api-integrations.md. Every one degrades gracefully when unconfigured.
- **Tailwind v4** is configured CSS-first via `@theme inline` in `globals.css` — there is no `tailwind.config.ts`.

Architecture

Tech choices live in tech-stack.md. This doc covers structure: multi-tenancy, auth, roles, the route map, the module layout, and i18n plumbing.

Multi-tenancy

Every business table carries `agencyId` (tenant roots) or inherits it through a parent (children). **All** reads/writes are scoped by agency, enforced in actions and pages via `requireAgencyUser()` → `user.agencyId`. References (`BKG-…`, `PRD-…`) are unique **per agency**. Re-verified by `scripts/test-tenant-isolation.ts`. See security.md for the isolation guarantees and database.md for tenancy columns.

Auth & onboarding

- Better Auth (`src/lib/auth.ts`) — email/password. The `user.create.before` hook makes signup **invitation-only**: it requires a pending `agency_invite` matching the email, stamps `agencyId` + role, and rejects everyone else (blocks the raw signup endpoint too). `BETTER_AUTH_URL`/`baseURL` + `trustedOrigins` set so the deployed domain is trusted.
- Guards (`src/lib/permissions.ts`): `requireUser`, `requireAgencyUser` (tenant + agency-suspension lockout), `requireManager`, `requireCapability`, `requirePlatformAdmin`.

Platform admin (vendor)

A user with `isPlatformAdmin = true`, `agencyId = null` — above all tenants, routed to `/platform`. Cannot enter a tenant app except via impersonation.

Impersonation (cookie-driven, platform-admin only)

- `viewAsAgency(agencyId)` → cookie `platform_view_agency` → acts as agency **admin**.
- `viewAsUser(userId)` → cookie `platform_view_user` (takes precedence) → adopts that user's identity, agency and **role** (full fidelity — an agent sees only their work).
- Resolved in `requireUser`; `user.impersonating` = `"agency"` | `"user"` | `null` drives the exit banner. `exitAgencyView()` clears both cookies.

Per-role landing

`roleHome(role)` routes finance → `/finance`, support → `/support`, else `/dashboard` (which itself adapts: agency-wide for admin/manager, scoped "Your work" for agents). App-shell nav is role-aware via a `show(role)` predicate per item.

Roles & permissions

Defined in `src/lib/domain.ts`. Full capability semantics are in business-rules.md.

Role	Sees all data	Team mgmt	Payments	Finance view	Support view	Delete	Home
admin	✓	✓	✓	✓	✓	✓	/dashboard
manager	✓	✓	✓	✓	✓	✓	/dashboard
finance	✓	—	✓	✓	—	—	/finance
support	✓	—	—	—	✓	—	/support
agent	own only	—	—	—	—	—	/dashboard (scoped)

Capability helpers: `seesAllData`, `canManageTeam`, `canAssignAdmin`, `canManagePayments`, `canViewFinance`, `canViewSupport`, `canDeleteRecords`, `roleHome`. Only an **admin** can assign/change the admin role.

Scale targets

The capacity Atlas is designed toward, with implications for the current architecture (single Neon Postgres + Vercel serverless).

Target	Implication / current state
1,000 agencies	Multi-tenant single DB; fine with `agencyId` indexes (present).
50,000 users	Better Auth + Postgres; fine at this scale.
5,000,000 bookings	Needs solid indexing + pagination (currently hardcoded `limit`s — see tracker).
500,000,000 activities	`activity_log` at this size needs **partitioning + archival/retention** — not in place today (single unpartitioned table).
99.9% uptime	Relies on Vercel + Neon SLAs; no DR runbook yet (see security.md).
100 concurrent hotel searches	Bound by Hotelbeds quota; content cache serves photos quota-free. No concurrency/queue control or rate limiting yet.
100 concurrent flight searches	Bound by Duffel quota; `safeSearch` degrades to sample data on error. No queue/backpressure yet.

> These are design targets. The gating work to reach them — table partitioning for `activity_log`, real pagination, rate limiting/backpressure on search, and a DR plan — is in the gap tracker.

Route map

Authenticated app (`(app)/`, gated by `requireAgencyUser`): `dashboard`, `finance`, `commissions`

(canViewFinance), `support`, `bookings` (+ `new`, `[id]`, `[id]/edit`, `[id]/itinerary`), `clients` (+ `new`, `[id]`, `[id]/edit`), `opportunities` (+ `new`, `[id]`, `[id]/edit`), `products` (+ `new`, `[id]`, `[id]/edit`), `suppliers` (+ `new`, `[id]`, `[id]/edit`) (canManageTeam), `operations` (Pipeline board), `reports` (canViewFinance — BI export hub), `search`, `hotels` (+ `[code]` details), `assistant`, `team` (canManageTeam), `billing` (admin-only), `settings`, `profile`.

Client portal (`portal/`, gated by `requirePortalSession`): `portal` (trip list), `portal/bookings/[id]` (detail + pay), `portal/proposals` (list), `portal/proposals/[id]` (view + sign), `portal/login`.

Platform (`platform/`, gated by `requirePlatformAdmin`): `platform`, `platform/agencies/new`, `platform/agencies/[id]`.

Auth (`(auth)/`): `login`, `register` (invite-only notice), `forgot-password`, `reset-password`. **Accept invite:** `invite/[token]`.

Public / docs: `i/[token]` (shareable itinerary, unauth), `p/[token]` (public signable proposal) + `p/[token]/pdf`, `proposal/[id]` (internal preview) + `proposal/[id]/pdf`, `booking-docs/[id]/voucher`, `booking-docs/[id]/invoice`.

API: `api/auth/[...all]` (Better Auth), `api/chat` (AI assistant), `api/export/[entity]` (BI export: CSV/XLSX per dataset + `workbook` = all sheets), `api/stripe/webhook` (subscription reconciliation), `api/stripe/connect-webhook` (Connect payment reconciliation), `api/portal/auth/request` · `verify` · `signout` (portal magic-link flow).

Module layout

`src/lib/`

- `domain.ts` — roles, capabilities, enums (statuses, stages, item types, currencies + controlled vocabularies: lead source, travel purpose, trip type, gender, title, lost reason, industry), `roleHome`, status/role metadata.
- `analytics.ts` — pure dashboard helpers (see analytics.md).
- `reference/countries.ts` — ISO 3166-1 list (name, demonym, derived flag) + `normalizeCountry()` alias mapping.
- `export/` — `csv.ts` (RFC-4180 + UTF-8 BOM), `xlsx.ts` (exceljs workbook), `datasets.ts` (BI dataset registry with dual code/label columns).
- `permissions.ts` — auth guards + impersonation resolution.
- `auth.ts` / `auth-client.ts` — Better Auth config + client.
- `invites.ts` — create/find/accept invite tokens (7-day TTL).
- `queries.ts` — shared agency-scoped pickers.
- `activity.ts` — `logActivity` (agency-scoped audit).
- `db.ts` (validates env via `getServerEnv`), `schema.ts`, `env.ts`, `config.ts`, `format.ts`, `utils.ts`, `itinerary.ts`, `storage.ts`.
- `notifications/email.ts` (Resend adapter) + `notifications/templates.ts` (HTML).
- `billing/stripe.ts` — SaaS subscriptions (distinct from `payments/stripe.ts`).
- `payments/stripe.ts` — Stripe Connect: account, onboarding link, destination charges.
- `suppliers/` — `index.ts` (`getFlightSupplier`/`getHotelSupplier` + `safeSearch`), `duffel.ts`, `hotelbeds.ts`, `content-cache.ts`, `amadeus.ts` (legacy), `mock.ts`, `types.ts`.

- `documents/proposal-pdf.tsx` + `proposal-data.ts` — proposal PDF rendering.
- `portal-session.ts` — `getPortalSession` / `requirePortalSession` (httpOnly cookie).

`src/lib/actions/` (server actions, all agency-scoped): `clients`, `opportunities`, `products`, `proposals-public`, `bookings`, `payments`, `notifications`, `team`, `invites`, `platform`, `billing`, `settings`, `search`, `suppliers`, `commissions`, `portal-payments`, `portal-proposals`, `portal-invite`, `onboarding`, `ai`.

`src/components/` — `app/` (shell, page-header, stat-card, status-badge, getting-started-card), `charts/` (insight-charts incl. `HBarInsight`, `FunnelInsight`), `reference/` (country-combobox, city-input), `reports/`, `settings/`, `team/`, `platform/`, `auth/`, `bookings/` (lifecycle stepper, search sheet, board), `clients/` (portal invite, timeline), `products/` (convert-to- booking, AI quote builder), `opportunities/`, `billing/`, `commissions/`, `suppliers/`, `portal/`, `search/`, `documents/`, `ui/` (shadcn).

Internationalization

- **Locales:** `en`, `fr`, `ar` (Arabic = RTL). Cookie-based (`locale`), no URL-locale routing — all routes unchanged.
- `src/i18n/` — `config.ts` (locales, metadata, dir), `request.ts` (next-intl request config reading the `locale` cookie). Messages: `messages/{en,fr,ar}.json`.
- Root layout sets `<html lang dir>` and applies IBM Plex Sans Arabic for RTL (`html[dir="rtl"] body` in `globals.css`).
- Translated surfaces: nav, login, dashboard, settings. Others fall back to English. **To translate a string:** add the key to all three `messages/*.json` and swap to `t("…")` (`getTranslations` in server, `useTranslations` in client).
- Language is changed in **Settings**; choice is saved to `user.locale` and the cookie. **Known gap:** a fresh device without the cookie shows English until the user re-picks — `user.locale` isn't yet synced to the cookie on login (see roadmap.md open items).

Domain Model

The core business entities and how they relate. This is the **data/relationship** view; the process view (states, who does what) is the golden workflow. Tables and columns live in database.md.

The chain

```
Lead → Client → Opportunity → Proposal → Booking → Supplier →
Invoice → Payment → Accounting → Travel → Feedback
```

The **client is the aggregate root** — everything hangs off it, and every entity is scoped to an `agencyId` (founding principle #2).

Entities

Domain concept	Table(s)	Status	Key relations
Lead	`client` (status `lead`, `source` code)	✓	becomes a Client; no separate table
Client	`client` (+ `client_contact`)	✓	root; owns opportunities, proposals, bookings
Opportunity	`opportunity`	✓	belongs to a client; `assignedToId`; links to a proposal
Proposal	`product` (+ `product_item`)	✓	belongs to client; e-sign; converts → Booking
Booking	`booking` (+ `booking_traveller`, `booking_item`, `booking_day`, `booking_supplier_ref`, `booking_event`, `booking_document`, `booking_idempotency`)	✓	belongs to client; lifecycle states; items reference suppliers; Sprint 1 tables power real booking, audit log, and idempotency
Supplier	`supplier` (+ `supplier_contract`, `supplier_rate`)	✓	referenced by `booking_item.supplierId` + `product_item`
Invoice	— (on-demand PDF: `booking-docs/[id]/invoice`)	🟡	no managed invoice entity yet (planned module)
Payment	`payment` (child of booking)	✓	deposits/installments; Stripe Connect
Accounting	`commission` ledger only	🟡	partial; no GL/accounting entity (planned module)

Travel	`booking` (state `completed`) + `booking_day`	✓	itinerary timeline; vouchers
Feedback	— (`activity_log` / notes)	◍	no review/feedback entity (planned module)

Aggregates & ownership

- **Client aggregate** — `client` → `client_contact`; the spine that opportunities, proposals, and bookings all reference by `clientId`.
- **Booking aggregate** — `booking` → `booking_traveller`, `booking_item`, `booking_day`, `payment`, `booking_supplier_ref`, `booking_event`, `booking_document`, `booking_idempotency`. Children carry **no** `agencyId`; they inherit tenancy through the parent booking (scoped via the parent on every query). `booking_event` is append-only (audit + analytics); `booking_idempotency` prevents duplicate supplier orders on retry.
- **Supplier aggregate** — `supplier` → `supplier_contract` → `supplier_rate`; `commission` rows link a booking item back to the earning supplier/agent.
- **Reference data** — `hotel_content` is **global** (not tenant-scoped); ISO countries/vocabularies are canonical lookups, not per-agency rows.

Where the chain is aspirational

Three links are not yet first-class domain entities — they're the bridge to the planned modules:

- **Invoice** — today an on-demand PDF; the Accounting module turns it into a numbered, ledgered record.
- **Accounting** — only the two-ledger `commission` exists; no general ledger.
- **Feedback** — not modelled; "Travel completed → request review" is an unbuilt automation trigger.

See the entity standard for the baseline fields every one of these should carry.

Database

Schema lives in `src/lib/schema.ts`. PostgreSQL (Neon) + Drizzle ORM. Tenancy column shown where present; see architecture.md and security.md for how scoping is enforced.

Entity standard

Every business entity should carry this baseline. It operationalizes the never rules (one source of truth, no hard delete, no cross-tenant exposure) at the schema level.

Field	Purpose	Current state
UUID	Primary key, randomly generated (non-Better-Auth IDs)	✓ convention in place
Reference	Human-facing id unique per agency (`BKG-…`, `PRD-…`)	⚠ only `booking` + `product` today
createdAt	Creation timestamp	✓ widespread
updatedAt	Last-modified timestamp	⚠ on roots, missing on some children
createdBy	`createdById` → user	✓ on main entities
agencyId	Tenant scope (root) or inherited (child)	✓ enforced everywhere
Status	Lifecycle/state enum	⚠ per-entity, not universal
Activity log	Audit trail of changes	⚠ shared `activity_log` table, not wired for every entity
Soft delete	`deletedAt` — never hard delete	✕ **no soft-delete column exists yet**
Notes	Free-form internal notes	⚠ only some entities

> **Gap to close:** soft delete is **not implemented** — there is no `deletedAt` anywhere, so the "never hard delete" rule is currently aspirational. Adding a nullable `deletedAt` + filtering it out of reads (and a `reference` on the remaining entities) would need a migration. Until then, deletes are real.

Tables

Table	Tenancy	Notes
`agency`	(root)	name, slug, status (active/suspended); **Stripe billing**: stripeCustomerId, stripeSubscriptionId, subscriptionStatus, priceId, currentPeriodEnd, trialEndsAt; **Connect**: stripeConnectAccountId, stripeConnectOnboarded; `onboardingDismissedAt` (getting-started card)

agency_invite	agencyId	email, role, token, status, expiresAt
user	agencyId (nullable)	+ isPlatformAdmin, role, active, locale, commissionRatePercent (Better Auth)
session, account, verification	via user	Better Auth
portal_session	via client	passwordless client portal sessions; token + expiresAt
client	agencyId	+ client_contact (child); source (lead-source code), industry (code), country stored as canonical ISO name
opportunity	agencyId	pipeline stage, value, currency; travel_purpose code; lost_reason code
product	agencyId	proposal; ref unique per agency; + product_item (child; supplierId FK optional); **e-sign**: shareToken (unique), acceptedAt/declinedAt, signerName/Email, signatureData, signerIp/ UserAgent
booking	agencyId	ref unique per agency; shareToken; travel_purpose + trip_type codes
booking_traveller (+ title, gender codes), booking_item (+ supplierId FK), payment, booking_day	via booking	children; booking_traveller carries email + phone (required by supplier APIs for the lead passenger)
booking_supplier_ref	via booking + booking_item	structured supplier confirmation: providerId, confirmationNumber, pnr, supplierOrderId, rawPayload; replaces untyped JSONB in booking_item.details
booking_event	bookingId + agencyId	append-only event log (audit + analytics); stable event codes: search_initiated, offer_selected, price_validated, price_changed, booking_submitted, booking_confirmed, booking_failed, booking_cancelled, payment_started, payment_completed
booking_document	via booking (+ optional booking_item)	documents: type (voucher/ticket/invoice/itinerary/ receipt), url (Vercel Blob), rawData (supplier payload for re-generation)
booking_idempotency	via booking (+ optional booking_item)	idempotency key registry; key = sha256(bookingId+itemId+offerId); prevents duplicate supplier orders on retry; expiresAt (24 h TTL)
notification	agencyId	comms log

`activity_log`	agencyId	audit trail
`supplier`	agencyId	managed supplier directory (hotels, airlines, DMC, etc.)
`supplier_contract`	agencyId	commission basis/rate, validity dates, file URL
`supplier_rate`	via contract	structured per-product rates within a contract
`commission`	agencyId	earnings ledger: supplier→agency and agency→agent; bookingId, supplierId, agentUserId, basis, rate, amount, status
`hotel_content`	(global)	Hotelbeds content cache (photos, facilities, coords) — shared reference data, **not** tenant-scoped; PK is the Hotelbeds hotel code

ID convention

All ID columns **not** related to Better Auth use UUIDs, randomly generated. See coding-standards.md.

Migrations

Migration	What it adds
`0006`	Tenancy + backfill (`agencyId` on all tables)
`0007`	`agency_invite` table
`0008`	`user.locale`
`0009`	Agency Stripe billing columns
`0010`	Product e-signature columns
`0011`	`hotel_content` cache
`0012`	Stripe Connect columns on `agency`
`0013`	`portal_session` table
`0014`	`supplier`, `supplier_contract`, `supplier_rate` tables; `supplierId` FK on `booking_item` + `product_item`
`0015`	`commission` table; `user.commissionRatePercent`
`0016`	`agency.onboardingDismissedAt`
`0017`	Controlled-vocab columns: `client.industry`, `opportunity.travel_purpose`, `booking.travel_purpose`/`trip_type`, `booking_traveller.title`/`gender` (all nullable, additive)
`0018`	(see existing)

0019	Sprint 1 booking architecture: booking_supplier_ref, booking_event, booking_document, booking_idempotency tables; booking_traveller.email + .phone columns

> Migrations 0000–0005 predate the multi-tenant rework (the pre-tenancy base schema) and are not itemized here.

Workflow

```
db:generate   →   db:migrate     # NEVER db:push
```

Run on prod after deploy: POSTGRES_URL=<prod-url> npx drizzle-kit migrate. Branches: dev ep-wandering-sunset-aitlty78 · prod ep-misty-thunder-aixz34vy. Full setup in development-guide.md.

Schema Reference

Full per-table, per-column reference. Schema is Drizzle ORM + PostgreSQL (Neon). Source of truth: `src/lib/schema.ts`. High-level overview: database.md.

Convention — `id` is always `uuid PRIMARY KEY DEFAULT gen_random_uuid()` on business tables unless noted. Better Auth tables use `text` PKs. All timestamps are `timestamptz`.

Auth — Better Auth managed

`user`

Column	Type	Constraints	Notes
`id`	`text`	PK	Better Auth managed
`name`	`text`	NOT NULL	Display name
`email`	`text`	NOT NULL UNIQUE	Login credential
`email_verified`	`boolean`	NOT NULL DEFAULT false	
`image`	`text`		Avatar URL
`agency_id`	`uuid`	FK → `agency` CASCADE	null = platform admin or not yet onboarded
`is_platform_admin`	`boolean`	NOT NULL DEFAULT false	Vendor console access
`role`	`text`	NOT NULL DEFAULT `"agent"`	`admin` | `manager` | `finance` | `support` | `agent`
`locale`	`text`		`"en"` | `"fr"` | `"ar"` — UI language preference
`active`	`boolean`	NOT NULL DEFAULT true	Soft-disable without deleting history
`commission_rate_percent`	`numeric(5,2)`		Default commission % this agent earns
`created_at`	`timestamp`	NOT NULL DEFAULT now()	
`updated_at`	`timestamp`	NOT NULL	Auto-set on update

Indexes: `user_email_idx`, `user_agency_idx`

session

Column	Type	Notes
id	text	PK
expires_at	timestamp	NOT NULL
token	text	NOT NULL UNIQUE
ip_address	text	
user_agent	text	
user_id	text	FK → user CASCADE
created_at	timestamp	NOT NULL
updated_at	timestamp	NOT NULL

account

OAuth/credential accounts linked to a user. Better Auth managed.

Column	Type
id	text PK
account_id	text NOT NULL
provider_id	text NOT NULL
user_id	text FK → user CASCADE
access_token, refresh_token, id_token	text
access_token_expires_at, refresh_token_expires_at	timestamp
scope, password	text

verification

Magic-link / email verification tokens. Better Auth managed.

Column	Type
id	text PK
identifier	text NOT NULL
value	text NOT NULL
expires_at	timestamp NOT NULL

Agency & onboarding

agency

Column	Type	Notes
id	uuid	PK
name	text	NOT NULL
slug	text	NOT NULL UNIQUE
status	text	"active" | "suspended" DEFAULT "active"
Stripe billing		
stripe_customer_id	text	
stripe_subscription_id	text	
subscription_status	text	Stripe status string
price_id	text	Stripe price for the current plan
current_period_end	timestamp	Subscription period end
trial_ends_at	timestamp	14-day trial expiry on first provision
Stripe Connect		
stripe_connect_account_id	text	Agency's Express account
stripe_connect_onboarded	boolean	DEFAULT false — true after Connect flow
Onboarding		
onboarding_dismissed_at	timestamp	Set when agency dismisses the getting-started card
created_at	timestamp	NOT NULL
updated_at	timestamp	NOT NULL

agency_invite

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
email	text	NOT NULL
role	text	NOT NULL — target role after sign-up
token	text	NOT NULL UNIQUE — unguessable magic link
status	text	"pending" | "accepted" | "expired" DEFAULT "pending"
invited_by_id	text	FK → user SET NULL
expires_at	timestamp	NOT NULL
created_at	timestamp	NOT NULL

CRM

`client`

Column	Type	Notes
`id`	`uuid`	PK
`agency_id`	`uuid`	FK → `agency` CASCADE
`name`	`text`	NOT NULL
`type`	`text`	`"individual"` | `"corporate"` DEFAULT `"individual"`
`status`	`text`	`"lead"` | `"active"` | `"inactive"` DEFAULT `"active"`
`email`	`text`	
`phone`	`text`	
`company`	`text`	Corporate name
`address`	`text`	
`city`	`text`	
`country`	`text`	Canonical ISO 3166-1 name (full name, not code)
`source`	`text`	Lead-source vocab code
`industry`	`text`	Industry vocab code (corporate clients)
`notes`	`text`	
`owner_id`	`text`	FK → `user` SET NULL — relationship owner
`created_by_id`	`text`	FK → `user` SET NULL
`created_at`	`timestamp`	NOT NULL
`updated_at`	`timestamp`	NOT NULL

Indexes: `client_agency_idx`, `client_owner_idx`, `client_status_idx`, `client_name_idx`, `client_agency_created_idx` **Unique:** none (names may repeat)

`client_contact`

Additional contacts for corporate clients.

Column	Type
`id`	`uuid` PK
`client_id`	`uuid` FK → `client` CASCADE NOT NULL
`name`	`text` NOT NULL

job_title	text
email	text
phone	text
is_primary	boolean DEFAULT false NOT NULL
created_at	timestamp NOT NULL

Sales pipeline

opportunity

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
title	text	NOT NULL
client_id	uuid	FK → client CASCADE NOT NULL
stage	text	"lead" | "qualified" | "proposal" | "booked" | "won" | "lost" DEFAULT "lead"
value	numeric(12,2)	DEFAULT 0
currency	text	DEFAULT "EUR"
probability	integer	0–100 DEFAULT 10
destination	text	
travel_start_date	timestamp	
travel_end_date	timestamp	
pax_count	integer	DEFAULT 1
travel_purpose	text	Vocab code
expected_close_date	timestamp	
lost_reason	text	Vocab code — only when stage = "lost"
notes	text	
assigned_to_id	text	FK → user SET NULL
created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL
updated_at	timestamp	NOT NULL

Indexes: `opportunity_agency_idx`, `opportunity_client_idx`, `opportunity_stage_idx`, `opportunity_assigned_idx`, `opportunity_agency_stage_idx`, `opportunity_agency_created_idx`

Proposals (Products)

`product`

Column	Type	Notes
`id`	`uuid`	PK
`agency_id`	`uuid`	FK → `agency` CASCADE
`reference`	`text`	NOT NULL — e.g. `"PRD-1042"` unique per agency
`title`	`text`	NOT NULL
`client_id`	`uuid`	FK → `client` SET NULL
`opportunity_id`	`uuid`	FK → `opportunity` SET NULL
`status`	`text`	`"draft"` | `"sent"` | `"accepted"` | `"rejected"` | `"expired"` DEFAULT `"draft"`
`destination`	`text`	
`start_date`, `end_date`	`timestamp`	
`pax_count`	`integer`	DEFAULT 1
`currency`	`text`	DEFAULT `"EUR"`
`markup_percent`	`numeric(5,2)`	DEFAULT 0
`total_cost`	`numeric(12,2)`	DEFAULT 0
`total_price`	`numeric(12,2)`	DEFAULT 0
`summary`	`text`	Client-facing narrative (AI-generated)
`valid_until`	`timestamp`	
E-signature		
`share_token`	`text`	UNIQUE — public `/p/[token]` link
`accepted_at`, `declined_at`	`timestamp`	
`signer_name`, `signer_email`	`text`	Non-repudiation
`signature_data`	`text`	Typed name or drawn-signature data URL
`signer_ip`, `signer_user_agent`	`text`	

created_by_id	text	FK → user SET NULL
created_at, updated_at	timestamp	NOT NULL

Unique: product_agency_reference_unique (agency_id, reference)

product_item

Line items within a proposal.

Column	Type	Notes
id	uuid	PK
product_id	uuid	FK → product CASCADE NOT NULL
supplier_id	uuid	FK → supplier SET NULL (optional)
type	text	"flight" | "hotel" | "activity" | "transfer" | "insurance" | "other"
title	text	NOT NULL
description, supplier	text	
quantity	integer	DEFAULT 1 NOT NULL
unit_cost, unit_price	numeric(12,2)	DEFAULT 0 NOT NULL
currency	text	DEFAULT "EUR"
start_date, end_date	timestamp	
details	jsonb	Raw supplier offer payload
sort_order	integer	DEFAULT 0
created_at	timestamp	NOT NULL

Bookings

booking

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
reference	text	NOT NULL — e.g. "BKG-1042" unique per agency
client_id	uuid	FK → client SET NULL

status	text	"draft" | "awaiting_payment" | "confirmed" | "ticketed" | "completed" | "cancelled" DEFAULT "draft"
destination	text	
depart_date, return_date	timestamp	
travel_purpose	text	Vocab code
trip_type	text	"one_way" | "round_trip" | "multi_city"
currency	text	DEFAULT "EUR"
notes	text	
total_amount	numeric(12,2)	DEFAULT 0
share_token	text	UNIQUE — shareable itinerary /i/[token]
created_by_id	text	FK → user SET NULL
created_at, updated_at	timestamp	NOT NULL

Lifecycle prerequisite rules (server-side enforced):

- confirmed requires: trip items present AND zero outstanding balance.
- ticketed requires: trip items present.

Unique: booking_agency_reference_unique (agency_id, reference)

booking_traveller

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
full_name	text	NOT NULL
title	text	Vocab code (mr | mrs | ms | dr | prof)
gender	text	Vocab code (male | female | unspecified)
passport_number	text	
passport_expiry	timestamp	
nationality	text	ISO country name
date_of_birth	timestamp	

passport_issue_date, passport_issue_place	timestamp / text	
email	text	Required by Duffel for lead passenger
phone	text	Required by Duffel for lead passenger
is_lead	boolean	NOT NULL DEFAULT false — first passenger = lead
sort_order	integer	DEFAULT 0
created_at	timestamp	NOT NULL

booking_item

A purchased line item (flight, hotel, transfer, fee, etc.).

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
supplier_id	uuid	FK → supplier SET NULL (optional)
type	text	"flight" | "hotel" | "excursion" | "transfer" | "insurance" | "fee" | "other"
title	text	NOT NULL
description, supplier	text	
booking_ref	text	Supplier confirmation / PNR reference
start_date, end_date	timestamp	
quantity	integer	DEFAULT 1
amount	numeric(12,2)	DEFAULT 0 — price charged to client (per unit)
currency	text	DEFAULT "EUR"
item_status	text	"pending" | "confirmed" | "ticketed" | "cancelled" DEFAULT "pending"
confirmation_number	text	Supplier confirmation number once booked
day_index	integer	Itinerary day (0-based from departure; null = auto by date)
details	jsonb	Raw supplier offer payload
sort_order	integer	DEFAULT 0
created_at	timestamp	NOT NULL

payment

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
amount	numeric(12,2)	NOT NULL
currency	text	DEFAULT "EUR"
kind	text	"deposit" | "installment" | "payment" | "refund" DEFAULT "payment"
method	text	"manual" | "card" | "transfer" | "cash" | "stripe" DEFAULT "manual"
status	text	"pending" | "completed" | "failed" | "refunded" DEFAULT "completed"
reference	text	Stripe payment id or manual reference
stripe_session_id	text	Stripe Checkout Session id (Connect flow)
checkout_url	text	Hosted Checkout URL sent to client
note	text	
created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL

booking_day

Per-day itinerary titles and notes.

Column	Type
id	uuid PK
booking_id	uuid FK → booking CASCADE NOT NULL
day_index	integer NOT NULL (0-based)
title	text
notes	text

notification

Communications log.

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
booking_id	uuid	FK → booking CASCADE (nullable)

channel	text	"email" | "sms" | "whatsapp" | "push" DEFAULT "email"
recipient	text	NOT NULL — email address or phone
subject	text	
body	text	
kind	text	"confirmation" | "voucher" | "receipt" | "custom" | "invite" | "proposal" DEFAULT "custom"
status	text	"sent" | "failed" | "logged" DEFAULT "sent"
error	text	Error message when status = "failed"
created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL

Sprint 1 — Booking lifecycle tables

booking_supplier_ref

Structured supplier confirmation. Replaces ad-hoc JSONB in booking_item.details.

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
booking_item_id	uuid	FK → booking_item CASCADE NOT NULL
provider_id	text	NOT NULL — "duffel" | "hotelbeds" | "mock"
confirmation_number	text	NOT NULL — shown to client
pnr	text	Airline PNR / record locator
supplier_order_id	text	Provider's internal order id (e.g. Duffel order_xxx)
raw_payload	jsonb	Full raw response for debugging
created_at	timestamp	NOT NULL

Indexes: booking_supplier_ref_booking_idx, booking_supplier_ref_item_idx, booking_supplier_ref_provider_idx

booking_event

Append-only audit + analytics log. Never update or delete rows.

Column	Type	Notes
id	uuid	PK

booking_id	uuid	FK → booking CASCADE NOT NULL
agency_id	uuid	FK → agency CASCADE NOT NULL (for agency-scoped analytics)
event	text	NOT NULL — stable event code (see booking-architecture.md §6)
provider_id	text	"duffel", "hotelbeds", "mock", or null for UI events
correlation_id	text	Request-level trace id
metadata	jsonb	Structured payload (offer id, price, error code, etc.)
created_at	timestamp	NOT NULL

Event codes: search_initiated, offer_selected, price_validated, price_changed, booking_submitted, booking_confirmed, booking_failed, booking_cancelled, payment_started, payment_completed

Indexes: booking_event_booking_idx, booking_event_agency_idx, booking_event_event_idx, booking_event_created_idx, booking_event_booking_created_idx

booking_document

Documents generated for a booking (vouchers, tickets, invoices, itineraries).

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
booking_item_id	uuid	FK → booking_item SET NULL (null = booking-level doc)
type	text	NOT NULL — "voucher" | "ticket" | "invoice" | "itinerary" | "receipt"
provider_id	text	Supplier that issued the document
url	text	Vercel Blob URL or supplier CDN link
raw_data	jsonb	Supplier payload for re-generation
generated_at	timestamp	When the document was issued
created_at	timestamp	NOT NULL

booking_idempotency

Key registry preventing duplicate supplier orders on retry.

Column	Type	Notes
key	text	PK — sha256(bookingId:itemId:offerId)
booking_id	uuid	FK → booking CASCADE NOT NULL
booking_item_id	uuid	FK → booking_item CASCADE

provider_id	text	NOT NULL — provider the key was sent to
status	text	"pending" | "success" | "failed" DEFAULT "pending"
supplier_ref	text	Confirmation number when status = "success"
created_at	timestamp	NOT NULL
expires_at	timestamp	NOT NULL — 24h TTL; safe to clean up after expiry

Indexes: booking_idempotency_booking_idx, booking_idempotency_expires_idx

Supplier management

supplier

Agency's managed supplier directory (hotels, airlines, DMCs, etc.).

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
name	text	NOT NULL — unique per agency
type	text	"hotel" | "airline" | "car_rental" | "transfer" | "dmc" | "insurance" | "other"
status	text	"active" | "inactive" DEFAULT "active"
email, phone, website	text	
address, city, country	text	
contact_name	text	
notes	text	
created_by_id	text	FK → user SET NULL
created_at, updated_at	timestamp	NOT NULL

Unique: supplier_agency_name_unique (agency_id, name)

supplier_contract

Commercial contracts with commission terms.

Column	Type	Notes

id	uuid	PK
supplier_id	uuid	FK → supplier CASCADE NOT NULL
agency_id	uuid	FK → agency CASCADE (denormalized for query efficiency)
name	text	NOT NULL
reference	text	Contract number
commission_basis	text	"percent" | "fixed" | "net"
commission_rate	numeric(5,2)	% or fixed amount depending on basis
valid_from, valid_to	timestamp	
file_url	text	Vercel Blob URL for the PDF contract
notes	text	
created_at, updated_at	timestamp	NOT NULL

supplier_rate

Structured per-product rates within a contract.

Column	Type	Notes
id	uuid	PK
contract_id	uuid	FK → supplier_contract CASCADE NOT NULL
name, description	text	
rate_type	text	Type of rate (product/season/tier)
amount	numeric(12,2)	
currency	text	
valid_from, valid_to	timestamp	
created_at	timestamp	NOT NULL

commission

Two-ledger earnings record. Auto-generated on booking confirm/ticket.

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
booking_id	uuid	FK → booking CASCADE

booking_item_id	uuid	FK → booking_item (nullable)
supplier_id	uuid	FK → supplier SET NULL
agent_user_id	text	FK → user SET NULL
ledger	text	"supplier_to_agency" | "agency_to_agent"
basis	text	"percent" | "fixed" | "net"
rate	numeric(5,2)	
amount	numeric(12,2)	NOT NULL — computed commission
currency	text	
status	text	"pending" | "confirmed" | "paid" | "cancelled"
notes	text	
created_at, updated_at	timestamp	NOT NULL

Client portal

portal_session

Passwordless sessions for the client-facing traveler portal. Entirely separate from the Better Auth staff session system.

Column	Type	Notes
id	uuid	PK
client_id	uuid	FK → client CASCADE NOT NULL
token	text	NOT NULL UNIQUE — magic-link token (15 min) → session token (7 days)
expires_at	timestamp	NOT NULL
created_at	timestamp	NOT NULL

Magic-link flow: token is short-lived (15 min); on verification the row is updated with a rotated long-lived (7-day) token stored in an httpOnly cookie.

Reference data (global, not tenant-scoped)

hotel_content

Hotelbeds Content API cache. PK is the Hotelbeds hotel code (text), **not** UUID. Shared across all agencies — intentionally not scoped to agency_id.

Column	Type	Notes
code	text	PK — Hotelbeds hotel code (e.g. "12345")
name	text	NOT NULL
stars	integer	DEFAULT 0
hotel_type	text	
description	text	
address, city, country, postal_code	text	
latitude, longitude	numeric(10,6)	
destination_code	text	Hotelbeds destination code (e.g. "BCN")
segments	jsonb	Marketing segment tags: string[]
facilities	jsonb	Amenity names: string[]
images	jsonb	{ url: string; roomCode?: string }[]
updated_at	timestamp	NOT NULL — sync timestamp

Index: hotel_content_destination_idx (destination_code)

Audit

activity_log

Records every meaningful action for manager oversight.

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
user_id	text	FK → user SET NULL
action	text	"created" | "updated" | "deleted" | "stage_changed" | "sent" | "status_changed"
entity_type	text	"client" | "opportunity" | "product" | "user"
entity_id	text	
entity_label	text	Human-readable name kept even if entity is deleted
metadata	jsonb	Diff / context payload
created_at	timestamp	NOT NULL

Indexes: `activity_agency_idx`, `activity_user_idx`, `activity_entity_idx (entity_type, entity_id)`, `activity_created_idx`

Migrations summary

Migration	Description
`0000`–`0005`	Pre-tenancy base schema (not itemized)
`0006`	Multi-tenant `agencyId` backfill
`0007`	`agency_invite`
`0008`	`user.locale`
`0009`	Agency Stripe billing columns
`0010`	Product e-signature columns
`0011`	`hotel_content` cache
`0012`	Stripe Connect columns on `agency`
`0013`	`portal_session`
`0014`	`supplier`, `supplier_contract`, `supplier_rate`; `supplier_id` FK on `booking_item` + `product_item`
`0015`	`commission`; `user.commission_rate_percent`
`0016`	`agency.onboarding_dismissed_at`
`0017`	Controlled-vocab columns (additive, nullable): `client.industry`, `opportunity.travel_purpose`, `booking.travel_purpose`/`trip_type`, `booking_traveller.title`/`gender`
`0018`	(see git log)
`0019`	Sprint 1: `booking_supplier_ref`, `booking_event`, `booking_document`, `booking_idempotency`; `booking_traveller.email` + `.phone`

API Integrations

Every external integration **degrades gracefully** to sample or logged behaviour when its keys are unset, so the app runs end-to-end with only `POSTGRES_URL` + `BETTER_AUTH_SECRET`. Env vars are summarized in development-guide.md.

Provider architecture

Booking providers plug into **one abstraction** so adding Hotelbeds, Amadeus, TravelgateX, Booking.com, or Expedia never touches calling code. Interfaces + registry live in `src/lib/suppliers/providers/`. Environment resolution lives in `src/lib/suppliers/config.ts` — the single entry point for credential/hostname config; no other file reads `process.env` for supplier credentials directly.

- **Capability-segmented interfaces** (`types.ts`) — a provider implements only what it offers: `HotelSearchCapable`, `HotelBookingCapable`, `FlightSearchCapable`, `FlightBookingCapable`, `CancelCapable`, `ContentCapable` (hotel enrichment / name-search), `AutocompleteCapable` (airport suggestions). A hotels-only aggregator implements the hotel interfaces and nothing else (no empty stubs).
- **Full booking lifecycle** — `search → quote (re-validate price/rate) → book (idempotent) → cancel`. Real bedbank/GDS rates expire, so `quoteHotel`/ `quoteFlight` re-price immediately before `book*`; book requests carry an `idempotencyKey` so replays never double-book.
- **Normalized errors** — every provider throws `ProviderError` with a stable `code` (`rate_expired`, `sold_out`, `rate_limited`, `provider_unavailable`, …) and a `retryable` flag, so callers branch on the code, not provider strings.
- **Tenant-aware context** — every call takes a `ProviderContext` (`agencyId`, currency, locale, correlationId, abort signal); providers never read globals.
- **Registry** (`registry.ts`) — adapters `register()` themselves at startup; callers resolve by vertical + capability + priority (`providerRegistry.pick("hotels", "search")`) with capability type-guards (`canSearchHotels`, `canBookFlights`, …). Replaces the hardcoded `getFlightSupplier()/getHotelSupplier()` if/else.
- **Registration** (`register.ts`, Wave 2) — `registerBuiltInProviders()` wires the three shipped adapters into the registry: `MockBookingProvider` (both verticals, priority 0, always configured — the fallback), `DuffelBookingProvider` (flights, priority 50) and `HotelbedsBookingProvider` (hotels, priority 50). It is idempotent (module-level guard) and runs as a side effect when the providers barrel is imported, so the registry is self-wiring. The mock outranks nothing, so `pick()` returns a real provider whenever one is configured and falls back to the mock otherwise.
- **Provider catalog** (`PROVIDER_CATALOG`) — one logic-free source of truth for which providers exist, their verticals/capabilities, env vars, and status (`live` / `legacy` / `planned`):

Provider	Verticals	Status
Mock	flights + hotels	live

Duffel	flights	live
Amadeus	flights (+ hotels)	legacy
Hotelbeds	hotels	live
TravelgateX	hotels + flights	planned
Booking.com	hotels	planned
Expedia (Rapid)	hotels	planned

Travel Platform sprint complete (Sprint 2): search, content, autocomplete, and booking are all routed through the registry. Business logic no longer calls provider-specific functions; adding a new provider is a `register()` call only.

- **Booking** (`booking-service.ts`) — full `quote → book → cancel` lifecycle via `providerRegistry.pick(...)`, idempotent, writes `booking_supplier_ref` / `booking_event` / `booking_idempotency`.
- **Search + content** (`src/lib/travel-platform/index.ts`) — single facade for `searchFlights`, `searchHotels` (availability, name-search, content fallback, thumbnail enrichment), `searchAirports`, `searchHotelDestinations`. Server actions delegate here; the `getFlightSupplier` / `getHotelSupplier` / `safeSearch` layer is no longer imported by consumer code (kept for backward compatibility only).
- **New capabilities registered:** `ContentCapable` (Hotelbeds: name-search, enrichment, room rates) and `AutocompleteCapable` (Duffel: airport suggestions).

Open item #1 (real supplier booking) is closed — swap in production credentials and both search and booking flows go live.

Flights — Duffel

- `src/lib/suppliers/duffel.ts`, behind `getFlightSupplier()` + `safeSearch()`.
- Airport autocomplete, one-way/round-trip, flight codes, connecting airports.
- Falls back to sample data when `DUFFEL_API_TOKEN` is unset.
- Amadeus self-service (`amadeus.ts`) is kept **only as a legacy fallback**.
- **Sprint 1:** `DuffelBookingProvider` implements `FlightSearchCapable` + `FlightBookingCapable` through the registry, adding `quoteFlight` (price revalidation) + `bookFlight` (idempotent order creation) to the existing search capability.

Hotels — Hotelbeds (APITUDE)

- `src/lib/suppliers/hotelbeds.ts` + `content-cache.ts`.
- Availability + content APIs. Booking.com-style search/results/details, dynamic occupancy pricing, filters, room photos, compare.
- **Content cache** (`hotel_content` table, global/non-tenant) serves real photos, facilities and coords quota-free; synced via `scripts/sync-hotel-content.ts`.
- **Search by hotel name** resolves matching hotels via the Content API

(`searchHotelbedsHotelsByName`) then prices them live; falls back to estimated rates.
- Keys: `HOTELBEDS_API_KEY` / `HOTELBEDS_SECRET`. Hostname: `HOTELBEDS_HOSTNAME` (defaults to test endpoint; set to `https://api.hotelbeds.com` for production). Falls back to sample data when keys are unset.
- **Sprint 1:** `HotelbedsBookingProvider` implements `HotelSearchCapable` + `HotelBookingCapable` through the registry, adding `quoteHotel` + `bookHotel`.

Travel Platform facade

`src/lib/travel-platform/index.ts` is the single entry point for all travel operations. Server actions import from here only; no supplier-specific code leaks into the action or page layer.

Key exports: `searchFlights`, `searchHotels`, `searchAirports`, `searchHotelDestinations`, `getActiveFlightProvider`, `getActiveHotelProvider`, `isFlightProviderConfigured`, `isHotelProviderConfigured`.

The hotel search path handles three cases internally:

1. Name search → `ContentCapable.searchHotelsByName`
2. Availability search → `HotelSearchCapable.searchHotels` + content enrichment via `ContentCapable.fetchHotelContent`
3. Content fallback (empty/degraded availability) → DB cache (`listHotelOffersCached`) → `ContentCapable.searchHotelsByName` with city as query

Supplier abstraction (legacy)

`src/lib/suppliers/index.ts` — `getFlightSupplier` / `getHotelSupplier` / `safeSearch` — is kept for backward compatibility. New code must use the Travel Platform facade instead. This is separate from the managed **supplier directory** (the `supplier` table — agency's own hotel/airline/DMC contacts; see business-rules.md).

Billing — Stripe subscriptions (vendor → agency)

- `src/lib/billing/stripe.ts`. Vendor bills agencies; 14-day trial on provision.
- Checkout, billing portal, and **manual webhook signature verification**.
- `api/stripe/webhook` reconciles subscription status; `requireAgencyUser` gates on a lapsed subscription.
- Keys: `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`.

Payments — Stripe Connect (traveler → agency)

- `src/lib/payments/stripe.ts` — **distinct** from billing/stripe.ts.
- Agencies onboard a connected Express account to receive traveler payments directly (**destination charges**); the platform takes a configurable fee.
- `api/stripe/connect-webhook` reconciles Connect payments. Powers the client portal "Pay

now".
- Keys: `STRIPE_CONNECT_WEBHOOK_SECRET`, `STRIPE_PLATFORM_FEE_PERCENT`.

Email — Resend

- `src/lib/notifications/email.ts` (adapter) + `templates.ts` (HTML).
- Sends invite emails, password-reset, proposal acceptance, portal invites.
- When unconfigured, logs to console + the `notification` table; invite/portal links are made copyable so the flow still works.
- Keys: `RESEND_API_KEY`, `EMAIL_FROM`.

AI — Vercel AI SDK + OpenRouter

- Powers the assistant and inline AI features; see ai.md.
- Key: `OPENROUTER_API_KEY`. AI features no-op/degrade when unset.

Storage — Vercel Blob (optional)

- Supplier contract PDF uploads. Key: `BLOB_READ_WRITE_TOKEN`.

Booking Architecture (Sprint 1)

Detailed technical reference for the provider abstraction system, booking service, idempotency model, and event log introduced in Sprint 1. For a high-level overview see api-integrations.md.

---

1. Overview

The booking architecture introduces a **three-layer** stack on top of the existing search-only supplier adapters:

```
Server Action (bookings.ts)
       │
       ▼
Booking Service (booking-service.ts)   ← orchestration
       │
       ▼
Provider Registry (registry.ts)        ← routing
       │
       ▼
Concrete Provider (duffel / hotelbeds / mock)
       │
       ▼
Supplier API (Duffel REST / Hotelbeds APITUDE)
```

The server action collects intent (which booking item to fulfil), builds a `ProviderContext`, and calls `serviceBookFlight` or `serviceBookHotel`. The booking service runs the full lifecycle — idempotency check, quote, book, event logging, supplier-ref persistence — without any booking-specific code leaking into the action layer.

---

2. Provider abstraction

2.1 Capability-segmented interfaces

`src/lib/suppliers/providers/types.ts`

A provider implements **only the capabilities it actually offers**. There are no empty stubs. The capability contract:

Interface	Methods	Who implements
`ProviderDescriptor`	`id`, `label`, `verticals`, `capabilities`, `priority`, `isConfigured()`	**every** provider

`HotelSearchCapable`	`searchHotels(params, ctx)`	Hotelbeds, Mock
`HotelBookingCapable`	`quoteHotel(offer, ctx)`, `bookHotel(req, ctx)`	Hotelbeds, Mock
`FlightSearchCapable`	`searchFlights(params, ctx)`	Duffel, Mock
`FlightBookingCapable`	`quoteFlight(offer, ctx)`, `bookFlight(req, ctx)`	Duffel, Mock
`CancelCapable`	`cancel(ref, ctx)`	(future)

A concrete provider is typed as:

```
type BookingProvider = ProviderDescriptor &
  Partial<HotelSearchCapable & HotelBookingCapable &
          FlightSearchCapable & FlightBookingCapable & CancelCapable>
```

Callers narrow capabilities using type-guard functions:

```
canSearchHotels(p)   // p is HotelSearchCapable
canBookHotels(p)     // p is HotelBookingCapable
canSearchFlights(p)  // p is FlightSearchCapable
canBookFlights(p)    // p is FlightBookingCapable
```

2.2 ProviderContext

Passed to every provider call so providers never read globals:

```
type ProviderContext = {
  agencyId:      string;          // tenant scope + per-agency cred lookup
  currency?:     string;          // ISO code, e.g. "DZD"
  locale?:       string;          // e.g. "en"
  correlationId?: string;         // traces a request across providers and logs
  signal?:       AbortSignal;     // timeout / user cancellation
}
```

2.3 Booking lifecycle DTOs

`RateQuote` — returned by `quoteHotel` / `quoteFlight`:

Field	Type	Purpose
`quoteId`	`string`	Opaque token passed to `book*`
`providerId`	`ProviderId`	Which provider issued the quote
`vertical`	`"flights" | "hotels"`	—
`priceTotal`	`number`	Price after re-validation

`currency`	`string`	—
`refundable`	`boolean`	—
`expiresAt`	`string` (ISO)	Quote expiry deadline
`cancellationDeadline?`	`string` (ISO)	Free-cancel deadline (when refundable)
`priceChanged?`	`boolean`	True when price differs from search result

`HotelBookingRequest` — sent to `bookHotel`:

Field	Type	Purpose
`offer`	`HotelOffer`	Original search offer
`guests`	`GuestDetails[]`	Guest roster (lead first)
`idempotencyKey`	`string`	Caller-generated; same key = no double-book
`quoteId?`	`string`	Fresh rate key from `quoteHotel`
`agencyReference?`	`string`	Cross-linking reference (e.g. `BKG-1001`)

`FlightBookingRequest` — sent to `bookFlight`:

Field	Type	Purpose
`offer`	`FlightOffer`	Original search offer
`passengers`	`FlightPassenger[]`	Passenger roster
`idempotencyKey`	`string`	Caller-generated; same key = no double-book
`quoteId?`	`string`	Fresh quote token
`agencyReference?`	`string`	Cross-linking reference

`BookingResult` — returned by `book*` on success:

Field	Type
`ref.providerId`	`ProviderId`
`ref.confirmationNumber`	`string`
`ref.raw?`	`unknown` (raw supplier response)
`status`	`"confirmed" | "pending"`
`priceTotal`	`number`
`currency`	`string`

2.4 Normalized error model

All providers throw `ProviderError` with a stable `code`:

Code	Meaning	Retryable
`auth`	Bad/expired credentials	false
`validation`	Malformed request	false
`rate_expired`	Quote/rate no longer valid → re-quote	true
`sold_out`	Availability gone	false
`rate_limited`	Provider throttled us → backoff	true
`provider_unavailable`	Upstream down/timeout	true
`not_supported`	Capability not offered by this provider	false
`unknown`	Unclassified	false

Callers branch on `code`, never on provider-specific error strings:

```
} catch (err) {
  if (err instanceof ProviderError && err.code === "rate_expired") {
    // re-quote then retry
  }
}
```

3. Provider registry

`src/lib/suppliers/providers/registry.ts`

The registry is a singleton. Providers self-register at startup and callers resolve by vertical + capability.

3.1 API

```
// Register (called by each provider at startup):
providerRegistry.register(provider: BookingProvider): void

// Resolve (returns highest-priority configured provider for vertical+capability):
providerRegistry.pick(vertical: ProviderVertical, capability: ProviderCapability): B

// Introspect:
providerRegistry.list(): BookingProvider[]
providerRegistry.all(vertical, capability): BookingProvider[]
```

`pick()` returns the highest-priority `isConfigured()` provider for the requested vertical + capability. If no configured provider exists, it returns `undefined` — the caller must handle that gracefully (the mock always fills this gap since it is always configured at priority 0).

3.2 Registration (register.ts)

`src/lib/suppliers/providers/register.ts` wires all three built-in adapters into the registry **as a**

module side effect — importing the providers barrel triggers registration automatically:

```
export function registerBuiltInProviders(): void {
  if (registered) return;          // module-level idempotency guard
  registered = true;
  providerRegistry.register(new MockBookingProvider());      // priority 0 — always
  providerRegistry.register(new DuffelBookingProvider());    // priority 50
  providerRegistry.register(new HotelbedsBookingProvider()); // priority 50
}
registerBuiltInProviders();        // runs on import
```

3.3 Provider catalog (PROVIDER_CATALOG)

Logic-free metadata record used to drive UI ("Available integrations") and future dynamic registration. The catalog is not the registry — it is a static description of what *could* be wired:

Provider	Verticals	Capabilities	Status	Env vars
`mock`	flights + hotels	search, quote, book	live	(none)
`duffel`	flights	search, quote, book	live	`DUFFEL_API_TOKEN`
`amadeus`	flights (+ hotels)	search	legacy	`AMADEUS_CLIENT_ID`, `AMADEUS_CLIENT_SECRET`
`hotelbeds`	hotels	search, quote, book	live	`HOTELBEDS_API_KEY`, `HOTELBEDS_SECRET`
`travelgatex`	hotels + flights	search	planned	—
`booking_com`	hotels	search	planned	—
`expedia`	hotels	search	planned	—

3.4 Priority resolution

When multiple providers are configured for the same vertical:

- **Highest priority wins** (higher number = higher priority).
- `MockBookingProvider` is always priority **0** — the unconditional fallback.
- `DuffelBookingProvider` and `HotelbedsBookingProvider` are priority **50**.
- Add a future aggregator at **100** to override the defaults.

4. Booking service

`src/lib/suppliers/booking-service.ts`

The orchestration layer. Server actions call `serviceBookFlight` or `serviceBookHotel` and receive a

`ServiceBookingResult`:

```
type ServiceBookingResult =
  | { confirmed: true;  confirmationNumber: string; providerId: string }
  | { confirmed: false; confirmationNumber: string; reason: string }
```

The booking service never throws. Failures are returned as `{ confirmed: false }` so the server action can write a provisional reference and surface a user-facing error without crashing the booking flow.

4.1 Flight booking flow (serviceBookFlight)

```
1. Derive idempotency key
   sha256("bookingId:bookingItemId:offerId")

2. Replay check
   IF booking_idempotency.status = "success" THEN return cached result (no supplier

3. Pick provider
   providerRegistry.pick("flights", "book") → DuffelBookingProvider (or mock)
   IF no provider → return { confirmed: false, reason: "No configured flight booking

4. Register idempotency key as "pending" (ON CONFLICT DO NOTHING)
   → prevents duplicate calls even if two requests race to the same booking

5. Quote (price revalidation) — non-fatal
   provider.quoteFlight(offer, ctx)
   → emit "price_validated" or "price_changed" event

6. Emit "booking_submitted" event

7. Book
   provider.bookFlight({ offer, passengers, idempotencyKey, agencyReference }, ctx)

8. On success:
   - INSERT booking_supplier_ref (confirmationNumber, pnr, rawPayload)
   - UPDATE booking_idempotency → status = "success", supplierRef = confirmationNumb
   - emit "booking_confirmed" event
   - return { confirmed: true, confirmationNumber, providerId }

9. On failure:
   - UPDATE booking_idempotency → status = "failed"
   - emit "booking_failed" event
   - return { confirmed: false, confirmationNumber: "REF-<timestamp>", reason }
```

4.2 Hotel booking flow (serviceBookHotel)

Same as 4.1 with two differences:

1. **offerId** is `offer.rateKey ?? offer.id` (Hotelbeds uses rate keys).
2. **Quote carries the fresh `rateKey`** — `quoteHotel` returns a new `quoteId` (the refreshed Hotelbeds `rateKey`), which is passed as `quoteId` in the `HotelBookingRequest`. This ensures

the book call always uses the freshest rate, never the one from the initial search.

4.3 Quote step — non-fatal by design

If `quoteFlight`/`quoteHotel` throws, the booking service logs a warning and proceeds to book. This prevents a quote-API timeout from blocking an entire booking attempt — the supplier's `book*` endpoint will reject a stale rate explicitly if it expired, surfacing as a `ProviderError("rate_expired")` from the book step itself.

For hotels, if the quote fails the `quoteId` is `undefined` and the book request uses the original `rateKey` from the search offer.

---

5. Idempotency model

5.1 Key derivation

```
sha256(`${bookingId}:${bookingItemId}:${offerId}`)
```

- `offerId` is the supplier's offer/rate identifier (e.g. Duffel offer id, Hotelbeds `rateKey`).
- The same triple always produces the same 64-hex-char key.
- Keys are written with `ON CONFLICT DO NOTHING`, so two concurrent requests for the same booking item cannot both insert a "pending" row.

5.2 Lifecycle states

State	Meaning
`pending`	Row inserted before the supplier call
`success`	Supplier confirmed; `supplierRef` is the confirmation number
`failed`	Supplier rejected; safe to retry with a fresh key (new offer/rate)

5.3 Replay semantics

- On success (`status = "success"`): return `supplierRef` immediately — no supplier call. This handles browser re-submits, network retries, and serverless double-invocations.
- On pending (`status = "pending"`): the prior call is still in flight (or crashed mid-flight). `ON CONFLICT DO NOTHING` means the new attempt skips the insert and proceeds to quote/book. The supplier's own idempotency key prevents double-booking at the supplier level.
- On failed (`status = "failed"`): the key represents a bad offer; the agent should select a fresh offer (which will have a different `offerId`) and book again with a new key.

5.4 TTL

Keys expire after **24 hours** (`expiresAt` column). A future cron job can clean up expired rows. Expired

rows do not block new bookings — expiry is advisory.

---

6. Event log (`booking_event`)

Append-only. Never update or delete rows.

6.1 Event types

Event	When	Metadata
`search_initiated`	Agent starts a search	`{ destination, checkIn, nights, guests }`
`offer_selected`	Agent picks an offer from results	`{ offerId, priceTotal, currency }`
`price_validated`	Quote returned same price	`{ priceTotal, currency }`
`price_changed`	Quote returned different price	`{ priceTotal, currency }`
`booking_submitted`	Just before calling `book*`	`{ offerId, priceChanged }`
`booking_confirmed`	`book*` returned success	`{ confirmationNumber, status, priceTotal, currency }`
`booking_failed`	`book*` threw	`{ reason }`
`booking_cancelled`	Cancellation succeeded	`{ penaltyAmount, currency }`
`payment_started`	Stripe Checkout session created	`{ sessionId, amount }`
`payment_completed`	Stripe webhook confirmed	`{ sessionId, amount }`

6.2 Dual purpose

- **Compliance audit trail** — every state transition is permanently recorded with timestamp, providerId, and correlationId. Never deleted (even if the booking itself is cancelled).
- **Analytics source** — booking funnel metrics (search→select, select→book, book→confirm, confirm→cancel) can be derived directly from this table without a third-party SDK.

6.3 Schema

Column	Type	Notes
`id`	`uuid`	PK
`bookingId`	`uuid`	FK → `booking`

agencyId	uuid	FK → agency (for efficient agency-scoped analytics)
event	text	Stable event code (see 6.1)
providerId	text?	"duffel", "hotelbeds", "mock", or null for UI events
correlationId	text?	Traces a request across logs
metadata	jsonb?	Structured payload (offer id, price, error, etc.)
createdAt	timestamp	Append timestamp

Indexes: bookingId, agencyId, event, createdAt, composite (bookingId, createdAt).

---

7. Supplier reference (booking_supplier_ref)

Structured record of the supplier's confirmation for a booking item. Replaces the ad-hoc JSONB in booking_item.details so references are **queryable** for status polling, cancellation, and voucher retrieval.

Column	Type	Notes
id	uuid	PK
bookingId	uuid	FK → booking
bookingItemId	uuid	FK → booking_item
providerId	text	"duffel", "hotelbeds", etc.
confirmationNumber	text	Shown to client
pnr	text?	Airline PNR / record locator
supplierOrderId	text?	Provider's internal order id
rawPayload	jsonb?	Full raw response for debugging + future re-parsing
createdAt	timestamp	—

Multiple refs per bookingItemId are possible (e.g. re-booking after a failure leaves both the failed attempt and the successful one).

---

8. Document store (booking_document)

Documents generated for a booking: vouchers, e-tickets, invoices, itineraries.

Column	Type	Notes
id	uuid	PK
bookingId	uuid	FK → booking

bookingItemId	uuid?	FK → booking_item (null = booking-level doc)
type	text	"voucher" | "ticket" | "invoice" | "itinerary" | "receipt"
providerId	text?	Supplier that issued the document
url	text?	Vercel Blob URL or supplier CDN link
rawData	jsonb?	Supplier's payload for re-generation
generatedAt	timestamp?	When the document was issued
createdAt	timestamp	—

9. Adding a new provider

1. **Create the provider class** in `src/lib/suppliers/providers/<name>-provider.ts`.
   - Implement `ProviderDescriptor` on every provider.
   - Implement only the capability interfaces the supplier supports.
   - Throw `ProviderError` (never native `Error`) on failure.
   - Return `true` from `isConfigured()` only when all required env vars are set.
2. **Export from the barrel** — add to `src/lib/suppliers/providers/index.ts`:

```
export * from "./<name>-provider";
```

3. **Register in `register.ts`**:

```
providerRegistry.register(new MyNewProvider());
```

4. **Add to PROVIDER_CATALOG** — one entry in `registry.ts` with `id`, `label`, `verticals`, `capabilities`, `envVars`, and `status: "live"`.
5. **Set priority** — choose a number relative to existing providers. If it should win over Duffel/Hotelbeds (priority 50), use `> 50`. Use `< 50` if it should only fill the gap when the primary provider is unconfigured.

No calling-code changes are needed — the registry and booking service automatically route to the new provider when it is configured.

10. Configuration and activation

All provider credentials are resolved in `src/lib/suppliers/config.ts`:

```
ATLAS_ENV=development   → mock only (default when unset)
ATLAS_ENV=sandbox       → provider sandbox endpoints
ATLAS_ENV=production    → live endpoints
```

Provider	Required env vars	Optional
Duffel (flights)	`DUFFEL_API_TOKEN`	—
Hotelbeds (hotels)	`HOTELBEDS_API_KEY`, `HOTELBEDS_SECRET`	`HOTELBEDS_HOSTNAME`

When `ATLAS_ENV=production` and the credentials are set, `pick()` returns the real provider at priority 50. The mock continues to serve as a fallback only when the real provider's `isConfigured()` returns `false`.

Server Actions Reference

All server-side mutations are Next.js Server Actions under `src/lib/actions/`. Every action:

1. Calls `requireAgencyUser()` to authenticate + get `(userId, agencyId)`.
2. Validates input with Zod (most actions; see gap tracker in roadmap.md).
3. Returns `ActionResult<T>` — `{ success: true; data: T }` or `{ success: false; error: string }`.

```
type ActionResult<T = void> =
  | { success: true;  data: T }
  | { success: false; error: string }
```

---

bookings.ts

The largest action file (≈1000 lines). Orchestrates the full booking lifecycle.

`createBooking(input)`

Creates a new booking file.

Input: `{ clientId, destination?, departDate?, returnDate?, currency?, notes? }`
Guards: client must belong to the agency (tenant isolation). **Side effects:** auto-generates a human reference (`BKG-NNN`).

`updateBooking(bookingId, input)`

Updates booking header fields (destination, dates, currency, notes, etc.).

Guards: booking must belong to the agency.

`deleteBooking(bookingId)`

Hard-deletes a booking and all its children.

Guards: `canDeleteRecords(role)` (admin/manager only). **Note:** soft delete is not yet implemented — see database.md gap tracker.

`advanceStatus(bookingId)`

Advances the booking to the next lifecycle state.

Lifecycle: `draft → awaiting_payment → confirmed → ticketed → completed → cancelled`

Prerequisites enforced:

- `confirmed`: trip items present AND zero outstanding balance.

- `ticketed`: trip items present.

Side effects:

- On `confirmed` or `ticketed`: `autoGenerateCommissions(bookingId, agencyId)` (idempotent).
- On `ticketed`: fires `confirmItemBooking` for all unconfirmed trip items (flights, hotels, transfers) → calls `serviceBookFlight` / `serviceBookHotel` via the provider registry.

`bookItem(bookingItemId)`

Attempts to confirm a single booking item with the supplier.

Side effects: calls `serviceBookFlight` or `serviceBookHotel` in `booking-service.ts`. Returns `{ confirmed: boolean; confirmationNumber: string; reason?: string }`.

`addTraveller(bookingId, input)` / `updateTraveller(travellerId, input)` / `deleteTraveller(travellerId)`

CRUD for `booking_traveller`. Input includes personal details (name, title, gender, passport, DOB, nationality) plus `email`/`phone` (required for the lead passenger by supplier APIs).

`addBookingItem(bookingId, input)` / `updateBookingItem(itemId, input)` / `deleteBookingItem(itemId)`

CRUD for `booking_item`. `type` must be a `BOOKING_ITEM_TYPES` code.

`updateItemOrder(bookingId, orderedIds)`

Reorders booking items by updating `sort_order`.

`assignDay(itemId, dayIndex)`

Assigns a `booking_item` to a specific itinerary day.

`updateBookingDay(bookingId, dayIndex, input)`

Upserts a `booking_day` title/notes row for a given day index.

`shareBooking(bookingId)` / `unshareBooking(bookingId)`

Generates or clears the `booking.shareToken` (itinerary public link `/i/[token]`).

`generateVoucher(bookingId)` / `generateInvoice(bookingId)`

Server-rendered PDF generation. Guards: booking must have trip services. Sends the PDF link via `notification` log.

`sendConfirmationEmail(bookingId)`

Sends a booking confirmation email to the client via Resend. Falls back to logging when Resend is unconfigured.

---

clients.ts

`createClient(input)`

Creates a client record. `type` defaults to `"individual"`. Validates country against the ISO reference list.

`updateClient(clientId, input)`

Updates client fields. Validates country, source, and industry codes.

`deleteClient(clientId)`

Hard-deletes the client and all descendant records. **Guards:** `canDeleteRecords(role)`.

`addContact(clientId, input)` / **`updateContact(contactId, input)`** / **`deleteContact(contactId)`**

CRUD for `client_contact`.

`convertToClient(clientId)`

Flips a lead (`status = "lead"`) to an active client.

---

opportunities.ts

`createOpportunity(input)`

Creates an opportunity. `stage` defaults to `"lead"`.

`updateOpportunity(opportunityId, input)`

Updates opportunity fields including stage, value, and travel details.

`deleteOpportunity(opportunityId)`

Guards: `canDeleteRecords(role)`.

`advanceOpportunityStage(opportunityId)`

Advances through: `lead → qualified → proposal → booked → won`. `won` is terminal; `lost` can be set via `updateOpportunity`.

products.ts (proposals)

`createProduct(input)`

Creates a proposal draft. Auto-generates `PRD-NNN` reference.

`updateProduct(productId, input)`

Updates proposal fields including markup and dates.

`deleteProduct(productId)`

Guards: `canDeleteRecords(role)`.

`addProductItem(productId, input)` / **`updateProductItem(itemId, input)`** / **`deleteProductItem(itemId)`**

CRUD for `product_item` line items.

`sendProposal(productId)`

Sets `status = "sent"` and generates a `shareToken` for the public link. Sends a notification email to the client.

`shareProposal(productId)` / **`unshareProposal(productId)`**

Generates or clears the `product.shareToken`.

`convertProposalToBooking(productId)`

One-click conversion: creates a `booking` from an accepted `product`, copying client, dates, and line items. Flips the linked opportunity to `"won"`.

---

proposals-public.ts

Actions callable from the **public proposal page** (`/p/[token]`) — no auth.

`acceptProposal(token, input)`

- Validates token; finds product.
- Sets `acceptedAt`, stamps signer name/email/IP/UA and `signatureData`.
- If `opportunityId` is linked, advances opportunity to `"won"`.

`declineProposal(token)`

Sets `declinedAt`.

payments.ts

`createPayment(bookingId, input)`

Records a payment against a booking. `kind` must be `PAYMENT_KINDS` code. Re-calculates outstanding balance.

`deletePayment(paymentId)`

Guards: `canManagePayments(role)`.

`createStripeCheckout(bookingId, amount)`

Creates a Stripe Checkout Session (Connect: destination charge to agency). Returns `{ checkoutUrl }`.

commissions.ts

`autoGenerateCommissions(bookingId, agencyId)`

Idempotent — safe to call multiple times. Generates two commission rows per bookable item: `supplier_to_agency` (from the supplier contract) and `agency_to_agent` (from `user.commissionRatePercent`). Called automatically by `advanceStatus` on `confirmed` / `ticketed`.

`updateCommission(commissionId, input)`

Adjusts `status` (`confirmed → paid`, etc.) or `amount` / `notes`.

`deleteCommission(commissionId)`

Guards: `canViewFinance(role)`.

suppliers.ts

`createSupplier(input)` / **`updateSupplier(supplierId, input)`** / **`deleteSupplier(supplierId)`**

CRUD for the managed supplier directory.

`createSupplierContract(supplierId, input)` / **`updateSupplierContract(contractId, input)`** / **`deleteSupplierContract(contractId)`**

CRUD for supplier contracts.

`createSupplierRate(contractId, input)` / `updateSupplierRate(rateId, input)` / `deleteSupplierRate(rateId)`

CRUD for per-product rates within a contract.

`uploadContractFile(contractId, formData)`

Uploads a PDF contract to Vercel Blob and saves the URL.

---

notifications.ts

`sendNotification(bookingId, input)`

Sends an email/SMS/WhatsApp/push notification to a recipient. Logs to `notification` table regardless of delivery status.

---

invites.ts

`inviteTeamMember(input)`

Creates an `agency_invite` row with a signed, expiring token. Sends an invitation email via Resend (or logs + returns the link when unconfigured).

`revokeInvite(inviteId)`

Marks the invite `status = "expired"`.

---

team.ts

`updateTeamMember(userId, input)`

Updates `role`, `active`, `commissionRatePercent`. Role changes are gated: only an admin can assign/change the `admin` role (`canAssignAdmin(role)`).

`deactivateTeamMember(userId)`

Sets `active = false`.

---

settings.ts

`updateAgencySettings(input)`

Updates agency `name`, `slug`.

`updateUserProfile(input)`

Updates the current user's `name`, `locale`.

---

onboarding.ts

`dismissOnboarding()`

Sets `agency.onboardingDismissedAt` to close the getting-started card.

---

search.ts

`searchAirportsAction(query)`

Routes through the Travel Platform facade (`src/lib/travel-platform`). Uses the registry's `AutocompleteCapable` provider (Duffel when configured); falls back to a built-in airport list in dev.

`searchHotelDestinationsAction(query)`

Filters the static `HOTEL_DESTINATIONS` list (no provider call).

`searchFlightsAction(params)`

Delegates to `searchFlights(params, ctx)` from the Travel Platform facade. Returns `{ ok, results, source, degraded }`. Degrades to mock on provider error.

`searchHotelsAction(params)`

Delegates to `searchHotels(params, ctx)` from the Travel Platform facade, which handles three paths internally: name-search (ContentCapable), availability search (HotelSearchCapable + thumbnail enrichment), and content fallback (DB cache → ContentCapable with city query). Returns `{ ok, results, source, degraded, estimatedPricing }`.

`getHotelDetailsAction(code)`

Loads rich content (photos, description, address) for one hotel via `getHotelContentCached` — DB content cache, quota-free.

`searchHotelRoomsAction(params)`

Re-prices every room of one hotel for a specific occupancy + date range. Calls `getHotelbedsRates` when Hotelbeds is configured; falls back to `mockHotelRates`. Returns `{ ok, rooms: HotelRoomRate[], degraded }`.

ai.ts

`generateItinerary(bookingId, prompt)`

Calls OpenRouter (streamed) to generate a day-by-day itinerary. Returns a `ReadableStream` — consumed by the assistant UI.

`generateQuote(opportunityId, prompt)`

AI-assisted quote: generates product items + summary text.

`draftEmail(context, prompt)`

Drafts a client communication email. Output is shown in a preview step before sending.

`visaAssistant(destination, nationality)`

Returns visa requirement guidance. Advisory only — never authoritative.

---

billing.ts

`createCheckoutSession()`

Starts a Stripe subscription Checkout for an agency. Redirect to Stripe.

`createBillingPortalSession()`

Opens the Stripe Billing Portal for subscription management.

---

platform.ts

Vendor-only (requires `is_platform_admin = true`).

`listAgencies()` / **`getAgencyStats(agencyId)`**

Vendor console: list all agencies with billing status and usage stats.

`suspendAgency(agencyId)` / **`activateAgency(agencyId)`**

Toggle `agency.status` (`"suspended" \| "active"`). Suspended agencies are locked out via `requireAgencyUser`.

---

portal-*.ts

Actions for the **client portal** (`/portal/...`). Authenticated via `portal_session.token` in an httpOnly cookie — entirely separate from Better Auth.

`portal-invite.ts`

`sendPortalInvite(clientId)`

Generates a magic-link token (`portal_session`), emails it to the client, and returns the URL (so the agent can copy it when email is unconfigured).

`portal-payments.ts`

`createPortalCheckout(bookingId, amount)`

Stripe Connect destination charge — same backend as `payments.ts`, but verifies the caller is the scoped client (via `portal_session.clientId`).

`portal-proposals.ts`

`portalAcceptProposal(productId, input)` / **`portalDeclineProposal(productId)`**

In-portal e-sign flow. Same logic as `proposals-public.ts` but verifies the caller is the client the proposal belongs to.

Development Guide

Setup

```
npm install --legacy-peer-deps    # better-auth peer range
npm run dev                       # http://localhost:3000
npm run check                     # lint + typecheck
npm run build:ci                  # next build
```

After completing any change, run `npm run check` and `npm run build:ci` to verify code quality (lint, typecheck, build). See coding-standards.md.

Environment (`.env`)

Required: `POSTGRES_URL`, `BETTER_AUTH_SECRET`.

Optional (all integrations degrade gracefully to sample/logged behaviour when unset — see api-integrations.md):

- `OPENROUTER_API_KEY` — AI features
- `BLOB_READ_WRITE_TOKEN` — uploads
- `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`
- `NEXT_PUBLIC_APP_URL`
- `RESEND_API_KEY` / `EMAIL_FROM` — email
- `STRIPE_SECRET_KEY` / `STRIPE_PRICE_ID` / `STRIPE_WEBHOOK_SECRET` — billing
- `STRIPE_CONNECT_WEBHOOK_SECRET` / `STRIPE_PLATFORM_FEE_PERCENT` — Connect payments
- `DUFFEL_API_TOKEN` — flights
- `HOTELBEDS_API_KEY` / `HOTELBEDS_SECRET` — hotels
- `PROTECTED_DB_HOSTS` — makes destructive scripts refuse a prod DB (override with `ALLOW_PROD=1`). See security.md.

Database

- Branches: dev `ep-wandering-sunset-aitlty78` · prod `ep-misty-thunder-aixz34vy`.
- After schema changes: `db:generate` → `db:migrate` (**never** `db:push`). `db:studio` to browse. See database.md.
- **Always run migrations on prod after deploy:** `POSTGRES_URL=<prod-url> npx drizzle-kit migrate`.

Maintenance scripts

```
# Promote an existing account to the platform super-admin
npx tsx --env-file=.env scripts/make-platform-admin.ts <email>
```

```
# Seed / reset the Demo Agency (idempotent — wipes its data, keeps users, reseeds)
npx tsx --env-file=.env scripts/seed-demo-data.ts

# Cross-tenant isolation test (seeds 2 agencies, asserts no leak, cleans up)
npx tsx --env-file=.env scripts/test-tenant-isolation.ts

# Normalize legacy free-text country values to canonical ISO names
npx tsx --env-file=.env scripts/backfill-countries.ts

# Sync Hotelbeds hotel content (photos/facilities/coords) into the cache table
npx tsx --env-file=.env scripts/sync-hotel-content.ts            # curated destinati
npx tsx --env-file=.env scripts/sync-hotel-content.ts BCN MAD    # specific codes
```

Testing

Use the project's testing tools/scripts (e.g. `test-tenant-isolation.ts`) to verify changes — never assume changes work. If no tooling covers a change, ask whether testing should be skipped. See coding-standards.md.

Coding Standards

> **Source of truth:** `AGENTS.md` (aliased by `CLAUDE.md`) holds the enforced working conventions. This doc summarizes and points there — it does not restate or override it.

Working conventions (from AGENTS.md)

- **Responses** — keep concise and to the point unless asked otherwise.
- **Planning** — always ask clarifying questions; never assume design, stack or features; use deep-dive sub-agents for research and to review plans.
- **Change/edit mode** — prefer sub-agents to implement features; act as a coordinator; parallelize independent work; use premium models for complex tasks (coding) and mid-tier for simpler ones (documentation).
- **Quality gates** — after completing features, run lint, type check and `next build` (`npm run check`, `npm run build:ci`). See development-guide.md.
- **Testing** — use available testing tools; never assume changes work; if no tooling exists for a change, ask whether to skip.

Engineering rules

Hard rules for how code is written. Status reflects the current codebase (✓ holds · 🟡 partial).

Rule	Status	Notes
Never duplicate logic	🟡	Shared helpers in `src/lib` (`queries`, `analytics`, `domain`); enforce by review.
Always use server actions	✓	20/21 files in `src/lib/actions` are `"use server"`; mutations go through them.
Always validate input	🟡	Validated where it matters, not universally.
Always use Zod	🟡	Zod v4 present, used in ~13 files; not every action yet.
No business logic inside components	🟡	Logic lives in actions/lib; enforce by review.
No direct SQL in UI	✓	Pages call Drizzle queries server-side / via actions, not raw SQL in client components.
Every action logs activity	🟡	`logActivity` in ~11/21 action files — not yet every mutation (see security.md).
Every mutation is tenant scoped	✓	All actions go through `requireAgencyUser()` → `agencyId` scope (see business-rules.md).

> The 🟡 rows (universal Zod validation, universal activity logging) are in the spec-vs-reality gap tracker.

Database changes

- After any schema change, run **drizzle generate + migrate**. **Never** `db:push`. Workflow detail in database.md.
- All ID columns **not** related to Better Auth use **UUIDs**, randomly generated.

UI

- Follow the design system in `DESIGN.md` when creating or reviewing components/pages. See ui-ux.md.

UI / UX

> **Source of truth:** `DESIGN.md` defines the full visual design system — colors (oklch tokens), typography, spacing, radius, shadows, animations, layout patterns, shadcn/ui component conventions, and dark mode. All new components and pages **must** follow it. This doc points there and captures product-level UX patterns only; it does not duplicate the token tables.

Atlas design principles

The UI-level expression of the ten founding Atlas principles. The product guardrails every page and feature is measured against:

1. **Never ask the user twice for the same information.** Reuse what's already captured (controlled vocabularies, reference data, client records) instead of re-prompting.
2. **Everything starts from the client.** The client record is the spine — leads, opportunities, proposals, bookings and payments all hang off it.
3. **Every action must be reversible.** Prefer soft states, confirmations and undo over destructive, irreversible operations.
4. **One source of truth.** No duplicated or conflicting data; canonical values (ISO countries, enums, references unique per agency) over free text.
5. **Automation before manual work.** Default to generating (commissions, itineraries, quotes, documents) rather than asking the user to do it by hand.
6. **Every page should answer "What should I do next?"** Surface the next action — stepper advance, CTA, empty-state prompt — never leave a dead end.
7. **Managers want insights; agents want speed.** Tailor each role's surface — analytics-dense for managers, fast and scoped for agents.
8. **No empty pages.** Every list/chart/detail has an empty state with a clear next step, never blank space.
9. **Data quality is more important than flexibility.** Constrain inputs (dropdowns, pickers, validation) to keep reporting clean.
10. **Everything should be mobile friendly.** Layouts work on small screens and in both LTR and RTL.

These principles inform the patterns below and the business rules. See also the product vision.

Design system summary

- **Stack:** Next.js + Tailwind v4 (CSS-first `@theme inline`, no config file), shadcn/ui (new-york, neutral), Lucide icons, Geist fonts, next-themes dark mode.
- **Tokens:** semantic oklch color tokens, `--radius` 10px base, Tailwind-default shadows, custom `fade-in`/`fade-up`/`scale-in` animations. Use `cn()` for class merging.
- See `DESIGN.md` for every value.

Performance budgets

Target response times — what "fast enough" means per surface. These back design principle #6 (agents want speed) and dictate where to lean on caching, streaming, and loading states.

Surface	Budget
Dashboard	< 1 s
Search (in-app lookup)	< 500 ms
Booking creation	< 2 s
Hotel search (live Hotelbeds)	< 5 s
Reports / export	< 10 s

> Budgets are **targets, not yet measured** — there's no perf instrumentation in place. The slow surfaces (hotel search, reports) are I/O-bound on external APIs and the export workbook; the content cache already keeps hotel photos quota- and latency-free (api-integrations.md). Every surface over its budget must show a loading state.

Data table standard

The target capabilities for every data table (clients, bookings, opportunities, proposals, suppliers, commissions). These belong in **one reusable `DataTable` component**, not re-implemented per page:

- **Sorting** — click a column header to sort; multi-column where useful.
- **Filtering** — per-column filters plus the page-level search/filter bar.
- **Column chooser** — show/hide columns; remember the user's choice.
- **Export** — export the current view (respecting sort/filter) to CSV/Excel via the existing `/api/export` endpoint.
- **Infinite scrolling** — load more on scroll instead of hard `limit` truncation (fixes the silent-cap issue in the page audit).
- **Sticky header** — header stays visible while the body scrolls.
- **Keyboard shortcuts** — arrow-key row navigation, Enter to open, `/` to focus search.
- **Context menu** — right-click a row for quick actions (open, edit, duplicate, delete — gated by permissions).

> **Current state:** tables today are plain shadcn `Table` markup with none of the above — no sorting, column chooser, infinite scroll, sticky header, shortcuts or context menu, and a hardcoded row `limit`. This standard is the target for a shared `DataTable`; building it also clears most of the systemic gaps in the page audit (export, pagination, bulk actions).

Page requirements checklist

Every list/index page must ship with all of these — a page missing any of them is incomplete, not "v1":

- [ ] **Search** — free-text lookup over the page's records
- [ ] **Filters** — by the page's key dimensions (status, type, date range, owner)
- [ ] **Export** — CSV/Excel of the current view (see analytics.md)
- [ ] **Bulk actions** — multi-select + an action that applies to the selection
- [ ] **Empty state** — a prompt with a clear next step (never blank — principle #8)
- [ ] **Loading state** — skeleton/spinner while data resolves
- [ ] **Error state** — a recoverable message, not a crash
- [ ] **Pagination** — bounded result sets, never an unbounded dump
- [ ] **Mobile layout** — works on small screens, LTR and RTL (principle #10)
- [ ] **Permissions** — role-scoped data + role-gated actions (see business-rules.md)

This operationalizes the design principles at the page level. Detail pages additionally answer "what should I do next?" via a primary CTA or lifecycle action.

Product UX patterns

These are app-specific patterns layered on the design system:

- **Getting-started checklist** — dismissible 4-step card on the dashboard for new agencies; dismissed state persists in DB (`agency.onboardingDismissedAt`) across devices and team members.
- **Lifecycle stepper** — horizontal progress bar on booking detail with an "Advance to [next]" button and soft/hard prerequisite guards (see business-rules.md).
- **Role-gated nav** — locked nav items shown dimmed with a tooltip for non-admin roles; nav is filtered by a `show(role)` predicate.
- **List/Board toggle** — Bookings and Pipeline share data with a table↔kanban switch.
- **Inline search sheet** — "Search flights/hotels" on the trip-services panel opens a sheet pre-scoped to the booking's destination.
- **Consolidated sharing** — proposal sharing lives under one "Share with client ▾" dropdown.
- **Empty states** — charts and lists show empty-state badges/CTAs rather than blank space.

RTL & i18n

Arabic is RTL (IBM Plex Sans Arabic, `html[dir="rtl"]`). Layout must work in both directions. i18n plumbing is in architecture.md.

Business Rules

Domain logic and constraints. Enums and capability helpers are defined in `src/lib/domain.ts`; server-side enforcement lives in the actions under `src/lib/actions/`.

Automation triggers

The event → action automation layer Atlas is targeting (design principle #5, *automation before manual work*). Each is a domain event that should fire a follow-up action:

Event	Action	Current state
Client created	Send welcome email	✕ not implemented (Resend only sends invites/reset/proposal/portal)
Opportunity won	Generate proposal	✕ manual (new proposal + AI quote builder)
Proposal accepted	Generate booking	⚠ one-click *convert proposal → booking*, not automatic
Booking confirmed	Generate invoice	✕ invoices are on-demand PDFs, not auto-generated
Payment received / booking confirmed	Update accounting	⚠ partial — `autoGenerateCommissions` fires on confirm/ticket (bookings.ts); no full accounting
Travel completed	Request review	✕ not implemented

> **Current state:** the only true event-driven automation today is commission generation on booking confirm/ticket (idempotent). The rest are manual or on-demand. Building this trigger layer is the heart of the "Automation" pillar in the vision and the automated-quotation item in the roadmap.

Never rules (hard constraints)

Invariants Atlas must never violate. Each maps to an enforcement point — a violation is a bug, not a preference.

Never	Why / where enforced
Never use free text when a dropdown exists	Controlled vocabularies + reference pickers keep reporting clean (data quality > flexibility). See vocabularies.
Never duplicate client information	One source of truth — the client record is the spine; reuse it, don't re-enter it.
Never create a booking without a client	Every booking hangs off a client; the spine is mandatory.

Never hard delete data	Reversibility — prefer soft state / archival + the activity log; deletes are gated to admin/manager only.
Never expose another tenant	Every read/write is scoped by `agencyId` via `requireAgencyUser()`; verified by `test-tenant-isolation.ts`. See security.md.
Never sum different currencies	No FX — group by currency with a DZD headline. See Currency and analytics.md.
Never make users navigate five pages	Surface work where it happens — inline search sheets, detail-page panels, "what next?" CTAs.
Never require more than three clicks	Key actions (advance lifecycle, convert proposal→booking, share, pay) are one-to-three clicks.

These are the enforcement edge of the design principles.

Golden workflow

The canonical end-to-end happy path Atlas is built around. Everything starts from the client (design principle #2) and ends by feeding the next trip — a loop, not a line.

```
Lead → Client → Opportunity → Proposal → Customer accepts → Booking →
Supplier reservation → Payment → Ticketing → Travel → Feedback → Repeat client
```

Step	In Atlas	Notes
Lead	`client` with a `source` (lead-source code)	captured as an early-stage client
Client	`client` (+ contacts)	the spine; all records hang off it
Opportunity	`opportunity` (pipeline stage, value, `travel_purpose`)	shown on the Pipeline board
Proposal	`product` (PDF + e-sign)	shareable `/p/[token]` or in-portal
Customer accepts	e-sign stamps signer + flips opportunity to **won**	also acceptable in the client portal
Booking	`booking`, one-click **convert proposal → booking**	lifecycle starts at `draft`
Supplier reservation	`booking_item` + `supplier` picker + `booking_service.ts`	live Duffel/Hotelbeds search; real booking wired via provider registry (quote → book → idempotency → event log); activate with production credentials
Payment	`payment` (deposit/installments, Stripe Connect)	`confirmed` requires zero balance
Ticketing	lifecycle `ticketed`	requires trip items; auto-generates commissions
Travel	lifecycle `completed`; itinerary `/i/[token]`	day-by-day timeline, vouchers/invoices

Feedback	activity log / notes on the client timeline	closes the loop
Repeat client	back to **Opportunity** on the same client	retention, not re-acquisition

This maps onto the booking lifecycle below and the role landings in architecture.md.

Roles & capabilities

Five roles: **admin, manager, finance, support, agent**. The capability matrix is in architecture.md. Key rules:

- `seesAllData` — all roles except **agent** (agents see only their own records).
- `canManageTeam`, `canManagePayments` — admin/manager.
- `canViewFinance` — admin/manager/finance (gates `/finance`, `/commissions`, `/reports`).
- `canViewSupport` — admin/manager/support.
- `canDeleteRecords` — admin/manager.
- `canAssignAdmin` — **only an admin** can assign or change the admin role.

Onboarding & access

- **Invitation-only signup** — enforced at the auth layer (`user.create.before` hook). No invite ⇒ no account. See security.md.
- **Agency suspension** — a suspended agency locks out its users via `requireAgencyUser`.
- **Lapsed subscription** — gates access via `requireAgencyUser` (14-day trial on provision). See api-integrations.md.

Currency

- Supported: **DZD (default)**, EUR, USD.
- **No FX conversion.** The agency operates in DZD; Atlas never sums across currencies. Analytics group **by currency** with a DZD headline. See analytics.md.

References

- `BKG-…` (bookings) and `PRD-…` (proposals) reference numbers are unique **per agency**, not globally.

Booking lifecycle

States: `draft → awaiting_payment → confirmed → ticketed → completed`. A visual stepper drives it with an "Advance to [next]" button. **Hard prerequisites are enforced server-side:**

- `confirmed` requires trip items **and** zero outstanding balance.

- `ticketed` requires trip items.
- Vouchers/invoices are **blocked** when a booking has no trip services.

Proposals & e-signature

- Server-rendered PDF; shareable via public tokenized `/p/[token]` (no login) and in-portal signing.
- E-sign stamps signer name/email/IP/UA and flips the linked opportunity to **won**.
- **Convert accepted proposal → booking** in one click. (A `convertedBookingId` guard column is a pending open item — see roadmap.md.)

Commissions

- **Two ledgers:** `supplier_to_agency` (agency earns from a supplier per booking item) and `agency_to_agent` (agent earns from the agency per booking).
- **Auto-generated** when a booking is confirmed or ticketed; **idempotent**.
- Agent rate from `user.commissionRatePercent`; supplier basis from the supplier contract (percent / fixed / net).

Controlled vocabularies

Seven fields are enum dropdowns (codes stored, labels shown) for clean reporting: lead source, travel purpose, trip type, gender, title, lost reason, industry. Country/nationality use canonical ISO 3166-1 values (full name stored) so spellings never drift; city uses curated autocomplete suggestions.

Client portal

- Magic-link passwordless sessions scoped to **one client**, separate from staff auth (`portal_session`).
- Agents send the invite from a client or booking detail page; the link is copyable when email is unconfigured.
- Online "Pay now" appears only when the agency has onboarded Stripe Connect.
- Clients can accept/decline proposals in-portal with the same e-sign audit as the public flow.

Deployment

Vercel

- Project `atlasproject/agence_tool`, connected to GitHub.
- **Auto-deploy:** `git push origin main` → builds & ships production.
- **Manual:** `npx vercel deploy --prod --yes`.
- `vercel.json` → `buildCommand: npm run db:migrate && npm run build:ci`; `.npmrc` → `legacy-peer-deps=true`.
- Migrations run automatically on every deploy — no manual step needed for schema changes.

Vercel environment

`POSTGRES_URL`, `BETTER_AUTH_SECRET`, `NEXT_PUBLIC_APP_URL`, `BETTER_AUTH_URL`, `PROTECTED_DB_HOSTS=ep-misty-thunder-aixz34vy` (plus integration keys as needed — see api-integrations.md).

Migrations on prod

Migrations now run automatically as part of every Vercel deploy (`npm run db:migrate` is prepended to the build command). No manual step required.

For one-off manual runs (e.g. emergency hotfix outside a deploy):

```
POSTGRES_URL=<prod-url> npx drizzle-kit migrate
```

Branches: dev `ep-dawn-voice-ai8d6q3o` · prod `ep-misty-thunder-aixz34vy`. `PROTECTED_DB_HOSTS` makes destructive scripts refuse the prod DB unless `ALLOW_PROD=1`. See security.md and database.md.

Demo accounts

Throwaway accounts on the **Demo Agency**, all on the live site.

Role	Email	Password
Platform admin (vendor)	`ouksili.abdelmalek@gmail.com`	`Atlas!2026`
Manager	`yasmine@agence.test`	`Agency!2026`
Finance	`finance@demo.test`	`Finance!2026`
Support	`support@demo.test`	`Support!2026`
Agent	`karim@demo.test`	`Agent!2026`

Agent	`lina@demo.test`	`Agent!2026`
Agent	`omar@demo.test`	`Agent!2026`

> 🔒 **DEMO CREDENTIALS ONLY** — rotate or delete before any real production use. These live passwords are checked into the repo; treat as a known risk (see security.md).

Suggested demo flow: sign in as vendor → `/platform` → View as Demo Agency → manager dashboard → switch to finance/support/agent views → Settings → switch to Français or العربية.

Roadmap & Changelog

Phase status

Phase 1 — Sellable ✓ **COMPLETE** Email (Resend) · Stripe SaaS billing · Separate Neon DB branches (dev/prod) · PDF proposals · E-signature · Live flights (Duffel) · Live hotels (Hotelbeds + content cache).

Phase 2 — Competitive ✓ **COMPLETE** Supplier management (`/suppliers`, contracts, rates, picker) · Commissions tracking (auto-generated, two-ledger, `/commissions`) · Client portal (magic-link login, trip view, online payments via Stripe Connect, in-portal proposal signing + portal invite from agent).

Phase 3 — AI differentiation ✓ **COMPLETE** AI itinerary generation · AI quote builder · AI email drafting · AI visa assistant — all inline, embedded where the work happens. See ai.md.

UX overhaul ✓ **COMPLETE** (20 changes across 4 size categories) Getting-started checklist · Lifecycle stepper · Board view toggle · Inline search · Portal invite · Convert proposal→booking · Client funnel timeline · Booking hard guards · Vocabulary pass · Role nav tooltips · Empty-state badges.

Data quality & BI ✓ **COMPLETE** (4 phases) DZD-only currency (no FX) · **P1** richer analytics (revenue/conversion/pipeline funnel/AR aging/margin) · **P2** controlled-vocabulary enums (7 fields, codes+labels) · **P3** ISO country/nationality reference data + city suggestions · **P4** standardized CSV/Excel export (`/reports`, dual code+label columns). Hotel search-by-name added. See analytics.md.

Module maturity

Maturity levels: **Production** (stable, in daily use) · **Beta** (works, hardening) · **Alpha** (functional, evolving) · **Planned** (not built).

Module	Status
CRM	Production
Sales	Production
Hotels	Beta
Flights	Beta
Accounting	Planned
AI	Alpha
Supplier Portal	Planned

> **Flights/Hotels nuance:** live *search* is shipped for both (Duffel + Hotelbeds); the maturity above reflects the end-to-end **booking** flow — Hotels is further along (Beta), real Flights booking is Planned (Duffel orders). See api-integrations.md and Open item #1.

Open items

1. **Real supplier booking** — ✓ architecture complete (Sprint 1 Waves 1–3). Provider registry, quote→book→cancel lifecycle, idempotency, event log, and supplier-ref tables are all wired. Activate by setting `DUFFEL_API_TOKEN` (flights) and `HOTELBEDS_API_KEY`/`HOTELBEDS_SECRET` + `ATLAS_ENV=production` (hotels).
2. **Translate deeper pages** — bookings, clients, finance, support, platform (i18n plumbing ready).
3. **Cross-device locale** — sync `user.locale` → cookie on login.
4. **WhatsApp integration** — Meta Cloud API adapter (skeleton ready; Meta Business account needed).
5. **Automated quotations** — trigger quote generation from opportunity stage change.
6. **Agent performance scoring** — leaderboard from commission + booking counts.
7. **Convert proposal to booking guard** — add `convertedBookingId` column (needs migration).

Planned modules

The longer-horizon module backlog, grouped. Some already have partial foundations (noted) — the rest are net-new.

Accounting

- Accounting · General Ledger · Payroll — net-new finance suite.
- Invoices — ⚠ on-demand invoice PDFs exist today; this is the move to managed, numbered, ledgered invoices.

Marketing

- Marketing · Email campaigns — net-new (transactional email via Resend exists; campaigns do not).
- WhatsApp — ⚠ adapter skeleton ready (open item #4); needs a Meta Business account.
- SMS — net-new channel adapter.

Documents

- Document management — net-new (today: PDF generation + Vercel Blob uploads for supplier contracts).

Travel products

- eVisa · Visa tracking — ⚠ an AI visa assistant exists; these add structured application + tracking.
- Insurance — ⚠ exists as a trip-service item type; this is a managed insurance product/flow.
- Vehicle rental · Cruises — net-new verticals (alongside flights/hotels).
- Package builder — net-new (bundle multi-service trips).

Ecosystem

- Supplier portal — net-new (suppliers self-serve, parallel to the client portal).
- Customer loyalty · Affiliate program — net-new growth modules.

- API marketplace — net-new (expose/consume third-party integrations).

> These are directional, not scheduled. Phase status and the active gap list are above; this is the "what's next after the core is hardened" backlog.

Spec vs. reality gap tracker

The design system (principles, never-rules, page/table/entity standards, AI guardrails, automation triggers, perf budgets) describes a **target state**. This table consolidates every item where the spec is ahead of the code, so the gaps are one punch list instead of scattered "current state" notes.

Status: ✓ done · ◍ partial · ◍ not started

Item	Spec	Status	Notes
Agent visibility scoped to own records on list pages	security	✓	Fixed — clients/bookings/opportunities/products now scope by owner column (commit `134ceb3`).
Quality gate `npm run check` works (pnpm v11)	development-guide	✓	Fixed — config moved to `pnpm-workspace.yaml`; 0 lint/type errors (commit `7d5f4e1`).
Soft delete (`deletedAt`, filtered reads)	database	◍	No soft-delete column anywhere; "never hard delete" is currently aspirational. Needs migration.
`reference` on all entities	database	◍	Only `booking` + `product`; missing on clients/opportunities/suppliers. Needs migration.
Consistent `updatedAt`/`status`/`notes` on children	database	◍	Present on roots, inconsistent on child tables.
Page systemic-five: per-page **Export**	ui-ux	◍	Endpoint exists (`/api/export`); list pages lack buttons.
Page systemic-five: **Bulk actions**	ui-ux	◍	No `Checkbox` primitive exists; nothing built.
Page systemic-five: **Loading** states	ui-ux	◍	Only `assistant`/`dashboard` have `loading.tsx`. `skeleton.tsx` exists, unused.
Page systemic-five: **Error** states	ui-ux	◍	Only `assistant` has `error.tsx`.
Page systemic-five: **Pagination**	ui-ux	◍	Hardcoded `limit` (200/500) silently truncates.
Shared **DataTable** (sort, filter, column chooser, infinite scroll, sticky header, shortcuts, context menu)	ui-ux	◍	Tables are plain shadcn markup. Building this clears most of the systemic-five.

Mobile-friendly tables (`overflow-x-auto`)	ui-ux	🟡	Tables overflow on phones; only `assistant` wraps for scroll.
Automation triggers (welcome email, won→proposal, accepted→booking, confirmed→invoice, completed→review)	business-rules	🟢	Only commission auto-gen on confirm/ticket fires; rest manual/on-demand.
AI mutation behind a hard confirm step	ai	🟡	`createBooking` is intent-gated by prompt only, not an enforced confirm gate.
Performance instrumentation vs budgets	ui-ux	🟡	Budgets defined; nothing measured.
Real supplier booking (Duffel orders, Hotelbeds book)	api-integrations	✓	Architecture complete (Sprint 1 + 2). Activate with production credentials (see open item #1).
Cross-device locale sync	architecture	🟡	English until re-pick on a fresh device (also open item #3 above).
Rate limiting (auth + API)	security	🟡	No throttling anywhere.
GDPR subject export/erasure + consent	security	🟡	No flow; erasure also blocked by missing soft delete.
Disaster recovery runbook + RTO/RPO	security	🟡	Neon has provider backups; no documented DR plan.
Universal audit-log coverage	security · coding-standards	🟢	`logActivity` in ~11/21 action files; not all mutations logged.
Universal Zod input validation	coding-standards	🟢	Zod in ~13/21 action files; not every action validates.
`activity_log` partitioning + retention (500M-row target)	architecture	🟡	Single unpartitioned table; no archival.
Search concurrency/backpressure (100 concurrent)	architecture	🟡	No queue or rate control on hotel/flight search.

Changelog

Commit	Summary
`95e0c1b`	fix: run db:migrate on every Vercel deploy (migration 0019 was missing from prod)
`00c6885`	Sprint 2 Waves 1–4: Travel Platform facade — ContentCapable, AutocompleteCapable, `src/lib/travel-platform/index.ts`, consumer migration
`b7eab1b`	Sprint 1 Wave 3: booking-service — quote→book lifecycle, idempotency, event log, supplier ref

`cc65ce1`	Sprint 1 Wave 2: DuffelBookingProvider, HotelbedsBookingProvider, MockBookingProvider, registry wiring
`bb571d2`	Sprint 1 Wave 1: booking_supplier_ref/event/document/idempotency schema + config module (migration 0019)
`a6def26`	Phase 4: standardized BI data export (CSV + Excel) at `/reports`
`148d1fb`	Phase 3: country/nationality reference data + city suggestions
`e09289f`	Phase 2: controlled vocabularies (enums) for cleaner reporting
`0f6840f`	Phase 1 analytics: revenue/conversion/pipeline insights (DZD)
`bf71024`	Enrich demo seed: 100 clients, suppliers, commissions, notifications (DZD)
`0b5de21`	Restrict currencies to DZD/EUR/USD, default DZD; add hotel name search
`441fd6e`	UX large: persistent onboarding (DB), client timeline, booking hard guards
`284d00e`	UX medium: lifecycle stepper, board view, inline search, getting-started card
`805dbc8`	UX small: portal invite, convert proposal→booking, client funnel view, share consolidation
`9c7cc98`	UX quick wins: vocabulary, nav labels, empty charts, dashboard CTA
`23ddc35`	Phase 2 & 3: supplier mgmt, commissions, client portal, AI features
`94aaf08`	Phase 2: real bookings, Stripe Connect, traveler portal, full i18n
`2dd7650`	Embed live flight/hotel search in booking Add dialog
`a677b77`	Fix duplicate-reference crash on create booking/proposal
`169ee74`	Split atlas.md into focused docs under docs/atlas/
`8c1b24e`	Hotel module: full search/results/details, dynamic occupancy pricing, content cache
`b67cfa0`	Phase 1: email (Resend), Stripe billing, e-sign, PDF, suppliers
`a233d32`	View-as-user + i18n (EN/FR/AR + RTL) + Settings hub
`1896596`	Per-role workspaces (Finance + Support) + role-based landing/nav
`9e8fb4b`	Multi-tenant architecture + vendor platform console

Migrations: 19 (latest `0019`). Prod DB: `ep-misty-thunder-aixz34vy`.

Security

Security controls

The control areas Atlas is held to, with honest current status (✓ in place · 🟡 partial / provider-level · 🟠 not implemented). Details for the implemented ones follow in the sections below.

Control	Status	Notes
Tenant isolation	✓	`agencyId` scoping via `requireAgencyUser()`; verified by `test-tenant-isolation.ts`.
RBAC	✓	5 roles + capability helpers; agent visibility now scoped on list pages too.
Audit logs	🟡	`activity_log` + `logActivity` used in ~11 action files; not yet universal.
Encryption	🟡	TLS in transit + Neon at-rest (provider); Better Auth hashes passwords. No app-level field encryption.
Password policy	🟡	Better Auth defaults only; no enforced strength/rotation policy.
Secrets management	🟡	Vercel env vars; **demo credentials committed in the repo** (rotate before prod). No vault.
Backups	🟡	Neon automatic backups + branching (provider-level); not app-managed or restore-tested.
Rate limiting	🟠	None on auth or API routes.
GDPR	🟠	No subject data-export/erasure flow or consent tracking; right-to-erasure also blocked by missing soft delete.
Disaster recovery	🟠	No documented DR runbook or RTO/RPO targets.

> The 🟠/🟡 rows are tracked in the spec-vs-reality gap tracker.

Tenant isolation

Every business table carries `agencyId` (root) or inherits it via a parent (children). **All** reads/writes are scoped by agency through `requireAgencyUser()` → `user.agencyId`. Reference numbers (`BKG-…`, `PRD-…`) are unique per agency. Cross-tenant leakage is re-verified by `scripts/test-tenant-isolation.ts` (seeds two agencies, asserts no leak, cleans up). See architecture.md.

Authentication

- Better Auth (email/password). Signup is **invitation-only**: the `user.create.before` hook requires a matching pending `agency_invite`, stamps `agencyId` + role, and rejects everyone else — this also blocks the raw signup endpoint.

- `BETTER_AUTH_URL` / `baseURL` + `trustedOrigins` restrict trusted origins to the deployed domain.
- Invite tokens have a 7-day TTL (`src/lib/invites.ts`).

Authorization guards

`src/lib/permissions.ts`: `requireUser`, `requireAgencyUser` (tenant + agency-suspension + lapsed-subscription lockout), `requireManager`, `requireCapability`, `requirePlatformAdmin`. Capability rules in business-rules.md. Only an **admin** can assign/change the admin role.

Platform admin & impersonation

- Platform admin: `isPlatformAdmin = true`, `agencyId = null`; sits above all tenants and **cannot enter a tenant except via impersonation**.
- Impersonation is **cookie-driven and platform-admin only** (`platform_view_agency` / `platform_view_user`), resolved in `requireUser`, with a persistent exit banner. See architecture.md.

Client portal sessions

Passwordless magic-link sessions (`portal_session`) are scoped to a single client and **separate** from staff auth, stored in an httpOnly cookie (`portal-session.ts`).

Webhooks

Stripe webhooks use **manual signature verification** before acting: `api/stripe/webhook` (subscriptions) and `api/stripe/connect-webhook` (Connect payments). Secrets: `STRIPE_WEBHOOK_SECRET`, `STRIPE_CONNECT_WEBHOOK_SECRET`.

Database safety

`PROTECTED_DB_HOSTS` makes destructive scripts **refuse to run against a protected (prod) host** unless explicitly overridden with `ALLOW_PROD=1`. In production it is set to the prod branch host `ep-misty-thunder-aixz34vy`. Schema changes go through `db:generate` → `db:migrate`; **never** `db:push`.

Known risks

- **Demo credentials in the repo.** `deployment.md` lists live demo passwords — including the platform super-admin account. They must be rotated or deleted before any real production use. See deployment.md.
- **Locale cookie gap** — cosmetic only; a fresh device shows English until the user re-picks (see roadmap.md open items).

Analytics & BI

Decision-oriented dashboards plus standardized data export, all currency-safe.

Dashboards

Surfaced on the agency dashboard and `/finance`:

- Revenue evolution (monthly)
- Top destinations / clients / markets by revenue
- Revenue per agent
- Conversion rate, average booking value, MoM growth
- Pipeline funnel **by value** + weighted forecast / win rate
- AR aging buckets
- Margin %
- Commission-by-supplier

Currency-safe by design

No FX conversion (see business-rules.md). Every metric **groups by currency** rather than summing across, with a **DZD headline**.

`src/lib/analytics.ts`

Pure helpers (no I/O), unit-testable:

- `sumByCurrency` — totals grouped by currency code
- `monthlyBuckets` — time-series bucketing
- `topN` / `countBy` — rankings and tallies
- `conversionRate`, `growthPct` — rate/growth metrics
- `agingBuckets` — AR aging

Charts render via `src/components/charts/` (recharts 3, tokenized), including `HBarInsight` and `FunnelInsight`.

BI export (`/reports`)

Gated by `canViewFinance` (admin/manager/finance). One-click CSV + Excel export of every core dataset, plus a full multi-sheet workbook.

- `src/lib/export/` — `csv.ts` (RFC-4180 + UTF-8 BOM, Power BI friendly), `xlsx.ts` (exceljs workbook), `datasets.ts` (BI dataset registry).
- Each enum field emits **both** `*_code` and `*_label` columns; amounts in DZD, dates in ISO; a date-range filter applies.

- Served by `api/export/[entity]` — CSV/XLSX per dataset, or `workbook` = all sheets.

AI Features

Built on the **Vercel AI SDK + OpenRouter** (`OPENROUTER_API_KEY`). All features degrade/no-op gracefully when the key is unset. See api-integrations.md.

AI must never

Hard guardrails for every AI feature. These extend the never rules to the AI surface:

- **Never invent prices** — quote only live supplier rates or stored values, never a guessed number.
- **Never invent bookings** — only create a booking when explicitly asked.
- **Never invent clients** — never fabricate a client record.
- **Never invent payments** — never record or imply a payment that didn't happen.
- **Never modify data without confirmation** — mutations are explicit and reviewable, not silent side effects of a chat reply.
- **Never hide errors** — surface failures to the user; don't paper over a failed tool call with a plausible answer.
- **Never perform destructive actions silently** — no deletes/overwrites without a clear, confirmed user action.

> **Current state:** all assistant tools are agency-scoped. Of them only `createBooking` mutates — it creates a **draft** booking (zero balance, unconfirmed), and the system prompt forbids inventing supplier confirmations or PNRs and restricts creation to explicit requests. Everything else (`searchFlights`, `searchHotels`, `findClients`, `bookingsSummary`) is read-only. Gap: creation is intent-gated by the prompt, not yet behind a hard confirm step — the "modify without confirmation" guard is a convention, not an enforced gate.

AI assistant (`/assistant`)

Chat backed by `api/chat` with **agency-scoped tools**: find clients, bookings summary, create booking, search flights/hotels. Tools respect tenant scoping like every other action.

Inline features

Embedded where the work happens, not in a separate AI section:

- **AI itinerary generation** — one-click generation from booking items; saves to `booking_day` rows. Shareable via `/i/[token]`.
- **AI quote builder** — natural-language brief → structured proposal line items, pre-filled in the new-proposal form.
- **AI email drafting** — generate subject + body for confirmation, voucher, follow-up, or custom emails from the booking messages panel.
- **AI visa assistant** — per-nationality visa requirement summary from traveller passport

nationalities + booking destination.

Implementation

- Server actions in `src/lib/actions/ai.ts`.
- Itinerary helpers in `src/lib/itinerary.ts`.
- The assistant's tool definitions are agency-scoped wrappers over existing server actions, so AI cannot bypass security.md tenant isolation.

Agentic Coding Starter Kit

A production-oriented starter kit for building AI-powered web apps with an agentic development workflow. It gives you a working Next.js app, authentication, PostgreSQL, Drizzle ORM, AI SDK integration, shadcn/ui components, and project instructions that help coding agents plan, split, implement, review, and verify changes.

The goal is simple: install the starter, describe the product you want to build, and let your coding agent help turn the boilerplate into your actual POC, MVP, or internal tool.

What You Get

- **Next.js 16 and React 19** with the App Router
- **TypeScript** and a strict project setup
- **Better Auth** with email/password enabled by default
- **PostgreSQL and Drizzle ORM** for schema and migrations
- **AI SDK and OpenRouter** for chat and AI features
- **shadcn/ui, Tailwind CSS, and Lucide icons** for the UI foundation
- **Local or Vercel Blob file storage** through one storage abstraction
- **Agent instructions** through `AGENTS.md` and `CLAUDE.md`
- **Agent skills** for specs, implementation, reviews, security scans, UI work, and shipping

Quick Start

Create a new app with the CLI:

```
npx create-agentic-app@latest my-app
cd my-app
```

Or create the app in the current directory:

```
npx create-agentic-app@latest .
```

Then configure and run the app:

```
cp env.example .env
docker compose up -d
pnpm db:migrate
pnpm dev
```

Open http://localhost:3000.

The CLI copies the starter files, installs dependencies with your selected package manager, and prepares the environment file. If you use `npm`, replace the `pnpm` commands above with `npm run`.

Guided Setup with Claude Code (Optional)

If you use Claude Code, you can install the `create-agentic-app` skill and have Claude walk you through the entire setup — folder strategy, package manager, PostgreSQL (Docker / Neon / Vercel / BYO), `.env` config, migrations, optional integrations (OpenRouter, Vercel Blob, Polar, email), build verification, and dev-server check — ending at a verified `http://localhost:3000`.

Install the skill:

```
npx skills add leonvanzyl/agentic-coding-starter-kit@create-agentic-app --agent clau
```

The `--agent claude-code` flag is required. Without it, the installer drops the skill into 37+ IDE adapter folders at the project root.

Once installed, ask Claude something like:

```
Scaffold a new Agentic Coding Starter Kit project here.
```

Claude will run the skill end-to-end and ask you the few decisions it actually needs to make.

Prerequisites

- Node.js 18 or newer
- Git
- PostgreSQL, either through the included Docker Compose file or a hosted provider
- A package manager: `pnpm`, `npm`, or `yarn`
- Optional: an OpenRouter API key for AI chat features
- Optional: a Vercel account for deployment, hosted Postgres, and Blob storage

Environment Variables

Start from `env.example` and update values for your environment:

```
# Database
POSTGRES_URL=postgresql://dev_user:dev_password@localhost:5432/postgres_dev

# Authentication - Better Auth
BETTER_AUTH_SECRET=your-random-secret

# AI Integration via OpenRouter
OPENROUTER_API_KEY=
OPENROUTER_MODEL="openai/gpt-5-mini"

# Optional - for vector search only
OPENAI_EMBEDDING_MODEL="text-embedding-3-large"

# App URL
NEXT_PUBLIC_APP_URL="http://localhost:3000"
```

```
# File storage
BLOB_READ_WRITE_TOKEN=

# Polar payment processing
POLAR_WEBHOOK_SECRET=polar_
POLAR_ACCESS_TOKEN=polar_
```

For local development, the default database URL works with the included `docker-compose.yml`. For production, use the database URL from your hosting provider.

Generate a strong `BETTER_AUTH_SECRET` before deploying. The starter ships with a development value only so you can get moving quickly.

Default Auth

The starter now defaults to **email and password authentication** through Better Auth. This keeps the first setup small and helps you start building POCs and MVPs without creating OAuth credentials up front.

The current auth setup includes:

- user registration
- email/password login
- protected routes
- password reset flow
- email verification flow

In development, verification and password reset links are logged to the terminal instead of being sent through an email provider. When you are ready for production, ask your coding agent to connect an email service and update the Better Auth email callbacks.

Adding Google OAuth

Google OAuth is no longer the default, but adding it back is straightforward. Ask your coding agent:

```
Add Google OAuth to this Better Auth setup. Keep email/password login enabled, add t
```

Your agent should update the Better Auth config, add the required `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` variables, and adjust the login UI.

Build With an Agent

This starter is designed to be used with coding agents. The generated project includes instructions that tell agents how to plan, ask questions, split work, use sub-agents when useful, follow the design system, and verify changes.

- `AGENTS.md` is the main instruction file for Codex, Cursor, and other agent-compatible tools.

- `CLAUDE.md` points Claude users to the same project guidance.
- `.agents/skills/` and `.claude/skills/` include optional workflows for more specialized tasks.
- `DESIGN.md` defines the UI design system agents should follow.

The default workflow does not require slash commands or a separate spec file.

Recommended Default Workflow

1. Install the starter and open the project in your coding-agent environment.
2. Switch your agent tool to planning mode.
3. Describe the app you want to build in plain language.
4. Let the agent ask clarifying questions and shape a clear plan.
5. Confirm the plan once the goal, scope, constraints, and success criteria are clear.
6. Switch your agent tool to edit mode.
7. Ask the agent to implement the approved plan.
8. The main agent should split the work into parallel streams, silos, or feature chunks that fit within context.
9. The agent should use sub-agents to implement those chunks in parallel where useful, then coordinate the results.
10. The agent should run quality checks such as lint, typecheck, and build.
11. Review the result in the browser and iterate.

You do not need a special command for this default workflow. The project instructions already tell the agent how to plan, split implementation work, use sub-agents, and verify the result.

Starter Prompt

Use this as a first message to your coding agent after installing the starter:

```
I am using the Agentic Coding Starter Kit. Treat the existing app as boilerplate tha

Use the project instructions in AGENTS.md or CLAUDE.md. During planning, ask clarify

What I want to build:
[Describe your app here]
```

For example:

```
What I want to build:
A lightweight CRM for solo consultants. It should let users manage clients, track de
```

When to Use Specs

For most POCs and MVPs, the normal agent workflow is enough. Use a spec when the feature is large, long-running, risky, or needs to be split across multiple implementation sessions.

The starter includes two skills for that workflow:

- `create-spec`: turns a planning conversation into `specs/{feature}/` with requirements, task files, dependency waves, and manual action notes.
- `implement-feature`: reads a spec folder and coordinates implementation wave by wave with review gates.

Use this workflow when:

- the feature spans many files or modules
- multiple agents should work in parallel
- the implementation may take more than one session
- you need resumable progress tracking
- you want a written implementation record before coding starts

Example agent request:

```
Create a spec for the billing and subscriptions feature we just planned. Break it in
```

Then:

```
Implement the billing and subscriptions spec from specs/billing-subscriptions.
```

Project Structure

```
src/
├── app/
│   ├── (auth)/
│   │   ├── forgot-password/
│   │   ├── login/
│   │   ├── register/
│   │   └── reset-password/
│   ├── api/
│   │   ├── auth/
│   │   ├── chat/
│   │   └── diagnostics/
│   ├── chat/
│   ├── dashboard/
│   ├── profile/
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── auth/
│   ├── ui/
│   ├── site-footer.tsx
│   └── site-header.tsx
├── hooks/
└── lib/
    ├── auth.ts
    ├── auth-client.ts
    ├── db.ts
    ├── env.ts
    ├── schema.ts
```

```
    ├── session.ts
    ├── storage.ts
    └── utils.ts
```

Important root files:

- `AGENTS.md`: coding-agent behavior rules
- `CLAUDE.md`: Claude entrypoint for the same guidance
- `DESIGN.md`: UI design system and component guidance
- `drizzle.config.ts`: Drizzle migration configuration
- `docker-compose.yml`: local PostgreSQL service
- `env.example`: environment variable template
- `components.json`: shadcn/ui configuration

Available Scripts

```
pnpm dev           # Start the development server with Turbopack
pnpm build         # Run migrations, then build for production
pnpm build:ci      # Build without running migrations
pnpm start         # Start the production server
pnpm lint          # Run ESLint
pnpm typecheck     # Run TypeScript without emitting files
pnpm check         # Run lint and typecheck
pnpm format        # Format the repository
pnpm format:check  # Check formatting
pnpm setup         # Run the setup script
pnpm db:generate   # Generate Drizzle migrations
pnpm db:migrate    # Run Drizzle migrations
pnpm db:studio     # Open Drizzle Studio
```

The repository also contains Drizzle push/reset helper scripts for local experimentation. For schema changes you intend to keep, prefer:

```
pnpm db:generate
pnpm db:migrate
```

Do not use schema push as a replacement for migrations in real project work.

Database Workflow

For local development:

```
docker compose up -d
pnpm db:migrate
```

When your app needs schema changes, ask your agent to update `src/lib/schema.ts`, generate a migration, and run it:

```
pnpm db:generate
pnpm db:migrate
```

If you deploy to Vercel or another hosted environment, set `POSTGRES_URL` in that environment before running migrations or building the app.

AI Features

The starter uses the Vercel AI SDK with OpenRouter. Set these variables to enable AI chat:

```
OPENROUTER_API_KEY=sk-or-v1-your-key
OPENROUTER_MODEL="openai/gpt-5-mini"
```

OpenRouter lets you switch models without changing the application code. Update `OPENROUTER_MODEL` when you want to try a different model.

File Storage

The starter includes a storage abstraction that can use local storage in development or Vercel Blob in production.

For local development, leave `BLOB_READ_WRITE_TOKEN` empty. Files are stored under `public/uploads/`.

For Vercel Blob:

1. Create a Blob store in Vercel.
2. Copy the `BLOB_READ_WRITE_TOKEN`.
3. Add it to your production environment variables.

The app chooses the storage backend based on whether `BLOB_READ_WRITE_TOKEN` is configured.

Deployment

Vercel is the recommended deployment target.

```
npm install -g vercel
vercel --prod
```

Set the required production environment variables:

- `POSTGRES_URL`
- `BETTER_AUTH_SECRET`
- `NEXT_PUBLIC_APP_URL`
- `OPENROUTER_API_KEY`, if using AI features
- `OPENROUTER_MODEL`, if using AI features

- `BLOB_READ_WRITE_TOKEN`, if using Vercel Blob
- `POLAR_WEBHOOK_SECRET` and `POLAR_ACCESS_TOKEN`, if using Polar payments

The default `pnpm build` script runs database migrations before `next build`. If your CI or host should not run migrations during build, use `pnpm build:ci` and run migrations as a separate deployment step.

Troubleshooting

The app cannot connect to Postgres

Confirm Docker is running and start the database:

```
docker compose up -d
```

Then check that `POSTGRES_URL` in `.env` matches the database connection string.

Auth reset or verification emails are not arriving

In development, links are logged to the terminal. This is intentional. Connect an email provider before using password reset or verification in production.

AI chat is not working

Set `OPENROUTER_API_KEY` and restart the dev server. Also confirm `OPENROUTER_MODEL` is a model available to your OpenRouter account.

My agent is preserving too much boilerplate

Tell the agent directly that the starter UI is scaffolding and should be replaced:

```
Replace the starter UI with the actual product UI. Do not keep setup checklists, pla
```

I need Google login

Ask your agent to add Google OAuth through Better Auth while keeping email/password enabled. You will need Google OAuth credentials and production callback URLs.

Video Tutorial

Watch the original walkthrough:

Agentic Coding Boilerplate Tutorial

Some details in older videos may differ from the current starter. This README is the source of truth for the current default workflow.

Support This Project

If this starter kit helped you build something useful, you can support the project here:

Buy me a coffee

Contributing

1. Fork this repository.
2. Create a feature branch.
3. Make your changes.
4. Run the relevant checks.
5. Open a pull request.

License

This project is licensed under the MIT License.

Need Help?

- Check the repository issues: github.com/leonvanzyl/agentic-coding-starter-kit/issues
- Review `AGENTS.md`, `CLAUDE.md`, and `DESIGN.md`
- Open a new issue with the exact setup steps, error output, and environment details

Atlas — Travel Agency SaaS

A multi-tenant SaaS for travel agencies: each agency runs its bookings, clients, proposals and finance in an isolated workspace, while the vendor manages all agencies from a platform console. Built on the Agentic Coding Starter Kit.

Live: https://agencetool.vercel.app

Tech stack

- **Framework:** Next.js 16 (App Router) + React 19 + TypeScript
- **Styling:** Tailwind CSS v4 (CSS-first, `@theme inline`) + shadcn/ui (new-york)
- **Auth:** Better Auth (email/password, invitation-gated signup)
- **DB:** PostgreSQL (Neon) + Drizzle ORM
- **i18n:** next-intl (EN / FR / AR, cookie-based, RTL)
- **Charts:** recharts (tokenized to the design system)
- **AI:** Vercel AI SDK + OpenRouter (assistant — needs `OPENROUTER_API_KEY`)
- **Hosting:** Vercel (GitHub auto-deploy on `main`)

See `DESIGN.md` for the design system and `AGENTS.md`/`CLAUDE.md` for working conventions.

Architecture

Multi-tenancy

Every business table carries an `agencyId` and **all** queries are scoped by it, so agencies never see each other's data. References (`BKG-…`, `PRD-…`) are unique **per agency**. Tenancy is enforced at the data layer and re-verified by a test harness (`scripts/test-tenant-isolation.ts`).

Tenant-root tables: `agency`, `client`, `opportunity`, `product`, `booking`, `notification`, `activity_log`, `agency_invite`, plus `user.agencyId`. Child tables (travellers, items, payments, days, contacts) inherit tenancy through their parent.

Roles (RBAC)

Five roles, defined in `src/lib/domain.ts` with capability helpers:

Role	Sees all agency data	Manages team	Manages payments	Deletes records	Home
admin	✓	✓	✓	✓	`/dashboard`
manager	✓	✓	✓	✓	`/dashboard`

finance	✓	—	✓	—	`/finance`
support	✓	—	—	—	`/support`
agent	own work only	—	—	—	`/dashboard` (scoped)

Helpers: `seesAllData`, `canManageTeam`, `canAssignAdmin`, `canManagePayments`, `canViewFinance`, `canViewSupport`, `canDeleteRecords`, `roleHome`.

Auth & onboarding

- **Invitation-only signup**: the Better Auth create hook (`src/lib/auth.ts`) rejects any signup without a matching pending invite and stamps `agencyId` + role from it. Accept flow lives at `/invite/[token]`.
- Guards (`src/lib/permissions.ts`): `requireUser`, `requireAgencyUser` (tenant + suspension lockout), `requireManager`, `requireCapability`, `requirePlatformAdmin`.

Platform (vendor) console — `/platform`

The vendor is a user with `isPlatformAdmin = true` and `agencyId = null`, sitting above all tenants. From `/platform` they can:

- Create agencies (generates the first-admin invite link)
- Suspend / reactivate agencies (suspended → users locked out)
- **View as agency** (act as agency admin) or **View as user** (act as a specific user with *their* role + scoped data) — with an exit banner.

Impersonation is cookie-driven (`platform_view_agency` / `platform_view_user`) and only takes effect for a platform admin (resolved in `requireUser`).

Per-role workspaces

- `/dashboard` — agency overview (admin/manager) or "Your work" (agent); manager view includes analytics charts (bookings by country, team performance, status, monthly trend) + finance KPIs.
- `/finance` — payments / accounts-receivable + revenue, with charts.
- `/support` — action queue (bookings needing attention) + clients + ops.
- `/team`, `/clients`, `/bookings`, `/opportunities`, `/products`, `/operations`, `/search`, `/assistant`, `/settings`, `/profile`.

i18n

- English / French / Arabic, **cookie-based** (no URL-locale routing — routes unchanged). Arabic is full **RTL** with **IBM Plex Sans Arabic**.
- Config: `src/i18n/config.ts` + `src/i18n/request.ts`; messages in `messages/{en,fr,ar}.json`. Change language in **Settings**.
- Core surfaces (nav, login, dashboard, settings) are translated; deeper pages fall back to English. To translate more, add keys to **all three** message files and swap strings to `t("…")`.

Local development

```
npm install --legacy-peer-deps     # better-auth peer range needs this
npm run dev                        # http://localhost:3000
npm run check                      # lint + typecheck
npm run build:ci                   # next build (no migrate)
```

Required env (`.env`): `POSTGRES_URL`, `BETTER_AUTH_SECRET`. Optional (features off until set): `OPENROUTER_API_KEY` (AI chat), `BLOB_READ_WRITE_TOKEN` (uploads), Google OAuth.

Database

```
npm run db:generate   # after editing src/lib/schema.ts
npm run db:migrate    # apply migrations  (NEVER db:push)
npm run db:studio
```

Migrations in `drizzle/`. Notable: `0006` (tenancy + backfill), `0007` (invites), `0008` (user.locale).

Scripts

```
# Promote an existing account to platform super-admin (vendor)
npx tsx --env-file=.env scripts/make-platform-admin.ts <email>

# Seed / reset the Demo Agency with rich demo data (idempotent)
npx tsx --env-file=.env scripts/seed-demo-data.ts

# Verify cross-tenant isolation (seeds 2 agencies, asserts no leak)
npx tsx --env-file=.env scripts/test-tenant-isolation.ts
```

Deployment

Hosted on **Vercel** (`atlasproject/agence_tool`), connected to GitHub (`ouks1993/agence_tool`).

- **Auto-deploy:** `git push origin main` builds & ships production.
- **Manual:** `npx vercel deploy --prod --yes`
- Config in `vercel.json` (`buildCommand: npm run build:ci`) + `.npmrc` (`legacy-peer-deps=true`).
- Env vars on Vercel: `POSTGRES_URL`, `BETTER_AUTH_SECRET`, `NEXT_PUBLIC_APP_URL`, `BETTER_AUTH_URL` (the last fixes `INVALID_ORIGIN` on the deployed domain).

> ⚠ Production and local dev currently **share one Neon database**. Split prod onto its own Neon branch before onboarding real customers.

Demo

The **Demo Agency** is seeded for customer demos (re-run the seed script to reset). Accounts (Demo Agency unless noted):

Role	Email
Platform admin (vendor)	`ouksili.abdelmalek@gmail.com`
Manager	`yasmine@agence.test`
Finance	`finance@demo.test`
Support	`support@demo.test`
Agents	`karim@demo.test`, `lina@demo.test`, `omar@demo.test`

> Demo credentials are throwaway and set by the seed/admin scripts — **rotate or remove them before any real production use**. (Agent accounts use the seed default; reset any password via the scripts above.)

Suggested demo flow: sign in as the vendor → `/platform` → create an agency (or *View as* Demo Agency) → tour the manager dashboard charts → switch logins to show the finance, support and agent views → open Settings and switch to Français or العربية (RTL).

Roadmap / open items

- **Split prod database** onto a dedicated Neon branch (highest priority).
- **Translate deeper pages** (bookings, clients, finance, support, platform).
- **Cross-device locale**: apply a user's saved `locale` on a fresh device (sync `user.locale` → cookie on login).
- **Email delivery** for invite links (currently copy-paste; module logs to console).
- **Billing** / **subscriptions** (per-agency plans).
- **Traveler portal** (end-customer login to view their trips).

CRITICAL RULES - MUST FOLLOW

RESPONSES

- Keep responses concise and to the point - unless the user asks otherwise

PLANNING MODE

- Always ask clarifying questions
- Never assume design, tech stack or features
- Use deep-dive sub-agents to assist with research
- Use deep-dive sub-agents to review the different aspects of your plan before presenting to the user

CHANGE / EDIT MODE

- Never implement features yourself when possible - use sub-agents!
- Identify changes from the plan that can be implemented in parallel, and use sub-agents to implement the features efficiently
- When using sub-agents to implement features, act as a coordinator only
- Use the best model for the task - premium models for complex tasks (like coding) and mid-tier models for simpler tasks, like documentation
- After completing features (large or small), always run commands like lint, type check and next build to check code quality

DATABASE SCHEMA CHANGES

- Whenever you make changes to the database schema, ALWAYS run the drizzle generate and migrate commands
- NEVER run drizzle push!
- For all ID columns NOT related to BetterAuth, use UUID for the ID columns and be randomly generated

TESTING

- Use any testing tools, libraries available to the project for testing your changes
- Never assume your changes simply work, always test!
- If the project does not have any testing tools, scripts, MCP tools, skills, etc. available for testing, ask the user whether testing should be skipped.

UI DESIGN

- Always follow the UI design system when creating or reviewing components or pages.
- Design System: @DESIGN.md

FEATURE FILTER RULE

Before any feature is committed to the roadmap, it must satisfy at least one of:

1. **Reduces time to booking** — the agent completes the booking workflow faster
2. **Increases booking conversion** — more proposals become bookings, more searches become proposals
3. **Increases agency retention** — the agency becomes more operationally dependent on Atlas

If a proposed feature cannot clearly satisfy at least one criterion, it does not move forward. Apply this rule before design, before estimation, before any engineering discussion.

Examples:

- Supplier booking ✓ (reduces time to booking + increases conversion)
- Client portal depth ✓ (increases conversion + increases retention)
- Marketing campaigns module ✗ (does not directly satisfy any criterion — do not build)
- WhatsApp messaging platform ✗ (build as output channel/integration, not a messaging system)
- Accounting workflows (invoice generation + export) ✓ (reduces time to booking)
- Full GL accounting module ✗ (does not satisfy any criterion — partner instead)

PRE-COMMIT DOCUMENTATION RULE

Before every commit, you MUST:

1. **Review all modified files** — understand the full scope of what changed.
2. **Update affected documentation** — if the change touches behaviour, architecture, business rules, APIs, DB schema, UI patterns, or config, update the relevant file(s) in `docs/`, `atlas.md`, `DESIGN.md`, or `AGENTS.md` to stay in sync.
3. **Ensure no contradictions** — the docs must not describe something that the code no longer does. If they conflict, fix the docs before committing.
4. **Record architectural/business decisions** — if the change introduces a new architectural pattern, a significant tech choice, or a business-rule change, create a record under `docs/decisions/` using the template at `docs/decisions/_template.md`.
5. **Summarize doc updates in the commit message** — include a "Docs:" line listing which documentation files were updated and why.

Example commit message footer:

```
Docs: updated docs/business-rules.md (new booking prerequisite),
      docs/decisions/0001-soft-delete-strategy.md (new decision record)
```

Design System

This document defines the visual design system for the project. All new components and pages **must** follow these tokens, patterns, and conventions.

Stack

- **Framework:** Next.js (App Router) + React + TypeScript
- **Styling:** Tailwind CSS v4 (CSS-first config via `@theme inline` in `globals.css` — no `tailwind.config.ts`)
- **Components:** shadcn/ui (new-york style, neutral base)
- **Icons:** Lucide React
- **Fonts:** Geist (sans) + Geist Mono (mono) via `next/font/google`
- **Dark mode:** next-themes (class-based, system default)
- **Utilities:** `cn()` from `@/lib/utils` (clsx + tailwind-merge)

Colors

All values use the **oklch** color space. Colors are defined as CSS custom properties in `globals.css` and bridged to Tailwind via `@theme inline`.

Semantic Tokens

Token	Light	Dark	Usage
`background`	`oklch(1 0 0)`	`oklch(0.141 0.005 285.823)`	Page background
`foreground`	`oklch(0.141 0.005 285.823)`	`oklch(0.985 0 0)`	Primary text
`primary`	`oklch(0.21 0.034 270)`	`oklch(0.92 0.02 270)`	Buttons, links, accents
`primary-foreground`	`oklch(0.985 0 0)`	`oklch(0.21 0.006 285.885)`	Text on primary
`secondary`	`oklch(0.967 0.001 286.375)`	`oklch(0.274 0.006 286.033)`	Secondary buttons, subtle bg
`secondary-foreground`	`oklch(0.21 0.006 285.885)`	`oklch(0.985 0 0)`	Text on secondary
`muted`	`oklch(0.967 0.001 286.375)`	`oklch(0.274 0.006 286.033)`	Subdued backgrounds

muted-foreground	oklch(0.552 0.016 285.938)	oklch(0.705 0.015 286.067)	Subdued text, placeholders
accent	oklch(0.96 0.012 270)	oklch(0.28 0.018 270)	Hover backgrounds, highlights
accent-foreground	oklch(0.21 0.006 285.885)	oklch(0.985 0 0)	Text on accent
destructive	oklch(0.577 0.245 27.325)	oklch(0.704 0.191 22.216)	Error states, delete actions
card	oklch(1 0 0)	oklch(0.21 0.006 285.885)	Card backgrounds
card-foreground	oklch(0.141 0.005 285.823)	oklch(0.985 0 0)	Card text
popover	oklch(1 0 0)	oklch(0.21 0.006 285.885)	Popover/dropdown bg
popover-foreground	oklch(0.141 0.005 285.823)	oklch(0.985 0 0)	Popover/dropdown text
border	oklch(0.92 0.004 286.32)	oklch(1 0 0 / 10%)	Borders, dividers
input	oklch(0.92 0.004 286.32)	oklch(1 0 0 / 15%)	Input borders
ring	oklch(0.705 0.06 270)	oklch(0.552 0.05 270)	Focus rings

Chart Colors

Token	Light	Dark
chart-1	oklch(0.646 0.222 41.116)	oklch(0.488 0.243 264.376)
chart-2	oklch(0.6 0.118 184.704)	oklch(0.696 0.17 162.48)
chart-3	oklch(0.398 0.07 227.392)	oklch(0.769 0.188 70.08)
chart-4	oklch(0.828 0.189 84.429)	oklch(0.627 0.265 303.9)
chart-5	oklch(0.769 0.188 70.08)	oklch(0.645 0.246 16.439)

Sidebar Colors

Token	Light	Dark
sidebar	oklch(0.985 0 0)	oklch(0.21 0.006 285.885)
sidebar-foreground	oklch(0.141 0.005 285.823)	oklch(0.985 0 0)

`sidebar-primary`	`oklch(0.21 0.006 285.885)`	`oklch(0.488 0.243 264.376)`
`sidebar-primary-foreground`	`oklch(0.985 0 0)`	`oklch(0.985 0 0)`
`sidebar-accent`	`oklch(0.967 0.001 286.375)`	`oklch(0.274 0.006 286.033)`
`sidebar-accent-foreground`	`oklch(0.21 0.006 285.885)`	`oklch(0.985 0 0)`
`sidebar-border`	`oklch(0.92 0.004 286.32)`	`oklch(1 0 0 / 10%)`
`sidebar-ring`	`oklch(0.705 0.015 286.067)`	`oklch(0.552 0.016 285.938)`

Ad-hoc Status Colors

Use these Tailwind utilities for status indicators — they are not part of the token system but are used consistently:

- **Success:** `text-green-600` / `dark:text-green-400`, `bg-green-500`
- **Error:** `text-red-600`, `text-destructive`

Typography

Font Families

Token	Font	Usage
`--font-geist-sans`	Geist	All UI text (applied to body)
`--font-geist-mono`	Geist Mono	Code, monospace content

Body has `font-feature-settings: "rlig" 1, "calt" 1` and `antialiased` enabled.

Type Scale

Class	Size	Usage
`text-xs`	12px	Timestamps, shortcuts, helper text, code
`text-sm`	14px	Descriptions, labels, body copy, card descriptions
`text-base`	16px	Base text, inputs (mobile)
`text-lg`	18px	Dialog titles, sub-headings
`text-xl`	20px	Section titles, header logo
`text-2xl`	24px	Page titles, card titles

`text-3xl`	30px	Dashboard/profile headings
`text-4xl`	36px	Large display text
`text-5xl`	48px	Hero title

Font Weights

Class	Weight	Usage
`font-medium`	500	Buttons, labels, nav items
`font-semibold`	600	Card titles, section headings, badges, dialog titles
`font-bold`	700	Page titles, hero heading

Line Heights & Tracking

Class	Usage
`leading-none`	Labels, card titles
`leading-5`	Code blocks
`leading-6`	List items
`leading-7`	Paragraphs (markdown)
`tracking-tight`	Hero/display text
`tracking-widest`	Keyboard shortcuts

Spacing

Container Pattern

```
container mx-auto px-4
```

Responsive overrides where needed:

- Header: `px-3 sm:px-4`
- Footer: `px-4 sm:px-6 lg:px-8`

Max Widths

Class	Value	Usage
`max-w-sm`	24rem	Auth forms
`max-w-md`	28rem	Login/register cards, error pages

`max-w-lg`	32rem	Dialog content (sm+)
`max-w-2xl`	42rem	Large dialogs
`max-w-3xl`	48rem	Embeds, protected state
`max-w-4xl`	56rem	Main content pages

Vertical Spacing (space-y)

Class	Usage
`space-y-1`	Tight lists, inline stacks
`space-y-1.5`	Card header
`space-y-2`	Form field groups, small stacks
`space-y-3`	Footer stacks
`space-y-4`	Form sections, dialog content
`space-y-6`	Card content sections
`space-y-8`	Page-level sections

Padding

Class	Usage
`p-1`	Dropdown content, icon buttons
`p-2`	Code blocks, muted backgrounds
`p-3`	Chat bubbles, inputs
`p-4`	Grid items, action buttons, list items
`p-6`	Cards, dialog content

Page Vertical Padding

Class	Usage
`py-3 sm:py-4`	Header
`py-4 sm:py-6`	Footer
`py-8`	Standard content pages
`py-12`	Home page, dashboard
`py-16`	Error/not-found pages

Border Radius

Token	Value	Class
`--radius`	`0.625rem` (10px)	Base
`--radius-sm`	`calc(--radius - 4px)` = 6px	`rounded-sm`
`--radius-md`	`calc(--radius - 2px)` = 8px	`rounded-md`
`--radius-lg`	`var(--radius)` = 10px	`rounded-lg`
`--radius-xl`	`calc(--radius + 4px)` = 14px	`rounded-xl`
—	9999px	`rounded-full`

Usage:

- `rounded-md` — Buttons, inputs, textarea, code blocks, dropdowns
- `rounded-lg` — Cards, dialogs, feature cards, chat bubbles
- `rounded-xl` — Hero logo container
- `rounded-full` — Badges, avatars

Shadows

Class	Usage
`shadow-xs`	Inputs, textarea, secondary/outline buttons
`shadow-sm`	Card base
`shadow-md`	Card hover, dropdown content
`shadow-lg`	Dialogs, dropdown sub-content

No custom shadow definitions — all Tailwind defaults.

Animations

Custom Keyframes

Name	Effect	Duration	Easing
`fade-in`	Opacity 0 → 1	0.3s	ease-out
`fade-up`	Opacity 0 → 1 + translateY(8px → 0)	0.4s	ease-out
`scale-in`	Opacity 0 → 1 + scale(0.97 → 1)	0.2s	ease-out

Use via: `animate-fade-in`, `animate-fade-up`, `animate-scale-in`

tw-animate-css Animations

Used on dialogs and dropdowns:

- `animate-in` / `animate-out`
- `fade-in-0` / `fade-out-0`
- `zoom-in-95` / `zoom-out-95`
- `slide-in-from-{top|bottom|left|right}-2`

Transition Classes

Class	Usage
`transition-colors`	Links, hover color changes
`transition-opacity`	Avatar hover, reveal-on-hover
`transition-all duration-200`	Card interactive hover, buttons
`transition-[color,box-shadow]`	Input/textarea focus

Utility Classes

```
.card-interactive {
  @apply transition-all duration-200 ease-out;
}
.card-interactive:hover {
  @apply shadow-md -translate-y-0.5;
}
```

```
.auth-bg {
  background-image: radial-gradient(
    circle at 50% 0%,
    var(--accent) 0%,
    transparent 50%
  );
}
```

Layout

Root Structure

```
<body class="antialiased min-h-screen flex flex-col">
  <SiteHeader />
  <main id="main-content" class="flex-1">{children}</main>
  <SiteFooter />
  <Toaster />
</body>
```

Page Layout Patterns

Auth pages:

```
flex min-h-[calc(100vh-4rem)] items-center justify-center p-4
  → Card w-full max-w-md
```

Standard content pages:

```
container mx-auto px-4 py-8
  → max-w-4xl mx-auto
```

Error/not-found pages:

```
container mx-auto px-4 py-16
  → max-w-md mx-auto text-center
```

Grid Patterns

Pattern	Usage
`grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6`	Feature cards (4-col)
`grid grid-cols-1 md:grid-cols-2 gap-6`	Dashboard cards
`grid grid-cols-1 md:grid-cols-2 gap-4`	Profile info
`grid grid-cols-1 md:grid-cols-3 gap-4`	Quick actions

Responsive Breakpoints

Standard Tailwind breakpoints:

- `sm:` (640px) — Padding adjustments, text alignment, button sizing
- `md:` (768px) — Grid column changes (→ 2 col), input font size
- `lg:` (1024px) — Grid column changes (→ 4 col), wide padding

Icons

Library: Lucide React

Sizing Convention

Size	Classes	Usage
XS	`h-3 w-3`	Inline badge icons
SM	`h-3.5 w-3.5`	Copy buttons

Default	`h-4 w-4` or `size-4`	Standard UI icons
MD	`h-5 w-5`	Header logo icon
LG	`h-7 w-7`	Hero logo icon
XL	`h-16 w-16`	Error/empty state illustrations

Commonly Used Icons

`Bot`, `User`, `Lock`, `Shield`, `Mail`, `Calendar`, `Copy`, `Check`, `Loader2`, `LogOut`, `Sun`, `Moon`, `Github`, `ArrowLeft`, `RefreshCw`, `AlertCircle`, `FileQuestion`, `Database`, `Palette`, `Video`

Components (shadcn/ui)

All components live in `src/components/ui/`. They use `data-slot` attributes, accept `className` for overrides via `cn()`, and follow either `React.forwardRef` or functional component patterns.

Button

6 variants, 4 sizes (CVA-based):

Variant	Usage
`default`	Primary actions
`secondary`	Secondary actions
`outline`	Tertiary actions
`ghost`	Subtle/icon actions
`destructive`	Delete/danger actions
`link`	Inline text links

Size	Height	Padding
`sm`	h-8	px-3
`default`	h-9	px-4
`lg`	h-10	px-6
`icon`	size-9	—

Card

6 sub-components: `Card`, `CardHeader`, `CardTitle`, `CardDescription`, `CardContent`, `CardFooter`

Base: `rounded-lg border bg-card text-card-foreground shadow-sm`

Input / Textarea

- Height: `h-9` (input), `min-h-16` (textarea)
- Border: `border bg-transparent rounded-md shadow-xs`
- Focus: `focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]`
- Validation: `aria-invalid:border-destructive aria-invalid:ring-destructive/20`
- Responsive font: `text-base md:text-sm`

Badge

4 variants: `default`, `secondary`, `destructive`, `outline`

Base: `rounded-full border px-2.5 py-0.5 text-xs font-semibold`

Dialog

Radix-based with overlay (`bg-black/50`), fade + zoom animations, optional close button.

DropdownMenu

Radix-based. Content: `rounded-md border p-1 shadow-md min-w-[8rem]`. Items support a `destructive` variant.

Spinner

Sizes: `sm` (h-4 w-4), `md` (h-6 w-6), `lg` (h-8 w-8). Uses `Loader2` with `animate-spin`.

Toast (Sonner)

Custom icons per state (success, info, warning, error, loading). Themed via CSS variable overrides.

Focus & Interaction States

Focus Ring (Global)

```
outline-2 outline-offset-2 outline-ring/70
```

Component-level override:

```
focus-visible:ring-ring/50 focus-visible:ring-[3px]
```

Disabled

```
disabled:pointer-events-none disabled:opacity-50
```

Interactive Card Hover

```
transition-all duration-200 ease-out
hover:shadow-md hover:-translate-y-0.5
```

Dark Mode

- **Method:** Class-based via `next-themes` with `attribute="class"` and `disableTransitionOnChange`
- **Default:** System preference
- **Toggle:** 3-way dropdown — Light / Dark / System
- All semantic color tokens swap automatically via `.dark` CSS selector
- Use `dark:` prefix for component-specific overrides (e.g., `dark:bg-input/30`)

Branding

Logo Text

```
bg-gradient-to-r from-primary to-primary/70 bg-clip-text text-transparent
```

Logo Icon Container

```
w-8 h-8 rounded-lg bg-primary/10 flex items-center justify-center
```

Hero variant: `w-12 h-12 rounded-xl`

@AGENTS.md

Atlas — Production Readiness Audit Report

Date: 2026-06-28

> **Remediation status (applied 2026-06-28):** The safe high-priority tranche (both Criticals + all 13 Highs + several low-risk Mediums/Lows) has been implemented via coordinated sub-agents and verified with `tsc --noEmit` (clean), `npm run lint` (0 errors), and `next build` (all routes compile). See **§ Remediation Applied** at the end of this report for the full list of fixes, files changed, and remaining items. **One blocker is environmental:** the DB index migration `drizzle/0018_regular_karnak.sql` is generated but **not yet applied** — the dev Neon credentials in `.env` return `28P01 password authentication failed` (password rotated). Refresh `POSTGRES_URL` and run `npx drizzle-kit migrate` to apply it.

Executive Summary

Atlas has a fundamentally sound architecture: tenant isolation via `requireAgencyUser()` and double-condition `and(eq(agencyId, ...), ...)` scoping is consistent across all 20 server-action files, all inputs are Zod-validated, and there is no SQL injection or `dangerouslySetInnerHTML` exposure. However, the application is **not yet production-ready** without remediation. The two genuine blockers are silent fake-PNR fallback on failed airline bookings (agents see a confirmation for a flight that was never booked) and missing try/catch on Stripe webhook DB writes (crashes cause endless Stripe retries with duplicate side-effects). Counting across all six audits: **2 critical blockers, 11 high-severity, 21 medium-severity, and ~20 low-severity** issues. The high-severity cluster is dominated by missing error handling (no try/catch in 10 action files, no `error.tsx` in client-facing segments), absent `fetch` timeouts on all external supplier calls, RTL breakage in the mobile drawer, and a prompt-injection vector in the AI chat endpoint.

Critical Issues (must fix before production)

#	Category	File	Issue	Fix
C1	Error Handling	`src/lib/suppliers/duffel.ts:278`, `src/lib/actions/bookings.ts:382`	Failed Duffel order silently falls back to a fake `DF-XXXXXXXX` reference and marks booking `confirmed` — agent sees success, no real PNR exists	Return discriminated provisional/confirmed result; never set `status: "confirmed"` on a provisional; surface a visible UI warning

C2	Error Handling	`src/app/api/stripe/webhook/route.ts:75-86`, `src/app/api/stripe/connect-webhook/route.ts:40-46`	No try/catch around webhook DB updates; a DB throw returns 500 and Stripe retries indefinitely → duplicate side-effects. `connect-webhook` also returns 400 (stops retries) when temporarily misconfigured instead of 503	Wrap DB writes in try/catch returning 500 only on genuine transient errors; return 503 (not 400) for misconfiguration so Stripe retries cleanly

High Severity Issues

#	Category	File	Issue	Fix
H1	Security	`src/app/api/chat/route.ts:28-36`	Zod schema accepts `role: "system"` from client → prompt injection / jailbreak of the AI agent, including spurious `createBooking` calls	Drop `"system"` from the clie
H2	Security	`src/app/api/portal/auth/signout/route.ts:2,15`; `src/app/portal/layout.tsx:39`	Portal signout is `GET` via `<a href>` → CSRF forced-logout (CWE-352)	Change handler to `POST`; use
H3	Error Handling	10 action files (`clients.ts:45`, `team.ts`, `suppliers.ts`, `invites.ts`, `opportunities.ts`, `portal-proposals.ts`, `notifications.ts`, `proposals-public.ts`, `portal-invite.ts`, `onboarding.ts`)	Zero try/catch around DB calls → unhandled exceptions crash the action with a generic 500, no logging	Wrap DB writes; return `{ ok` `console.error("[actio`
H4	Error Handling	`src/app/api/chat/route.ts:77-277`	`streamText` and all tool `execute` bodies have no try/catch; `createBooking` item loop silently skips failed items	Wrap `streamText` and each

H5	Error Handling	`src/lib/suppliers/duffel.ts:39`, `hotelbeds.ts:159`, `payments/stripe.ts`	No `fetch` timeout (AbortController) on any external call → function hangs to Vercel's 30s timeout	Add `AbortController` + `s` base fetch wrapper
H6	Error Handling	`src/app/portal/`, `src/app/platform/`, `src/app/(app)/bookings/`, `src/app/(app)/clients/`	No segment-level `error.tsx`; errors bubble to root boundary whose "Go home" link points to `/` (inaccessible to portal clients)	Add `error.tsx` per segmer `portal/login`)
H7	Performance	`src/app/(app)/dashboard/page.tsx:83-85`	Counts done by fetching every row's `id` then `.length` — 3 full-table scans for 3 integers	Use `db.$count(table, w`
H8	Performance	`src/lib/actions/bookings.ts:53`, `src/lib/actions/products.ts:36`	`nextReference()` fetches all references in the agency and finds max in JS on every create	Single `max(...)` aggregatio
H9	Performance	`src/lib/actions/products.ts:57-67`	`recalcTotals()` issues one sequential UPDATE per item (N+1 writes)	Batch CASE-WHEN update c
H10	Performance	`src/app/(app)/dashboard/page.tsx`, `src/app/(app)/finance/page.tsx`	No Suspense boundaries; user sees blank screen until all DB queries finish (finance loads 1,000 bookings first)	Split into shell + Suspense as
H11	UX / RTL	`src/components/app/app-shell.tsx:223`	Mobile nav drawer hardcodes `slide-in-from-left-2` / `left-0` / `border-r` → appears from wrong side in Arabic (RTL)	Conditional `rtl:` variants or
H12	UX / A11y	All auth forms (`sign-in-button.tsx:83`, `sign-up-form.tsx:107`, `forgot-password-form.tsx:70`, `reset-password-form.tsx:25,100`, `accept-invite-form.tsx:99`)	Error `<p>` has no `role="alert"`/`aria-live` → screen readers never announce login failures	Add `role="alert"` to error

H13	Database	`src/lib/schema.ts:188-198` (`verification` table)	No index on `verification.identifier` → full scan on every login/OTP check	Add `index("verification_id` + migration

Medium Severity Issues

#	Category	File	Issue	Fix
M1	Security	`src/lib/actions/commissions.ts:28-31,74-76`	`recordCommission` does not cross-tenant validate `supplierId`, `bookingItemId`, `agentUserId`	Add ownership c scoped to `user.` before INSERT
M2	Security	`src/lib/notifications/templates.ts:14,52-72`	User strings (`clientName`, `agencyName`, `roleLabel`) interpolated into HTML email without escaping	Add `h()` HTML-e helper, apply to a sourced values
M3	Security	`src/lib/storage.ts:117`	File upload validates extension only, not MIME magic bytes; SVG payloads accepted; risky on local-fs fallback path	Add `file-type` byte check; rejec user images
M4	Error Handling	`src/lib/actions/commissions.ts:505-508`	`autoGenerateCommissions` failure swallowed with `console.error`; booking advances with no commissions and no warning	Return structured surface non-bloc warning
M5	Error Handling	`src/lib/actions/platform.ts:134-136`	Stripe customer-create failure only logged; agency saved without `stripeCustomerId` → later "Agency not found" or duplicate customers	Structured log + for reconciliation
M6	Error Handling	`src/lib/suppliers/index.ts:78-97` (`safeSearch`)	`degraded: true` from mock fallback not forwarded to AI in chat route (`api/chat/route.ts:96-101`) → mock prices presented as real	Include `source` in tool result
M7	Error Handling	entire `src/`	No structured logging library; only `console.error(string)` → cannot query/alert per tenant or correlate digests	Adopt Pino/Sentr line logging conv
M8	Performance	`src/lib/actions/bookings.ts:459-468`	`advanceStatus` ticketing loop: sequential external API + DB write per item	`Promise.all` th calls; batch DB w

M9	Performance	src/lib/actions/commissions.ts:258-277	getCommissions() has no .limit() → unbounded result set	Add .limit(50 pagination
M10	Performance	src/app/(app)/dashboard/page.tsx:109	Loads up to 500 bookings with payments+travellers to compute KPIs in JS	Move aggregatio scope traveller q passport alerts o
M11	Performance	src/app/(app)/finance/page.tsx:74	Loads up to 1,000 bookings+payments; KPIs computed via JS reductions	Replace with SQ aggregates; load only when balan
M12	Performance	src/lib/schema.ts	Missing composite indexes for (agencyId, status) / (agencyId, createdAt) on booking, opportunity, product, commission	Add composite i migrate
M13	Performance	src/lib/export/datasets.ts:74+	Export loaders are unbounded findMany → OOM/timeout on large agencies	Per-dataset limit streaming export
M14	Performance	src/components/app/app-shell.tsx:1	Entire nav shell is "use client" only for usePathname()	Extract active-lin tiny <NavLink> as server compo
M15	Performance	src/app/(app)/dashboard/page.tsx:33	Recharts (~150KB) eagerly imported for all users	next/dynamic ssr:false } c imports
M16	Performance	src/components/hotels/hotel-details-view.tsx:221, src/components/search/hotel-details-dialog.tsx:123	Raw <img> bypassing next/image optimization	Use next/imag Hotelbeds CDN images.remot
M17	Code Quality	9+ files (billing.ts, invites.ts, payments.ts, portal-payments.ts, portal-invite.ts, api/portal/auth/request/route.ts, robots.ts, sitemap.ts, auth-client.ts)	NEXT_PUBLIC_APP_URL ?? "localhost:3000" duplicated, bypassing env.ts	Export single AP config.ts
M18	Code Quality	src/lib/suppliers/ vs src/lib/suppliers/providers/	Two parallel supplier abstractions coexist; providers/ registry is empty and unused	Mark provider progress or com migration and re layer
M19	Code Quality	src/lib/actions/**/*.ts	62 of 94 exported server actions lack explicit return-type annotations	Annotate with : Promise<Acti

M20	UX / A11y	`client-form.tsx:62`, `booking-form.tsx:87`, `opportunity-form.tsx`	Server errors only via auto-dismissing toast, no persistent inline error	Add `serverErr` `role="alert"` message
M21	UX / A11y	All forms	`aria-invalid` never set on fields after failed validation	Set `aria-inva` empty/invalid rec
M22	UX / A11y	`clients/page.tsx:92`, `suppliers/page.tsx:85`	Search `<Input>` has no `<Label>` (placeholder is not an accessible name)	Add `<Label htmlFor="q" className="s`
M23	UX / A11y	`mode-toggle.tsx:19`, `create-agency-form.tsx:66`, `travellers-manager.tsx:303`, `payments-manager.tsx:227`	Icon-only buttons missing `aria-label`/`aria-haspopup`; save buttons lack `aria-busy`	Add `aria-labe` `haspopup="me` `busy`
M24	UX / RTL	~62 `mr-`/`ml-` + ~94 `pl-`/`pr-`/`border-l`/`border-r` across components	Physical CSS directions don't flip in RTL	Refactor to logic `ps-`/`pe-`/`borde` `border-e`
M25	UX / RTL	List pages with tables (`clients`, `bookings`, `products`, `suppliers`)	`text-right` numeric columns don't flip in RTL	Replace with `te`
M26	Database	`src/lib/schema.ts`	No `createdAt` index on booking/client/opportunity/product/payment/notification	Add `createdAt` (priority: booking opportunity > pro payment)
M27	Database	`src/lib/schema.ts:423,510-536`	`bookingItem.supplierId` / `productItem.supplierId` FKs have no index → full scans in commission/supplier joins	Add `supplierI` on both tables
M28	Database	`src/lib/schema.ts`	No soft-delete (`deletedAt`) anywhere; hard deletes orphan commissions, no undelete/audit	Add `deletedAt` booking/opportu product; filter list
M29	Database	`src/lib/schema.ts:985-1001`	`supplierRate` has no denormalized `agencyId` → direct agency-scoped queries need double join	Add `agencyId` (matches `supplierCont` pattern)
M30	Database	`src/lib/actions/bookings.ts:65-78`	`recalcTotal(bookingId)` queries `bookingItem` by `bookingId` with no agency scope (footgun if called with attacker-controlled id)	Pass `agencyId` filter `booking.a` or accept pre-loa

Low Severity Issues / Recommendations

#	Category	File	Issue
L1	Security	`src/proxy.ts` matcher	`/suppliers`, `/finance`, `/reports`, `/settings`, `/billing`, `/commissions` not in matcher (still safe via `requireAgencyUser`)
L2	Security	`src/app/api/portal/auth/request/route.ts`	No rate limiting on magic-link request → resource exhaustion / enumeration
L3	Security	`src/lib/actions/settings.ts:21-27`	Locale cookie `httpOnly:false` (intentional; validated via `isLocale()`)
L4	Error Handling	`src/lib/actions/settings.ts:34,65`	Empty `catch {}` with no logging
L5	Error Handling	`src/lib/suppliers/hotelbeds.ts:467`	Silent `catch {}` on availability fallback
L6	Error Handling	`src/lib/suppliers/content-cache.ts:110,121`	`.catch(() => {})` silent cache-write swallow → cache never heals
L7	Error Handling	clipboard handlers `ai-quote-builder.tsx:79`, `itinerary-builder.tsx:306,340`, `payments-manager.tsx:95`	Silent clipboard catch after success toast → "copied!" shown even on failure
L8	Error Handling	`src/app/error.tsx:16-17`	`error.digest` shown in UI but not logged server-side → can't correlate
L9	Performance	`src/lib/actions/commissions.ts:284-309`	`getCommissionsByBooking()` does two sequential queries
L10	Performance	`src/app/layout.tsx:54`	All i18n messages shipped to every client
L11	Performance	`src/app/proposal/[id]/page.tsx`, public token pages	No ISR/revalidate on public proposal pages
L12	Code Quality	`src/lib/session.ts`	Entire file is dead code (`requireAuth`, `getOptionalSession`, stale `protectedRoutes`)
L13	Code Quality	`src/proxy.ts`	Middleware never executes (not `middleware.ts`, no `middleware`/default export) → manifest empty
L14	Code Quality	`src/components/ui/github-stars.tsx`	Exported component with zero consumers

L15	Code Quality	`portal-session.ts:20`, `settings.ts:24,60`, `platform.ts:200,228`	`secure: NODE_ENV==="production"` duplicated 5×
L16	Code Quality	`analytics.ts:62`, `finance/page.tsx:176`	`Intl.DateTimeFormat("en-GB",{month:"short"})` duplicated
L17	Code Quality	`permissions.ts:53,193`	`session.user as unknown as {...}` cast duplicated
L18	Code Quality	`hotelbeds.ts:81`, `add-to-booking-dialog.tsx:67`, `add-to-proposal-dialog.tsx:57`, `payments-manager.tsx:97`, `hotel-details-view.tsx:297`	Non-null assertions without guard
L19	Code Quality	`api/portal/auth/request/route.ts:67`, `notifications/email.ts:30`	`console.log` violates `no-console` ESLint rule (warn/error only)
L20	UX / A11y	`hotel-search-experience.tsx:222-228`	Search button loses label text + no `aria-label` while loading
L21	UX / Mobile	Table wrappers in list pages	No visual scroll hint on overflowing tables
L22	UX / Mobile	`pipeline-board.tsx:101-108`	Stage-move trigger ~20px touch target (< WCAG 44px)
L23	UX / A11y	`hotel-details-view.tsx:221`, `hotel-details-dialog.tsx:123`	Gallery images use empty `alt=""` though not purely decorative
L24	UX / A11y	`portal/layout.tsx:25`	`<nav>` missing `aria-label`
L25	Code Quality	`billing/stripe.ts:17`, `payments/stripe.ts:9`	`isBillingConfigured()`/ `isStripeConfigured()` identical
L26	Database	`src/lib/schema.ts:1023-1025`	`commission.bookingItemId` uses `ON DELETE SET NULL` → orphaned, unreconcilable commissions
L27	Database	`src/lib/schema.ts:652-868`	Missing Drizzle `relations` for `supplier`, `supplierContract`, `supplierRate`, `commission` → breaks `with:` eager loads

L28	Database	`drizzle/0006_add_agency_tenancy.sql:343`	Unconditional backfill could set platform-admin `agencyId` to seed agency (non-issue in practice, all `is_platform_admin=false` then)
L29	Database	`src/lib/schema.ts:882-906`	`hotelContent` uses `text("code")` PK (justified natural key)
L30	Database	`payment`, `bookingTraveller` and other child tables	No denormalized `agencyId` (defense-in-depth gap)

Category Findings

1. Security

Overall posture is solid: `agencyId` is always server-derived from `session.user.agencyId`, every action calls `requireAgencyUser()`/`requireUser()`/`requirePlatformAdmin()`/`requireManager()` first, all mutations use `and(eq(id), eq(agencyId))` double-conditions, all inputs are Zod-validated, Drizzle parameterizes all queries, no secrets are in `NEXT_PUBLIC_`, no `dangerouslySetInnerHTML`, Stripe webhooks verify signatures, impersonation gates on `requirePlatformAdmin()` with `httpOnly` cookies, portal actions verify `clientId + agencyId`, and invite registration ties `agencyId`/`role` to the invite row (`input:false`).

- **H1 — Prompt injection via `role:"system"`** (`api/chat/route.ts:28-36`): client can POST `system` messages that flow into `convertToModelMessages` and override the server system prompt, including triggering `createBooking`. Restrict the enum to `["user","assistant"]`.
- **H2 — CSRF forced-logout** (`api/portal/auth/signout/route.ts:2,15`, `portal/layout.tsx:39`): `GET` signout via `<a href>` is exploitable with `<img src=...>`. Switch to `POST` + form. Impact limited to DoS but CWE-352.
- **M1 — Commission FK cross-tenant** (`commissions.ts:28-31,74-76`): `bookingId` is validated but `supplierId`/`bookingItemId`/`agentUserId` are not, allowing foreign-tenant FK references and an existence oracle.
- **M2 — Unescaped HTML email** (`notifications/templates.ts`): `clientName`/`agencyName`/`roleLabel` interpolated raw; low risk in mail clients but the same names render on admin pages.
- **M3 — Extension-only upload validation** (`storage.ts:117`): no magic-byte check; SVG/script payloads accepted; risky on local-fs fallback.
- **L1** missing middleware matcher routes, **L2** no rate limit on magic-link request, **L3** intentional non-`httpOnly` locale cookie (validated).

2. Error Handling

This is the weakest category and the source of both blockers.

- **C1 — Fake PNR fallback** (`duffel.ts:278`, `bookings.ts:382`): the single most dangerous

behavior in the app. A failed airline booking is recorded as `confirmed` with a provisional `DF-` reference and no agent-visible warning — agents could issue tickets against a non-existent booking.
- **C2 — Webhook DB writes unguarded** (`stripe/webhook/route.ts:75-86`, `connect-webhook/route.ts:40-46`): DB throw → 500 → Stripe retry storm with duplicate side-effects; `connect-webhook` returns 400 on misconfig (stops legitimate retries).
- **H3 — No try/catch in 10 action files**; **H4 — chat route + tools unguarded**; **H5 — no fetch timeouts** on Duffel/Hotelbeds/Stripe (hang to Vercel 30s); **H6 — missing `error.tsx`** in portal/platform/bookings/clients (root boundary sends portal clients to inaccessible `/`).
- Medium: **M4** silent commission auto-gen failure, **M5** Stripe customer-create failure unsurfaced, **M6** `degraded` flag not forwarded to AI, **M7** no structured logging.
- Low: **L4-L8** silent/empty catches and missing digest logging.

3. Performance

No catastrophic issues but several scale time-bombs as agencies grow.

- **H7** dashboard counts via full fetch + `.length`; **H8** `nextReference()` fetches all references per create; **H9** `recalcTotals()` N+1 UPDATEs; **H10** no Suspense → blank screen on dashboard/finance (finance loads 1,000 bookings first, has no `loading.tsx`).
- Medium: **M8** serial ticketing API/DB loop, **M9** unbounded `getCommissions()`, **M10**/**M11** large in-memory KPI computation on dashboard (500 bookings) and finance (1,000 bookings) that should be SQL aggregates, **M12** missing composite indexes, **M13** unbounded export → OOM, **M14** unnecessarily-client AppShell, **M15** eager Recharts ~150KB, **M16** raw `<img>`.
- Low: **L9** sequential commission-by-booking queries, **L10** full i18n payload per client, **L11** no ISR on public proposals.

4. Code Quality

Clean of TODO/FIXME/HACK markers; main issues are dead code and duplication.

- Dead code: **L12** `session.ts` (entire file), **L13** `proxy.ts` (middleware never executes — wrong filename/export), **L14** `github-stars.tsx`.
- Duplication: **M17** `APP_URL` fallback in 9+ files, **L15** `secure` cookie option ×5, **L16** `SHORT_MONTH` formatter, **L17** `session.user` cast ×2, **L25** identical Stripe-configured sentinels.
- Architecture: **M18** two parallel supplier systems (`suppliers/` live vs empty `suppliers/providers/`).
- Types: **M19** 62/94 actions lack return-type annotations; **L18** 5 unguarded non-null assertions; **L19** 2 `console.log` ESLint violations.

5. UX & Accessibility

Strongest gaps are RTL (Arabic) breakage and screen-reader announcements.

- **H11**/**H12** — mobile drawer slides from the wrong side in RTL (`app-shell.tsx:223`); auth error `<p>`s lack `role="alert"`.
- RTL: **M24** ~156 physical-direction utilities should be logical (`me-`/`ms-`/`ps-`/`pe-`/`border-s`/`border-e`); **M25** `text-right` → `text-end` on numeric table columns.

- Forms: **M20** errors only via dismissible toast, **M21** `aria-invalid` never set, **M22** search inputs missing `<Label>`.
- A11y: **M23** icon-only buttons missing `aria-label`/`aria-haspopup`/`aria-busy` (`mode-toggle`, `create-agency-form`, `travellers-manager`, `payments-manager`).
- Loading: **L1.1** 6 list pages have no `loading.tsx` skeleton; low-severity **L20-L24** scroll hint, touch target, gallery alt, nav label.

6. Database

Schema is well-structured with consistent UUID PKs and tenant scoping; gaps are indexing and soft-delete.

- **H13** — no index on `verification.identifier` (full scan every login).
- Indexing: **M26** missing `createdAt` indexes on high-volume tables, **M27** missing `supplierId` indexes on `bookingItem`/`productItem`, **M12** missing `(agencyId, status)` composites.
- Tenant isolation: **M29** `supplierRate` missing denormalized `agencyId`; **M30** `recalcTotal` queries `bookingItem` without agency scope (footgun); **L30** child tables (`payment`, `bookingTraveller`) lack defense-in-depth `agencyId`.
- Data lifecycle: **M28** no soft-delete anywhere; **L26** `commission.bookingItemId SET NULL` orphans commissions.
- Hygiene: **L27** missing relations for supplier/commission tables, **L28** migration 0006 backfill edge case, **L29** `hotelContent` text PK (justified).

Files That Need Immediate Attention

1. **`src/lib/suppliers/duffel.ts`** — C1 (fake PNR), H5 (no fetch timeout), L5/L18 — the highest-risk file; fake-booking behavior is a business blocker.
2. **`src/app/api/stripe/webhook/route.ts`** + **`connect-webhook/route.ts`** — C2 (unguarded DB writes → retry storm).
3. **`src/lib/actions/bookings.ts`** — C1 (confirmed status on provisional), H8 (`nextReference`), M8 (serial ticketing), M30 (`recalcTotal` scope).
4. **`src/app/api/chat/route.ts`** — H1 (prompt injection), H4 (no try/catch), M6 (degraded flag).
5. **`src/components/app/app-shell.tsx`** — H11 (RTL drawer), M14 (client component), M24 (logical props).
6. **`src/app/(app)/dashboard/page.tsx`** — H7 (count via fetch), H10 (no Suspense), M10/M15.
7. **`src/app/(app)/finance/page.tsx`** — H10 (no Suspense/loading), M11 (1,000-row JS KPIs).
8. **`src/lib/schema.ts`** — H13 (verification index), M12/M26/M27/M28/M29, L26/L27.
9. **`src/lib/actions/commissions.ts`** — M1 (cross-tenant FK), M4 (silent failure), M9 (no limit), L9.
10. **`src/lib/actions/products.ts`** — H8 (`nextReference`), H9 (N+1 `recalcTotals`).
11. **Auth form components** (sign-in/sign-up/forgot/reset/accept-invite) — H12 (`role="alert"`), M21.

Remaining Recommendations

- Adopt a structured logging / observability layer (Pino + Vercel log drain, or Sentry) so per-tenant

error rates can be alerted on and user-visible `error.digest` values correlate with server logs (**M7**, **L8**).
- Add rate limiting (Upstash or token bucket) to the magic-link request endpoint and any other unauthenticated POST endpoints (**L2**).
- Introduce a soft-delete strategy (`deletedAt`) on core entities for audit/undelete before data volume makes retrofitting expensive (**M28**).
- Complete or formally retire the `suppliers/providers/` registry to remove the dual-abstraction ambiguity (**M18**).
- Centralize repeated config (`APP_URL`, `isProduction`, `formatShortMonth`) in `config.ts`/`format.ts` and delete confirmed dead code (`session.ts`, `proxy.ts` or rename, `github-stars.tsx`) (**M17**, **L12-L17**).
- Run a single RTL refactor pass converting physical to logical Tailwind utilities across components, and add `loading.tsx` skeletons to the six list pages lacking them (**M24**, **L1.1**).
- Annotate all server actions with explicit `Promise<ActionResult>` return types to catch accidental return-shape drift in CI (**M19**).
- After schema index/FK changes, run `drizzle-kit generate && migrate` (never `push`, per AGENTS.md).

Remediation Applied (2026-06-28)

Implemented via parallel coder sub-agents, each owning a disjoint file set. Verification: `npx tsc --noEmit` clean · `npm run lint` 0 errors (48 pre-existing import-order warnings only) · `npx next build` all routes compile.

Critical

- **C1 — Fake-PNR fallback** — `duffel.ts` no longer fabricates a `DF-…` reference; `bookFlight` throws on provider failure. `bookings.ts` `confirmItemBooking` returns a discriminated result; failed items are set to `pending` (existing enum value), never `confirmed`. `advanceStatus` aborts the ticketing advance and returns `{ ok:false, reasons }` if any item failed. Files: `src/lib/suppliers/duffel.ts`, `src/lib/actions/bookings.ts`.
- **C2 — Stripe webhook retry storm** — DB writes wrapped in try/catch in both `webhook/route.ts` and `connect-webhook/route.ts`; misconfiguration in connect-webhook now returns 503 (was 400) so Stripe retries; signature failures stay 400; transient DB errors return 500 with logging.

High

- **H1** chat role enum restricted to `["user","assistant"]` (no client `system`).
- **H2** portal signout changed GET→POST + `<form method="POST">` (303 redirect).
- **H3** try/catch + `console.error` + friendly error returns added across 10 action files (clients, team, suppliers, invites, opportunities, portal-proposals, notifications, proposals-public, portal-invite, onboarding) and commissions.
- **H4** chat `streamText` + every tool `execute` wrapped; `createBooking` collects failed items/travellers instead of silently skipping.

- **H5** 15s `AbortController` fetch timeouts on Duffel, Hotelbeds, Stripe wrappers.
- **H6** segment `error.tsx` added for `portal/` (→ /portal/login), `(app)/` (→ /dashboard), `platform/` (→ /platform).
- **H7** dashboard counts now use `db.$count(...)` instead of fetch + `.length`.
- **H8** `nextReference()` uses a single `max(...)` aggregate.
- **H9** `recalcTotals()` N+1 UPDATEs replaced with `Promise.all`.
- **H10** Suspense around dashboard insights + new `finance/loading.tsx` skeleton.
- **H11** mobile drawer made RTL-aware via Tailwind `rtl:` variants.
- **H12** `role="alert"` added to all auth-form error messages.
- **H13** `verification_identifier_idx` added (in migration 0018, pending apply).

Medium / Low (included in tranche)

- **M1** commissions `recordCommission` now cross-tenant-validates `supplierId`/`bookingItemId`/`agentUserId` before insert.
- **M2** `escapeHtml()` applied to user-sourced values in email templates.
- **M6** `degraded`/`source` flag now part of `safeSearch` result type and forwarded to the AI chat tools (mock prices disclosable).
- **M12**/**M26**/**M27**/**L26**/**L27** composite + createdAt + supplierId indexes, commission cascade, and supplier/commission Drizzle relations — all in `drizzle/0018_regular_karnak.sql` (**pending apply** — see blocker above).
- **M30** `recalcTotal` now agency-scoped via join on `booking.agencyId`.
- **L12**/**L13**/**L14** deleted dead code: `src/lib/session.ts`, `src/proxy.ts` (dormant non-executing middleware; protections already enforced by `requireAgencyUser()`), `src/components/ui/github-stars.tsx`.

Deferred (explicitly out of this tranche — recommend follow-up)

- **M28** soft-delete (`deletedAt`) strategy across core entities.
- **M18** retire/complete the dual `suppliers/providers/` abstraction.
- **M24**/**M25** full RTL refactor (~156 physical→logical Tailwind utilities).
- **M7** structured logging/observability (Pino/Sentry).
- **L2** rate limiting on the magic-link request endpoint.
- Remaining Medium/Low UX, a11y, and perf items per the tables above.

Action required to fully close

1. Refresh `POSTGRES_URL` in `.env` (dev) with a valid Neon password, then run `npx drizzle-kit migrate` to apply `0018_regular_karnak.sql`.
2. Apply the same migration to **prod**: `POSTGRES_URL=<prod-url> npx drizzle-kit migrate`.

Production-Readiness Audit

Date: 2026-06-28 · **Auditor:** Lead Software Architect · **Scope:** security, error handling, performance, code quality, UX, database. No new features.

> **Note:** the brief mentioned "optimize Prisma" — Atlas uses **Drizzle ORM**, not Prisma. Drizzle findings are reported instead.

Summary

Atlas has **strong security foundations** — tenant isolation, authorization, and input safety are largely correct. The gaps are in **operational hardening** (rate limiting, error handling, loading states) and **reversibility** (no soft delete). One genuine horizontal-privilege-escalation issue was found **and fixed** during this audit.

Area	Verdict
Security	🟢 Solid; 1 escalation bug fixed, 2 hardening gaps
Error handling	🟡 Inconsistent try/catch; few error boundaries
Performance	🟢 Well-indexed; pagination missing
Code quality	🟢 Clean; minor lint/dup
UX	🟡 Loading/error/mobile gaps
Database	🟢 Good FKs/indexes; no soft delete

1. Security

#	Issue	Severity	Proposed fix	Status
S1	**Horizontal privilege escalation** — detail pages (`bookings/clients/opportunities/products/[id]`) were tenant-scoped (`agencyId`) but **not** agent-scoped, so an agent could open a colleague's record by direct URL.	**High**	Add agent own-scope (`createdById`/`ownerId`/`assignedToId`) to each detail query, gated by `seesAllData(role)`.	✓ **Fixed** (4 files this audit; list pages fixed earlier in `134ceb3`)
S2	**No rate limiting** on login, the public proposal token endpoints (`/p/[token]`, `acceptProposalByToken`), or API routes — brute-force / abuse vector.	**Medium**	Add an IP+route limiter (e.g. Upstash ratelimit) in middleware for auth + public token routes.	☐ Recommended

S3	**Demo credentials committed** in `docs/deployment.md` (incl. platform-admin).	**Medium**	Rotate the demo passwords; move them out of the repo or behind a private note.	☐ Recommended
S4	Supplier-detail booking-item count query not explicitly agency-scoped (logically safe via FK ownership).	**Low**	Add `agencyId` to the count `where` for defense-in-depth.	☐ Optional

Verified GOOD (no action):

- **Tenant isolation** — `requireAgencyUser()` on every `(app)` page (enforced in the layout) and every server action; `getSupplierById`, all `[id]` detail queries, and exports filter by `agencyId`. `scripts/test-tenant-isolation.ts` exists.
- **Authorization** — all 20 action files guard their exports (`billing` via a `requireBillingAdmin` wrapper; `platform` via `requirePlatformAdmin`; `platform.exitAgencyView` only clears cookies; `proposals-public` is token-authorized by design).
- **No SQL injection** — Drizzle parameterizes; no `sql.raw`/string-built queries.
- **No XSS sinks** — zero `dangerouslySetInnerHTML`; React auto-escapes.
- **Secrets** — `.env*` gitignored; no `.env` tracked; Stripe webhooks verify signatures.

2. Error Handling

#	Issue	Severity	Proposed fix
E1	Only **10/20** action files use `try/catch`; uncaught throws surface a generic Next error.	**Medium**	Standardize on the `ActionResult` return shape with user-friendly messages; wrap external calls (Stripe/Duffel/Hotelbeds/Resend) in try/catch.
E2	Only `assistant/` has an `error.tsx`; most routes have no error boundary.	**Medium**	Add `error.tsx` per route group (or one in `(app)/`) with a retry action.
E3	No structured logging; `logActivity` covers ~11/21 action files.	**Low**	Add a logger; extend `logActivity` to every mutation.

3. Performance

#	Issue	Severity	Proposed fix
P1	Hardcoded `limit` (200/500) on list queries — **silent truncation** and unbounded scan as data grows.	**Medium**	Real pagination (cursor/offset) — pairs with the planned `DataTable`.
P2	No obvious N+1 — queries use `with` joins and grouped counts.	—	None.

P3	Mostly Server Components (good); bundle not analyzed; no `next/dynamic` for heavy client islands (charts, search sheet).	**Low**	Run `@next/bundle-analyzer`; lazy-load chart/search client components.

DB is well-indexed (**53 indexes**) and Drizzle is parameterized — no slow-query red flags in static review (runtime profiling recommended).

4. Code Quality

#	Issue	Severity	Proposed fix
Q1	Redundant auth: `suppliers/[id]` calls `requireAgencyUser()` then `getSupplierById` calls it again.	**Low**	Drop the page-level call (helper already guards).
Q2	47 ESLint `import/order` warnings.	**Low**	`pnpm lint --fix` (auto-fixable).
Q3	No dead code or duplicated logic found; `src/lib` (domain/queries/analytics) and domain-foldered components are clean.	—	None.

5. UX

#	Issue	Severity	Proposed fix
U1	Loading states missing on most routes (only `assistant`/`dashboard`).	**Medium**	Add `loading.tsx` per route using the existing `skeleton.tsx`.
U2	Error states missing (see E2).	**Medium**	Add `error.tsx`.
U3	Tables overflow on mobile (no `overflow-x-auto`).	**Medium**	Wrap tables; ship the responsive `DataTable`.
U4	Accessibility not deeply audited.	**Low**	Run the web-interface-guidelines pass (focus order, labels, contrast).
U5	Empty states are present and filter-aware.	—	None (good).

6. Database

#	Issue	Severity	Proposed fix
D1	**No soft delete** (`deletedAt`) anywhere; FK cascades perform **hard** deletes — conflicts with the "everything is reversible" principle.	**Medium**	Add nullable `deletedAt` to core tables + filter reads (needs migration).
D2	`reference` only on `booking`/`product`; `updatedAt`/`status`/`notes` inconsistent on child tables.	**Low**	Backfill per the entity standard (needs migration).

D3	FKs (45 `onDelete`: 25 cascade / 20 set null), indexes (53), sequential migrations (latest `0017`), `generate→migrate` workflow.	—	**Verified good.**

7. Files modified (this audit)

- `src/app/(app)/bookings/[id]/page.tsx` — agent own-scope on detail query
- `src/app/(app)/clients/[id]/page.tsx` — agent own-scope on detail query
- `src/app/(app)/opportunities/[id]/page.tsx` — agent own-scope on detail query
- `src/app/(app)/products/[id]/page.tsx` — agent own-scope on detail query

(Earlier this session: list-page agent scoping `134ceb3`; quality-gate repair `7d5f4e1`.) Verified: `npm run check` (lint + `tsc`) passes clean.

Remaining recommendations (priority order)

1. **Rate limiting** (S2) — highest-value security add; small, middleware-level.
2. **Soft delete migration** (D1) — unblocks reversibility *and* GDPR erasure.
3. **Error boundaries + loading states** (E2/U1/U2) — mechanical, big UX win; `skeleton.tsx` already exists.
4. **Real pagination** (P1) — fold into the shared `DataTable` build.
5. **Rotate committed demo credentials** (S3).
6. **Standardize action error handling** (E1) on the `ActionResult` shape.
7. **Lint --fix + bundle analysis** (Q2/P3).

All items are tracked in the spec-vs-reality gap tracker.

Owner's-Eye UX Audit

Date: 2026-06-28 · **Lens:** "I own this travel agency and use Atlas all day." Walking every workflow looking for friction. **Recommendations only — nothing implemented.**

How to read severity: **P0** = trips me up daily / costs real time · **P1** = recurring annoyance · **P2** = polish.

Top findings (prioritized)

#	Issue	Sev	Why it hurts
1	**Opportunities page is orphaned from the nav**	P0	My core sales list isn't in the sidebar; I can only reach deals via a client or the "Operations" board. I lose my pipeline.
2	**"Operations" is really the sales pipeline**	P0	The label says fulfillment/ops; it's actually the deal board. I waste clicks hunting for where my pipeline lives — and it overlaps Opportunities + the Bookings board.
3	**Can't create a booking without first creating a client**	P0	A walk-in/phone booking forces me to stop, go create a client, come back. Two screens for one sale.
4	**Nav order contradicts daily work**	P1	Finance/Commissions/Reports sit *above* Bookings/Clients. My most-used screens are buried under money screens I open weekly.
5	**Three money destinations** (Finance, Commissions, Reports)	P1	I never know which to open for "how much did we make?" Overlapping, unlabeled difference.
6	**Fragmented sourcing** (Search vs Hotels vs Assistant vs inline)	P1	Four ways to look up inventory. I forget which finds what.
7	**Sales→Proposal is manual**	P1	Won an opportunity? I re-enter trip details into a new proposal. Duplicated work.
8	**Inconsistent terminology** (Proposal/Product, Opportunity, Operations)	P2	Same thing has different names across URL, page, and conversation.
9	**No nav grouping** (15 flat items)	P2	A wall of links; no Sales / Ops / Finance / Admin sections to scan.

Workflow walkthroughs

Finding new business → my pipeline (P0)

What I see: The sidebar has **Operations** (which opens a Pipeline kanban) but **no Opportunities link** — even though there's a full `/opportunities` page. So my deals live in two places with two names,

and the "real" one isn't in the menu.

Why it hurts: My pipeline is the heartbeat of the agency. Making me guess between "Operations" and a hidden Opportunities page costs clicks every single day and makes the tool feel unfinished.

Better workflow: One nav item — **"Sales"** or **"Pipeline"** — that opens the board with a list/board toggle (the toggle already exists on Bookings). Retire the "Operations" name. One pipeline, one entry point.

Taking a booking (P0)

What I see: On *New booking*, if the client doesn't exist I get "No clients yet. Add a client first" — I must leave, create the client, and return.

Why it hurts: Half my bookings start as a phone call from someone not yet in the system. The client-first rule is right for *data*, but it shouldn't be a *detour*. That's two screens and a lost context switch for one sale.

Better workflow: Inline " + New client" inside the booking form (a quick create that returns the new client selected), or a combined "New booking" that captures a minimal client on the same screen. Create the client record *as a side effect* of the sale, not a prerequisite step.

Building the quote (P1)

What I see: I win an opportunity, then open *New proposal* and re-type the destination, dates, and value I already captured on the opportunity. Convert proposal→booking is one click (great) — but opportunity→proposal isn't.

Why it hurts: Re-entering the same trip details is duplicated work and a place for transcription errors. It breaks the "never ask twice" principle right in the middle of my money-making flow.

Better workflow: A **"Create proposal" button on the opportunity** that pre-fills client, destination, dates, and value (the AI quote builder already exists — wire it to the opportunity). Won → proposal in one click, mirroring proposal → booking.

Seeing the money (P1)

What I see: Finance, Commissions, and Reports are three separate nav items, consecutive, with no stated difference.

Why it hurts: As the owner the #1 question is "how are we doing?" Three doors for that answer means I learn-and-forget which is which.

Better workflow: Group them under one **"Finance"** section with sub-tabs (Overview · Commissions · Export). One mental model, drill in for detail.

Sourcing flights/hotels (P1)

What I see: A **Search** page, a separate **Hotels** module, the **Assistant**, and an inline search sheet on the booking — four entry points.

Why it hurts: I can't form a habit. "Where do I price a hotel?" has four answers depending on context.

Better workflow: Make the **in-booking search the primary path** (I'm almost always sourcing *for a trip*), and frame standalone Search/Hotels as "explore without a booking." Fold the Assistant's search into the same sheet.

Terminology inconsistencies (P2)

Same concept, different words across URL / page title / everyday speech:

Concept	URL	UI label	Agency speak	Fix
Quote/ offer	`/products`	"Proposals"	"quote" / "proposal"	Rename route+code to `proposals`; pick one word everywhere.
Sales pipeline	`/operations` + `/opportunities`	"Operations" + (none)	"pipeline" / "deals" / "enquiries"	One name ("Pipeline"), one page.
Deal	`/opportunities`	"Opportunity"	often "enquiry" / "lead"	Confirm the agency's word; use it consistently.

Mismatched names make training new staff slower and make the product feel stitched together.

Missing automation (P1 — productivity multipliers)

As an owner I want the system to do the obvious next step:

- New client → **welcome email** (not sent today)
- Opportunity won → **draft proposal** (manual today)
- Booking confirmed → **invoice** (on-demand PDF today)
- Travel completed → **review request** (not built)

Each is a manual chore I'll forget under load. These are the automation triggers — building them is the difference between a record-keeping tool and one that runs the agency.

What already feels good (keep it)

- **Getting-started checklist** — I knew what to do on day one.
- **Booking lifecycle stepper** with "Advance to next" — I always know the state and the next action. This is exactly the "what do I do next?" principle done right.
- **Convert proposal → booking** in one click — more of this, please.
- **Client timeline** (activity + payments + notifications in one view) — this is how I think about a client.

- **Empty states with a CTA** — never a dead end.

Recommended sequence

1. **Fix the pipeline confusion** (#1, #2) — rename/merge Operations + Opportunities into one "Pipeline" nav item. Cheap, high daily relief.
2. **Inline client create in booking** (#3) — removes the worst detour.
3. **Reorder + group the nav** (#4, #9) — daily work on top, grouped sections.
4. **Opportunity → proposal generation** (#7) — wire the existing AI quote builder.
5. **Consolidate Finance** (#5) and **terminology pass** (#8).
6. **Automation triggers** (welcome email, invoice, review) as capacity allows.

These are UX/workflow recommendations; the engineering gaps behind some of them (automation, pagination, DataTable) are tracked in the gap tracker.

CTO Review — Atlas

Date: 2026-06-28 · **Reviewer:** CTO (pre-first enterprise customer) **Verdict:** Atlas is a well-structured, ambitious product with real engineering quality — but it is not enterprise-ready today. Several issues would cause visible failures under real-world load or scrutiny. This report is honest, not diplomatic.

Executive Summary

Dimension	Grade	One-line verdict
Architecture	B+	Solid Next.js 16/Drizzle/Neon foundation; DB connection model wrong for serverless
Security	B	Tenant isolation real; no middleware, no CSP, no rate limiting
Database	B+	Good schema; soft delete missing; 11 tables missing `updatedAt`
Folder structure	A-	Clean and predictable; minor naming confusion (products = proposals)
UI	B	Design system consistent; no loading/error states; mobile overflow
UX	C+	Core flows work; 3 P0 navigation problems; no automation
Business logic	B	Core RBAC solid; reference generation has race condition
Performance	C+	Finance page does 4 sequential awaits; DB connection wrong driver
Scalability	C	No connection pooler; no caching layer; no queue; single DB point of failure
API design	B-	REST-style routes adequate; no versioning; chat endpoint has no body size limit
Developer experience	C+	No CI/CD; one test file; no seed script for fresh dev; two DB drivers in package.json
Maintainability	B	`bookings.ts` at 976 lines needs splitting; products/proposals naming confusing
Testing	D	One test file in the entire codebase. Not acceptable for enterprise.
Deployment	B-	Vercel is fine; no health endpoint; no rollback plan documented
CI/CD	F	Zero — no `.github/workflows/`. No automated checks before merge.
Documentation	A-	Unusually thorough for this stage; gap tracker is a standout
Code quality	B+	Consistent patterns; 45 raw `console` calls instead of a real logger

---

CRITICAL ISSUES

Must be fixed before the first enterprise customer goes live.

C1 — No database connection pooler

File: `src/lib/db.ts`

```
const client = postgres(connectionString); // wrong driver for serverless
```

Atlas uses `postgres` (the `postgres.js` driver) on Vercel serverless functions. Every function invocation opens a **new TCP connection** to Neon. At 50 concurrent users that's 50 open connections. At 200 it exhausts Neon's connection limit and every query starts failing with "too many clients."

Neon provides a **serverless pooler** (`-pooler` suffix in the connection string) that multiplexes thousands of connections through ~10 persistent ones — but you need to switch to the **`@neondatabase/serverless`** driver to use it correctly over HTTP (not TCP), which is mandatory for Vercel edge/serverless.

You also have **two DB drivers** (`postgres` + `pg`) in `package.json` with no clear owner — `pg` is for Drizzle migrations, `postgres` for runtime. This needs to be explicit and documented.

Impact: 200 concurrent users = database down. Enterprise customer demos in front of their IT team = connection pool exhaustion at the worst moment.

Fix:

```
npm install @neondatabase/serverless
```

```
// src/lib/db.ts
import { neon } from "@neondatabase/serverless";
import { drizzle } from "drizzle-orm/neon-http";
const sql = neon(process.env.POSTGRES_URL!);
export const db = drizzle(sql, { schema });
```

Keep `pg` only for `drizzle-kit` migrations (it needs a real TCP connection).

C2 — Zero CI/CD pipeline

Finding: No `.github/workflows/` directory. Nothing runs automatically on `git push`.

This means:

- A broken build can be pushed to `main` and deployed to production instantly
- No automated type checking, linting, or tests before deploy
- The quality gate (`npm run check`) only runs if someone remembers to run it
- No way to enforce code review before merge

Impact: The first enterprise customer will eventually hit a regression caused by an unreviewed push.

At that point you have no safety net and no audit trail.

Fix: Create `.github/workflows/ci.yml`:

```
name: CI
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm install --legacy-peer-deps
      - run: npm run check       # lint + typecheck
      - run: npm run build:ci    # next build
```

Vercel already deploys on push — CI just adds the gate before it reaches Vercel.

C3 — No test coverage

Finding: One test file exists: `.claude/worktrees/*/src/lib/references.test.ts` — not even in the main source tree. Zero tests for:

- Server actions (RBAC, tenant scoping, business logic)
- API routes (auth, input validation)
- Domain logic (`domain.ts` — roles, capabilities, booking lifecycle rules)
- Utility functions

Impact: Any refactor silently breaks business rules. Enterprise customers expect vendors to have a test suite. Without it, you cannot safely hire additional engineers (they will introduce bugs you won't catch).

Fix (prioritized, not everything at once):

1. Integration tests for the 5 most critical server actions: `createBooking`, `advanceBookingStatus`, `acceptProposalByToken`, `createCommissions`, `createAgency`
2. Unit tests for `domain.ts` (RBAC logic is pure functions — trivial to test)
3. Tenant isolation test already exists as a script (`test-tenant-isolation.ts`) — promote it to the CI pipeline

---

C4 — No middleware (rate limiting, security headers enforcement)

Finding: No `src/middleware.ts`. This means:

- Login endpoint (`/api/auth/sign-in`) has no rate limiting — brute-forceable
- Public proposal endpoint (`/p/[token]`) has no rate limiting
- Portal magic-link request has no rate limiting — email flooding

- Security headers (set in `next.config.ts`) work but a middleware could short- circuit abuse before it hits server functions

CSP (`Content-Security-Policy`) is also absent from `next.config.ts`. Every major browser security audit flags missing CSP — an enterprise customer's security team will catch this immediately.

Impact: Any motivated attacker can brute-force login, flood the magic-link email, or scrape tokenized public proposals. An enterprise security review fails.

Fix:

```
// src/middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
// Use Upstash ratelimit (Redis-backed, works on Vercel Edge)
export function middleware(req: NextRequest) {
  // 1. Rate limit /api/auth and /api/portal/auth/request
  // 2. Add Content-Security-Policy header
  return NextResponse.next();
}
export const config = { matcher: ["/api/auth/:path*", "/api/portal/:path*"] };
```

C5 — Race condition in booking/proposal reference generation

File: `src/lib/actions/bookings.ts:55-68`

```
// Fetches ALL booking references for the agency, iterates in JS to find max
const rows = await db.select({ reference: booking.reference })
  .from(booking).where(eq(booking.agencyId, agencyId));
let max = 1000;
for (const r of rows) {
  const n = Number.parseInt(r.reference.replace(/\D/g, ""), 10);
  if (Number.isFinite(n) && n > max) max = n;
}
return `BKG-${max + 1}`;
```

At 10,000 bookings per agency this loads the full reference list into memory on every creation. Worse: two simultaneous booking creations can read the same max and produce duplicate references (unique constraint catches it, but the UX is a silent retry loop, not an atomic increment).

Impact: Agencies with high booking volume (enterprise customers) hit slow booking creation and occasional silent retries. Could silently fail under load.

Fix: Atomic SQL max:

```
import { max as sqlMax } from "drizzle-orm";
const [row] = await db
  .select({ max: sqlMax(booking.numericReference) })
  .from(booking)
  .where(eq(booking.agencyId, agencyId));
```

```
const next = (row?.max ?? 1000) + 1;
return `BKG-${next}`;
```

Requires adding a `numericReference` integer column (same migration as soft delete).

HIGH PRIORITY

Fix within the first sprint after launch.

H1 — No soft delete

File: `src/lib/schema.ts`

No `deletedAt` column exists anywhere. The "everything is reversible" founding principle is aspirational only. An agency that accidentally deletes a booking has no recovery path. This also blocks GDPR erasure (you can't "erase" what you can't soft-delete and then hard-purge on schedule).

Fix: Migration adding `deletedAt: timestamp("deleted_at")` to core tables (booking, client, opportunity, product), filter all reads with `.where(isNull(entity.deletedAt))`, and a restore action per entity.

H2 — `requireAgencyUser` makes a DB query on EVERY page render

File: `src/lib/permissions.ts:117-132`

```
const ag = await db.query.agency.findFirst({
  where: eq(agency.id, user.agencyId),
  columns: { status: true, subscriptionStatus: true },
});
```

This query fires on every authenticated server render. At 1,000 concurrent users that's 1,000 extra agency-status queries per second. The agency status doesn't change often — it's a perfect cache candidate.

Fix: Cache the agency status in the session token (updated on suspension/ reactivation) or use `unstable_cache` with a short TTL and `revalidateTag` on suspension events.

H3 — Finance page does 4 sequential DB queries (633 lines)

File: `src/app/(app)/finance/page.tsx`

The page is enormous (633 lines) and makes 4+ DB queries, some sequential. It will be slow as the dataset grows and impossible to maintain.

Fix: Split into smaller async components using React Suspense, parallelizing queries with `Promise.all` or per-section server components. Cap the page at 150 lines of layout; extract `<PaymentsSummary>`, `<CommissionsSummary>`, etc.

H4 — No structured logging (45 raw `console.*` calls in actions)

Finding: 45 `console.log/error/warn` calls scattered across action files. On Vercel these show up as unstructured text in Function Logs with no correlation IDs, no severity levels, no timestamps, no request tracing.

Impact: When an enterprise customer files a support ticket saying "booking creation failed at 14:32 UTC," you cannot find the relevant log entry.

Fix: A one-file logger:

```
// src/lib/logger.ts
export const logger = {
  info: (msg: string, meta?: object) =>
    console.log(JSON.stringify({ level: "info", msg, ...meta, ts: new Date().toISOSt
  error: (msg: string, meta?: object) =>
    console.error(JSON.stringify({ level: "error", msg, ...meta, ts: new Date().toIS
};
```

Replace all `console.*` calls. Ship a correlation ID from the request context.

H5 — `bookings.ts` is 976 lines — unmaintainable

File: `src/lib/actions/bookings.ts`

Nearly 1,000 lines covering booking CRUD, lifecycle advancement, traveller management, item management, itinerary, and commission generation. A new engineer cannot reason about it. Every bug fix risks breaking something else in the file.

Fix: Split into:

- `bookings/crud.ts` — create, update, delete
- `bookings/lifecycle.ts` — advance status, prerequisites
- `bookings/travellers.ts` — passenger CRUD
- `bookings/items.ts` — trip service items
- `bookings/commissions.ts` — commission generation

H6 — No health check endpoint

Finding: No `/api/health` route. Vercel's uptime monitors, load balancers, and the enterprise customer's infrastructure team expect one.

Fix:

```
// src/app/api/health/route.ts
export async function GET() {
  return Response.json({ status: "ok", ts: new Date().toISOString() });
}
```

H7 — Products are called "proposals" everywhere but the URL and the DB

Finding: The concept is **Proposal** in all UI, docs, and conversation. But the database table is `product`, the URL is `/products`, the actions file is `products.ts`, the type is `ProductInput`. An enterprise customer's developer integrating via the API will be confused, and new engineers will be too.

Fix: Rename — DB table to `proposal`, route to `/proposals`, action file to `proposals.ts`. One migration, one find-replace. Do it now before the codebase gets larger.

H8 — No input validation on chat endpoint (body size / message length)

File: `src/app/api/chat/route.ts`

No body size limit, no message length cap, no token budget per request. A user (or attacker) can send a 100KB message that gets passed to the LLM, costing significant API spend.

Fix: Add `export const maxDuration = 30` (Vercel), validate message array length and individual message size before calling the LLM, and add the rate limiter from C4.

MEDIUM PRIORITY

Address in the first 60 days.

M1 — 11 tables missing `updatedAt`

11 of 24 tables have no `updatedAt` — impossible to answer "when was this record last changed?" for audit, support, or sync scenarios.

M2 — No caching layer whatsoever

Dashboard, finance, analytics — every render hits the DB fresh. `unstable_cache` with `revalidateTag` on mutations would cut DB load by ~60% for read-heavy pages.

M3 — Opportunities page not in navigation

The core sales pipeline (`/opportunities`) is not in the sidebar — agents can only reach it via the operations board or a client page. Discovered in the owner UX audit.

M4 — "Operations" label wrong

`/operations` is the pipeline kanban — "Operations" implies fulfillment/logistics. Enterprise buyers will be confused during demos.

M5 — No background jobs / queue

Sending emails, generating PDFs, syncing hotel content, processing webhooks — all done synchronously in request handlers. Slow operations block the user. Vercel serverless has a 10s default timeout.

M6 — Amadeus marked as "legacy" but still in the active provider selection

`getFlightSupplier()` still falls back to Amadeus. The codebase comment says "decommissioned 2026-07-17". Remove it before it becomes a dead code maintenance burden.

M7 — No pagination on any list page

All list queries use hardcoded `LIMIT 200` or `LIMIT 500`. An agency with 5,000 clients or 10,000 bookings silently gets a truncated list with no indication that data is missing. This is a data integrity perception problem for enterprise.

M8 — Portal session expiry not enforced on the server

The portal session expiry check (`gt(portalSession.expiresAt, new Date())`) exists but the TTL is not clearly documented and there's no cleanup job to remove expired sessions. Old sessions accumulate forever.

M9 — No GDPR compliance path

No data export per client, no erasure flow, no data retention policy, no consent tracking. EU travel agencies (your primary market) are legally required to provide these. Blocked by missing soft delete (H1).

M10 — Two PostgreSQL drivers in package.json

`postgres` (runtime) and `pg` (drizzle-kit migrations) coexist with no explanation. Confusing for new engineers and a potential version conflict risk.

NICE-TO-HAVE

Post-stabilization improvements.

N1 — No PR template or contribution guidelines

No `PULL_REQUEST_TEMPLATE.md`, no `CONTRIBUTING.md`. Enterprise customers who want to contribute integrations or customizations have no guidance.

N2 — `picsum.photos` in production image whitelist

`next.config.ts` whitelists `picsum.photos` (lorem ipsum placeholder images) in production. This is a dev-only domain — remove it from production config.

N3 — `OPENROUTER_MODEL` defaults to `openai/gpt-5-mini`

The env default in `env.ts` references `gpt-5-mini` — a model that may not exist or may be deprecated. This should be an explicit, versioned model ID.

N4 — No API versioning

All API routes are unversioned (`/api/chat`, `/api/export`). When you change the chat protocol or export format, existing integrations break with no migration path. Add `/api/v1/` prefixing now while there are no external consumers.

N5 — Finance page at 633 lines is both a performance and maintainability problem

Covered in H3 — also a nice-to-have style cleanup even after the Suspense split.

N6 — No Storybook or component documentation

UI components have no visual documentation. When a designer or new frontend engineer joins, they have no way to browse the component library.

N7 — `PROTECTED_DB_HOSTS` is a soft guard

The protection against running destructive scripts against prod is a hostname string comparison. It can be bypassed. Not critical today, but consider a proper DB IAM role with no DROP/TRUNCATE in production.

What is genuinely good

I want to be clear: most codebases I review at this stage are far messier. These are the things that show real engineering judgment:

- **Tenant isolation** is correctly implemented and tested — `agencyId` on every table, enforced in every query, with a script that verifies no leaks.
- **`ActionResult<T>`** is a clean, consistent error-handling contract.

- **Zod validation** exists and is used, even if not universal.
- **The domain model** (`domain.ts`) is well-organized — roles, capabilities, enums, status metadata in one place.
- **The provider abstraction** (`src/lib/suppliers/`) is well-designed — the new `providers/` layer with capability-segmented interfaces is production-quality.
- **Security headers** in `next.config.ts` (X-Frame-Options, X-Content-Type-Options, Referrer-Policy) — most startups forget these.
- **The documentation** is the best I have seen at this stage. The gap tracker, the owner UX audit, the architecture decisions — this saves weeks of onboarding.
- **`env.ts`** validates all environment variables at startup with Zod — catches misconfigured deploys immediately.

Priority matrix

Priority	Issue	Effort	Risk if ignored
Critical	C1: No connection pooler	1 day	DB down at 200 users
Critical	C2: No CI/CD	1 day	Broken builds reach prod
Critical	C3: No tests	2 weeks	Regressions on every change
Critical	C4: No middleware/rate limiting	2 days	Brute force, abuse, CSP fail
Critical	C5: Reference race condition	2 hours	Duplicate refs under load
High	H1: No soft delete	3 days	Unrecoverable deletes, GDPR block
High	H2: Agency query on every render	4 hours	Slow at scale
High	H3: Finance page 633 lines	2 days	Slow, unmaintainable
High	H4: No structured logging	4 hours	Blind in production
High	H5: bookings.ts 976 lines	2 days	Bugs, slow PRs
High	H6: No health endpoint	30 min	Monitoring blind
High	H7: products = proposals naming	1 day	Developer confusion forever
High	H8: Chat no input limit	2 hours	LLM cost abuse
Medium	M1–M10	Various	Quality and compliance debt

Recommended 30-day plan before enterprise onboarding

Week 1 — Non-negotiable hardening

- C1: Switch to `@neondatabase/serverless` driver
- C2: Add GitHub Actions CI (lint + typecheck + build)
- C4: Create `middleware.ts` with rate limiting + CSP header

- C5: Fix reference generation with atomic SQL max
- H6: Add `/api/health` endpoint

Week 2 — Reliability

- H1: Soft delete migration for core tables
- H4: Replace `console.*` with structured logger
- H8: Chat endpoint body size + rate limit
- H2: Cache agency status

Week 3 — Code health

- H5: Split `bookings.ts` into 5 files
- H7: Rename products→proposals (DB + routes + components)
- M3/M4: Fix navigation (add Opportunities, rename Operations)
- C3: Start: integration tests for 5 critical actions

Week 4 — Enterprise readiness

- M9: Skeleton GDPR data export per client
- M5: Evaluate Vercel background tasks or QStash for async work
- H3: Split finance page with Suspense
- C3: Continue test coverage

This plan gets Atlas from "impressive prototype" to "defensible enterprise product" in 30 days. The bones are good. The hardening is what separates a product people trust with their revenue from one they pilot and abandon.

Decision 0001 — Navigation IA restructure

Date: 2026-06-28 **Status:** accepted **Deciders:** Product owner + CTO review

Context

Atlas had 15 flat navigation items in an order that contradicted the golden workflow (Finance appeared above Clients; Opportunities was entirely missing from the nav; Search and Hotels were two separate items for the same user task; Operations was a duplicate of the Bookings board view). An IA audit (docs/owner-ia-audit.md) identified these as P0/P1 usability issues.

Decision

Restructure navigation into 5 labelled sections following the golden workflow: **WORK** (Clients → Pipeline → Proposals → Bookings) · **SOURCING** (Flights · Hotels) · **FINANCE** · **TOOLS** · **ADMIN**. Rename `/products` → `/proposals` canonically (URL rewrite). Retire Operations as a nav item. Create `/sourcing/flights` as a dedicated page.

Reasoning

- Section-grouped nav reduces cognitive load from 15 flat items to 4-5 scannable groups
- Workflow order (Clients first, Finance last) matches how agents think
- Sourcing section scales cleanly to Cars, Cruises, Packages without nav restructure
- `/proposals` label matches what agents, clients and every competitor call the concept
- Operations was a duplicate of the Bookings board toggle — removing it eliminates confusion

Alternatives considered

Option	Why rejected
Keep flat nav, just reorder	Doesn't solve the 15-item cognitive load or the Search/Hotels duplication
Merge Search + Hotels into one page	Reasonable but defers the Sourcing section scaling problem
Rename Operations → Pipeline	The real pipeline is Opportunities; Operations was a board view, not a pipeline

Consequences

- Easier: adding new sourcing verticals (Cars, Cruises) — one line in NAV_SECTIONS
- Easier: onboarding new agents — nav reads like the job

- Harder: old `/products` and `/search` bookmarks need redirects (added to next.config.ts)
- Old URLs preserved via 301 redirects (products) and 307 redirects (search, hotels, operations)

Related

- docs/owner-ia-audit.md (full IA analysis)
- src/components/app/app-shell.tsx (implementation)
- next.config.ts (rewrites + redirects)
- src/app/(app)/sourcing/flights/page.tsx (new page)

Decision 0002 — Agency Network is a Stage 3 Direction

Date: 2026-06-28 **Status:** accepted (direction), deferred (implementation) **Deciders:** Founding team + board

Context

Board feedback on the strategic audit introduced the concept of Atlas as infrastructure connecting agencies to each other, not just software serving agencies individually. Agency A cannot fulfil a complex request; Atlas surfaces Agency B — a specialist — who can. Revenue sharing, inventory exchange, trusted subcontractor relationships, shared local suppliers. Network effects from participant relationships, not code.

This was not in the original strategic audit and represents a meaningfully stronger moat than data accumulation alone. Network effects in B2B do not decay; an agency earning revenue through Atlas referrals will not leave the platform.

Decision

Name this as the Stage 3 strategic direction. Begin designing for it in the data model now at minimal cost. Delay the product surface until Atlas has 300+ agencies (Stage 3 threshold).

Reasoning

Network effects require critical mass before generating value. At 50 agencies, the network is too thin for meaningful referral flow. Building the product surface before critical mass exists creates engineering overhead without corresponding user value and fragments attention from the supplier booking and automation work that must happen first to reach the scale where the network becomes possible.

Designing for it in the data model costs almost nothing. Agency records should carry specialisation attributes, destination expertise tags, and capacity signals from the beginning — these enable future matching without requiring a current product surface.

What this means now (Stage 1–2)

- Agency schema should include `specialisations`, `destinations_served`, and `agency_type` as structured fields (not free text) to enable future matching
- Activity and booking data should be structured to allow anonymised capability inference over time
- Do not build any product surface for inter-agency connection yet

- Do not market this capability yet — announcing a network that does not exist creates expectation debt

What this means at Stage 3 (300+ agencies)

- Surface: "Agency B specialises in this destination and has capacity"
- Revenue sharing mechanism for fulfilled referrals
- Trusted subcontractor relationships formalised in the platform
- Inventory and resource exchange between agencies

Alternatives considered

Option	Why deferred/rejected
Build the network now	Critical mass is required; building before scale creates overhead without value
Never build the network	This is the strongest long-term moat available; intentionally deferring is different from discarding
Build it as a marketplace immediately	Marketplace dynamics require both sides; agencies are not yet numerous enough to form a functioning marketplace

Consequences

- Stronger: when Atlas reaches 300 agencies, the network surface can be activated on data infrastructure already in place
- Harder: resisting the temptation to build this early when it is clearly a good idea
- Unchanged: no current engineering work required beyond schema attributes

Related

- docs/strategic-audit.md — the original moat analysis this extends
- docs/strategy-board-response.md — the board feedback that introduced this direction
- docs/domain.md — agency entity where specialisation attributes should be added

Decision 0003 — Sprint 1 Booking Architecture

Date: 2026-06-28 **Status:** accepted **Deciders:** engineering (Atlas Sprint 1)

Context

Atlas had functional search (Duffel flights, Hotelbeds hotels) but no complete booking workflow. Supplier references were stored as untyped JSONB. There was no price revalidation, no idempotency, no analytics event stream, and no clear environment separation between development, sandbox, and production. The `BookingProvider` interface existed in `providers/types.ts` but nothing used it.

The goal: build the full booking architecture in sandbox so that production credentials can be plugged in with configuration changes only — no code changes.

Decision

Build a layered booking architecture: **config → registry → provider adapters → booking services → server actions → UI**. Each layer is independent and testable. The UI never sees supplier types. Environment is resolved once in a config module.

Reasoning

Why consolidate to the new `BookingProvider` interface now

The legacy `SupplierProvider` (search-only, both verticals in one class) was a stepping stone. The new interface is capability-segmented and multi-step (search → quote → book → cancel). Building Sprint 1 on top of the new interface means provider adapters written now are the same ones production uses — no "sandwich migration" later.

Why four new database tables

`booking_supplier_ref` — supplier references must be queryable (status polling, cancellation, reporting). Storing them in JSONB is schema-less and unindexable.

`booking_event` — `activity_log` is too coarse for a booking funnel. A dedicated append-only event table serves as both the compliance audit trail and the analytics source without a third-party SDK dependency. One write, two consumers.

`booking_document` — vouchers and e-tickets are not line items. Separating them into their own table allows typed retrieval and future Vercel Blob integration.

`booking_idempotency` — without key tracking, a browser retry or a server-action re-execution after a timeout can create duplicate supplier orders. The key is derived deterministically so it is stable across retries for the same booking attempt.

Why a single config module (`suppliers/config.ts`)

Scattered `isDuffelConfigured()` / `process.env.*` checks across multiple files make it impossible to reason about which provider is active. A single config module resolves `EnvironmentTier` once and returns typed `FlightProviderConfig` / `HotelProviderConfig` objects. Providers receive config objects — they never read `process.env` directly.

Why in-process retry (no job queue)

Sprint 1 is sandbox-only. Async job queues (pg-boss, BullMQ) add operational complexity that is not justified until production traffic. The `BookingService` retry handler is synchronous and typed by error code (retryable errors get exponential backoff; non-retryable errors surface immediately to the UI).

Why post-booking stubs (not omitted)

Defining `cancel`, `refreshStatus`, `getVoucher`, `modifyPassengers` as typed interfaces with `NotImplementedError` now means Sprint 2 only fills in implementations — the UI is already wired against stable types. Omitting the interfaces forces Sprint 2 to design and implement simultaneously.

Alternatives considered

Option	Why rejected
Continue using `SupplierProvider` (legacy)	Would require a second migration when production arrives; two abstraction layers for the same concern
Store supplier refs in `booking_item.details` JSONB	Unqueryable, no index, no schema guarantee, impossible to cancel/poll
Third-party analytics SDK (PostHog, Segment)	External dependency with no offline fallback; `booking_event` table is queryable and sufficient for Sprint 1
Redis for idempotency keys	Redis is not in the stack; DB table with TTL expiry achieves the same guarantee without a new service
Async job queue for retry	Premature for sandbox; adds ops burden before production traffic justifies it

Consequences

Easier:

- Adding a new provider (TravelgateX, Expedia) requires one new adapter file and one `register()` call — no changes to services, actions, or UI.
- Switching sandbox → production is `ATLAS_ENV=production` + production tokens.
- Booking funnel analytics are queryable from day one via `booking_event`.
- Duplicate booking prevention is guaranteed at the DB level (idempotency PK).

Harder:

- The `booking_idempotency` table needs a periodic cleanup job (expired rows). Acceptable trade-off; a simple cron (`DELETE WHERE expires_at < NOW()`) suffices.
- Post-booking stubs surface `NotImplementedError` in the UI — must be surfaced gracefully ("Coming soon") rather than silently swallowed.

Related

- `src/lib/suppliers/config.ts` — environment config module (new, Sprint 1)
- `src/lib/suppliers/providers/types.ts` — `BookingProvider` interface (existing)
- `src/lib/suppliers/providers/registry.ts` — provider registry (existing, now wired)
- `src/lib/schema.ts` — schema additions (migration 0019)
- `docs/database.md` — updated table and migration entries
- `docs/api-integrations.md` — updated provider status

Decision 0004 — Travel Platform facade over direct supplier access

Date: 2026-06-29 **Status:** accepted **Deciders:** Engineering

Context

After Sprint 1 wired the booking lifecycle through the provider registry, the search / content / autocomplete path still called provider-specific functions directly from server actions and page components:

- `getFlightSupplier()`, `getHotelSupplier()`, `safeSearch()` hardcoded in `src/lib/actions/search.ts` (444 lines of Hotelbeds-specific orchestration)
- `isDuffelConfigured()`, `isHotelbedsConfigured()` scattered across 5 page files
- `searchDuffelPlaces`, `getHotelbedsContentBatch`, `searchHotelbedsHotelsByName`, etc. imported directly into the action layer

Adding a second search provider (RateHawk, Expedia, Booking.com) would require editing every consumer file. There was also no single home for the "availability search → content fallback → thumbnail enrichment" orchestration logic.

Decision

Introduce `src/lib/travel-platform/index.ts` as the single facade for all travel operations. Server actions and pages import from this module only; no provider-specific code leaks into the action or page layer.

Reasoning

- Keeps the registry pattern consistent: booking was already fully abstracted; search should be too.
- One module to update when a new provider is added. Zero consumer changes.
- The hotel search orchestration (availability → content fallback → enrichment) was duplicated and implicit in `search.ts`; moving it to the facade makes it explicit, testable, and shared.
- Preserves the exact same return shapes (`source`, `degraded`, `estimatedPricing`) so all UI components required zero changes.

Alternatives considered

Option	Why rejected

Move Hotelbeds orchestration into `HotelbedsBookingProvider.searchHotels()`	Cleaner long-term but larger surface area; the availability→content→enrichment flow spans three Hotelbeds APIs with separate quota budgets — better owned by the facade, which can route each step to the best provider
Keep `safeSearch` and just add guards	Provider-specific code stays in the action layer; adding provider #3 still requires editing `search.ts`
Full rewrite of legacy `suppliers/index.ts`	Sprint constraint: preserve working code, only add the abstraction layer

Consequences

Easier:

- Adding a new provider (Expedia, RateHawk, Booking.com) requires only: implement the relevant capability interfaces, call `providerRegistry.register()` — zero consumer changes.
- Hotel search orchestration (content fallback, enrichment) is in one place and can be unit-tested.
- `isFlightProviderConfigured()` / `getActiveFlightProvider().label` replace provider-specific checks in pages.

Harder / trade-offs:

- The content fallback path still calls `listHotelOffersCached` (DB cache) and falls back to `ContentCapable.searchHotelsByName` with city-as-query, which is an approximation. A future `ContentCapable.searchHotelsByDestination` would be more precise.
- The legacy `src/lib/suppliers/index.ts` layer (`getFlightSupplier` / `safeSearch`) remains in the codebase for backward compatibility — two layers until it is removed in a future cleanup sprint.

Related

- `src/lib/travel-platform/index.ts` — the facade
- `src/lib/suppliers/providers/` — registry, types, adapters (Duffel, Hotelbeds, Mock)
- `docs/api-integrations.md` — updated with Travel Platform section
- `docs/decisions/0003-booking-architecture-sprint1.md` — Sprint 1 booking registry

Phase 1 — "Sellable" Implementation Plan

Goal: take Atlas from demo to sellable. Seven features, dependency-ordered.

Decisions locked:

- **Billing:** vendor bills agencies (SaaS subscriptions) **now**; agencies bill travelers (Stripe Connect) **later**.
- **Email:** Resend · **Flights:** Amadeus · **Hotels:** Hotelbeds.

Key findings (much is already partially built)

- **Email** — a SendGrid adapter already exists at `src/lib/notifications/email.ts` with the right `sendEmail()`/`EmailResult` contract. Resend = one-file swap. Gaps: invites never actually send (toast is cosmetic, delivery is copy-paste); both Better Auth hooks (`auth.ts:85-97`) only `console.log`.
- **Stripe** — a fetch-based Checkout integration already exists for *traveler* booking payments (`src/lib/payments/stripe.ts`); `STRIPE_SECRET_KEY` is in `.env`. SaaS subscriptions are a **separate billing plane** on the `agency` table.
- **Travel APIs** — `SupplierProvider` abstraction exists with a working **Amadeus implementation for flights AND hotels** (`src/lib/suppliers/`). Flights ~done.
- **Bug** — `products/[id]/page.tsx:61` links to `/products/${id}/proposal`, a route that doesn't exist (real one is `/proposal/[id]`). Fix during PDF/e-sign work.

Sequence

```
1. DB split          — foundation; before billing & real customers
2. Email (Resend)    — unblocks invites, billing receipts, e-sign, proposals
3. Stripe SaaS billing — needs email + agency gating
4. PDF proposals ──┐  — pair
5. E-signature   ──┘    e-sign needs public token + email
6. Hotelbeds + flights  — parallel track, independent of 1-5
```

---

1. Separate databases — S (operational)

- Route `src/lib/db.ts:5-12` through `getServerEnv()` (currently bypasses zod).
- Add prod guard to destructive scripts (`seed-demo-data.ts`, `db:reset`, `test-tenant-isolation.ts`) — refuse unless `ALLOW_PROD=1` / host ≠ prod.
- User action: create a dev Neon branch; prod `POSTGRES_URL` only in Vercel env.
- Vercel `build:ci` skips migrations — confirm out-of-band migrate process.

2. Email delivery (Resend) — S

- Rewrite `src/lib/notifications/email.ts` to call Resend, keep the signature (zero caller changes); add optional `html?` for React Email later.
- Add `RESEND_API_KEY` + `EMAIL_FROM` to `src/lib/env.ts` (schema + `checkEnv`).
- Wire invites: capture `createInvite()` at `actions/invites.ts:67`, build link from `NEXT_PUBLIC_APP_URL` + `/invite/${token}`, send, persist to `notification`.
- Fill the two auth hooks (`auth.ts:85-97`): password reset + verification.
- Extend `notification.kind` (`invite`, `password_reset`, `proposal`) as an outbox.
- Watch: auth emails may have no `agencyId` (NOT NULL on `notification`).

3. Stripe SaaS billing (vendor → agencies) — M

- Migration `0009`: add to `agency` — `stripeCustomerId`, `stripeSubscriptionId`, `subscriptionStatus`, `priceId`, `currentPeriodEnd`, `trialEndsAt`.
- Gate in `requireAgencyUser` (`permissions.ts:136-144`) — already checks `agency.status`; add subscription-state block (`past_due`/`canceled`).
- `createAgency` (`actions/platform.ts:74`) creates Stripe customer + trial sub.
- New route `src/app/api/stripe/webhook/route.ts` (POST, raw `req.text()` body).
- Add the official `stripe` npm SDK (subscriptions + signature verification).
- Env: `STRIPE_WEBHOOK_SECRET` + price IDs. Surface status on platform detail page.
- Stripe Connect (traveler payments) = deferred to Phase 1.5, separate module on the `payment/booking` plane.

4. PDF proposals — M

- Today = HTML + `window.print()`. Add real renderer: Puppeteer + `@sparticuz/chromium` on Vercel serverless, store via `@vercel/blob` (`src/lib/storage.ts`). Alt: `react-pdf`. Fix the broken proposal link.

5. E-signature acceptance — M (greenfield)

- Add to `product`: `shareToken` (unique), `acceptedAt`, `declinedAt`, `signerName`, `signerEmail`, `signatureData`, `signerIp`, `signerUserAgent`.
- Public token route mirroring `/i/[token]`: `generateProposalLink` (copy `generateShareLink` `bookings.ts:383`), new unauth `/p/[token]`.
- New unauth `acceptProposalByToken` (first token-authed write) → set signature, `product.status=accepted`, `opportunity.stage=won`, send confirmation email.

6. Hotelbeds (hotels) + Amadeus (flights) — M

- New `src/lib/suppliers/hotelbeds.ts` mirroring `amadeus.ts` (Api-key + SHA-256).
- Split `getSupplier()` (`suppliers/index.ts:20`) into `getFlightSupplier()` / `getHotelSupplier()`; update `safeSearch` + call sites (`actions/search.ts`, `api/chat/route.ts`).
- Add `rateKey` to `HotelOffer`; persist supplier offers into `bookingItem.details` (AI

`createBooking` currently drops them; `confirmItemBooking` expects them).

- Add `HOTELBEDS_API_KEY`/`HOTELBEDS_SECRET` to `env.ts`.

Effort

#	Feature	Effort	Status
1	DB split (code parts)	S	✓ done — Neon branch is a manual step
2	Email (Resend)	S	✓ done
3	Stripe SaaS billing	M	✓ done (subscriptions); Connect deferred
4	PDF proposals	M	✓ done (@react-pdf, server-rendered)
5	E-signature	M	✓ done (public token + sign + audit)
6	Hotelbeds + flights	M	✓ done (per-vertical providers + rateKey)

Manual setup required (you)

- **Neon dev branch:** create a separate Neon branch/database for local dev; put its URL in local `.env`, keep prod `POSTGRES_URL` only in Vercel env. Set `PROTECTED_DB_HOSTS=<prod-neon-host>` in prod env so destructive scripts refuse to touch it (override with `ALLOW_PROD=1`).
- **Resend:** add `RESEND_API_KEY` + `EMAIL_FROM` (verified domain) to `.env` and Vercel. Without them, emails log to console (status `logged`).
- **Stripe billing:** add `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID` (the SaaS plan price), and `STRIPE_WEBHOOK_SECRET` to `.env` and Vercel. Register the webhook endpoint `POST /api/stripe/webhook` in the Stripe dashboard, subscribing to `customer.subscription.{created,updated,deleted}`. Enable the Billing Portal.
- **Suppliers:** add `AMADEUS_CLIENT_ID`/`AMADEUS_CLIENT_SECRET` (flights) and `HOTELBEDS_API_KEY`/`HOTELBEDS_SECRET` (hotels); optional `*_HOSTNAME=production`. Without them, search falls back to sample data. Hotelbeds keys availability by its own destination codes — pass a resolved `cityCode` for live hotel search (city name works against the mock).

Known follow-ups (not blocking Phase 1)

- AI `createBooking` tool (`api/chat/route.ts`) still doesn't persist the supplier offer into `bookingItem.details`, so AI-built items can't be auto-booked by `confirmItemBooking`. The human search→"Add to booking" flow does persist it.
- Hotelbeds destination-code resolution (content API) for city-name → code.

UI Polish & Responsive Improvements

Overview

Improve the visual quality, component consistency, and responsive behavior of the boilerplate UI across all pages. This includes refreshing the color palette with a subtle blue accent, replacing bare HTML elements with shadcn components, adding hover/transition effects via a reusable utility class, and introducing `sm:` breakpoints throughout for better tablet/mobile scaling. All custom styling goes in globals.css — no inline or custom CSS on components.

Quick Links

- Requirements — full requirements and acceptance criteria
- Action Required — manual steps needing human action

Dependency Graph

```
graph TD
    task-01-globals-css["01: globals.css Foundation"]
    task-02-layout["02: Layout Sticky Footer"]
    task-03-site-header["03: Site Header Responsive"]
    task-04-site-footer["04: Site Footer Responsive"]
    task-05-home-page["05: Home Page Overhaul"]
    task-06-dashboard["06: Dashboard Page"]
    task-07-chat-page["07: Chat Page"]
    task-08-profile-page["08: Profile Page"]
    task-09-auth-pages["09: Auth Pages"]
    task-10-setup-checklist["10: Setup Checklist"]
    task-11-starter-prompt-modal["11: Starter Prompt Modal"]
    task-01-globals-css --> task-05-home-page
    task-01-globals-css --> task-06-dashboard
    task-01-globals-css --> task-07-chat-page
    task-01-globals-css --> task-08-profile-page
    task-01-globals-css --> task-09-auth-pages
    task-01-globals-css --> task-10-setup-checklist
    task-01-globals-css --> task-11-starter-prompt-modal
    task-02-layout --> task-09-auth-pages
```

Waves

Wave	Tasks	Description
1	task-01, task-02, task-03, task-04	Foundation: globals.css enhancements, layout flex structure, header/footer responsive

2	task-05, task-06, task-07, task-08, task-09, task-10, task-11	All pages: Card adoption, responsive breakpoints, component swaps

Task Status

Wave 1

- [x] task-01-globals-css — globals.css color palette, animations, and utility classes
- [x] task-02-layout — Root layout sticky footer with flex column
- [x] task-03-site-header — Header responsive padding and sizing
- [x] task-04-site-footer — Footer responsive padding

Wave 2

- [ ] task-05-home-page — Home page Card adoption + responsive typography
- [ ] task-06-dashboard — Dashboard Card adoption + responsive grid
- [ ] task-07-chat-page — Chat Input swap + sticky form + responsive
- [ ] task-08-profile-page — Profile responsive stacking + grid fixes
- [ ] task-09-auth-pages — Auth pages animation, shadow, and background
- [ ] task-10-setup-checklist — Setup checklist Card adoption
- [ ] task-11-starter-prompt-modal — Textarea swap + responsive buttons

Action Required: UI Polish & Responsive Improvements

No manual steps required for this feature. All tasks can be implemented automatically.

Requirements: UI Polish & Responsive Improvements

Summary

The boilerplate UI is functional but visually flat — bare `<div>` elements instead of shadcn Cards, a monotone neutral-gray palette, no hover/transition effects, and minimal responsive breakpoints (most pages only use `md:`). The goal is to meaningfully improve visual quality, component consistency, and responsive behavior across all pages and components.

All changes must use Tailwind utilities and shadcn standards. Custom styling (animations, utilities, color tokens) goes in `globals.css`. No custom inline styles or CSS-in-JS on individual components. The result should look like a polished, high-quality default that can be customized by users of the boilerplate.

Goals

- Refresh the color palette with a subtle cool blue accent (shift primary/ring/accent from flat gray to hue 270)
- Replace all bare `<div>` cards with shadcn Card components for consistency
- Replace native `<input>`, `<button>`, and `<textarea>` elements with their shadcn equivalents
- Add hover/transition effects to interactive cards via a reusable `card-interactive` utility
- Introduce `sm:` breakpoints throughout for proper tablet/mobile scaling
- Add entrance animations to auth pages
- Fix the sticky footer layout so it works on short-content pages
- Ensure all pages scale well from 320px to 1536px+ viewports

Non-Goals

- No hamburger/mobile menu for the header — the current content (avatar + theme toggle) is minimal enough
- No new shadcn component installations — all needed components are already installed
- No changes to business logic, API routes, or authentication flows
- No new pages or routes
- No changes to the Geist font choice
- No Storybook or component documentation

Acceptance Criteria

- [ ] All pages render correctly in both light and dark modes
- [ ] No horizontal scroll on any page at 320px viewport width

- [ ] Feature cards, dashboard cards, and next steps cards use shadcn Card components
- [ ] Chat input uses shadcn Input, not a native `<input>`
- [ ] Footer sticks to the bottom on short-content pages (auth pages)
- [ ] Primary color has a visible cool-blue tint (not flat gray)
- [ ] Cards have hover lift effect with shadow transition
- [ ] All grids have `sm:` breakpoints for tablet-size viewports
- [ ] `pnpm lint` and `pnpm typecheck` pass with no errors

Assumptions

- Tailwind CSS v4 is in use (confirmed: `@import "tailwindcss"` syntax, `@theme inline` block)
- `tw-animate-css` is installed in dependencies but currently unused (confirmed in package.json)
- The ThemeProvider from next-themes renders children directly without a wrapping `<div>` (confirmed by reading the component)
- All shadcn components needed (Card, Input, Button, Textarea, Badge, Avatar, Dialog) are already installed

Technical Constraints

- All custom CSS (animations, utilities, color tokens) must go in `src/app/globals.css`
- Component files should only use Tailwind utility classes and shadcn components — no `style={{}}` props
- Must maintain both light and dark mode support via the existing OKLch CSS variable system
- Must not break existing functionality (auth flows, chat, diagnostics)

Task 01: globals.css Foundation

Status

complete

Wave

1

Description

Update `globals.css` to establish the visual foundation for the entire UI improvement. This includes importing the unused `tw-animate-css` package, refreshing the color palette with a subtle blue accent, adding custom keyframe animations, expanding the base layer with better defaults, and defining reusable utility classes. All subsequent tasks depend on the utilities and color tokens defined here.

Dependencies

Depends on: None (Wave 1) **Blocks:** task-05-home-page.md, task-06-dashboard.md, task-07-chat-page.md, task-08-profile-page.md, task-09-auth-pages.md, task-10-setup-checklist.md, task-11-starter-prompt-modal.md

Context from dependencies: N/A — this is the foundation task.

Files to Modify

- `src/app/globals.css` — Add tw-animate-css import, update color tokens, add animations, expand base layer, add utility classes

Technical Details

Implementation Steps

1. **Add tw-animate-css import** — The package `tw-animate-css@1.4.0` is already installed (in package.json) but never imported. Add the import right after the tailwindcss import:

```
@import "tailwindcss";
@import "tw-animate-css";
```

1. **Add animation tokens to `@theme inline`** — Add these at the end of the existing `@theme`

`inline` block (after the `--radius-xl` line):

```
--animate-fade-in: fade-in 0.3s ease-out;
--animate-fade-up: fade-up 0.4s ease-out;
--animate-scale-in: scale-in 0.2s ease-out;
```

1. **Update light mode color tokens in `:root`** — Change these 3 values (leave all other tokens unchanged):

```
/* CHANGE these 3 lines: */
--primary: oklch(0.21 0.034 270);          /* was oklch(0.21 0.006 285.885) */
--ring: oklch(0.705 0.06 270);              /* was oklch(0.705 0.015 286.067) */
--accent: oklch(0.96 0.012 270);            /* was oklch(0.967 0.001 286.375) */
```

1. **Update dark mode color tokens in `.dark`** — Change these 3 values (leave all other tokens unchanged):

```
/* CHANGE these 3 lines: */
--primary: oklch(0.92 0.02 270);            /* was oklch(0.92 0.004 286.32) */
--ring: oklch(0.552 0.05 270);              /* was oklch(0.552 0.016 285.938) */
--accent: oklch(0.28 0.018 270);            /* was oklch(0.274 0.006 286.033) */
```

1. **Add keyframe definitions** — Add these right before the `@layer base` block:

```
@keyframes fade-in {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes fade-up {
  from { opacity: 0; transform: translateY(8px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes scale-in {
  from { opacity: 0; transform: scale(0.97); }
  to { opacity: 1; transform: scale(1); }
}
```

1. **Expand the `@layer base` block** — Replace the existing `@layer base` block with:

```
@layer base {
  * {
    @apply border-border;
  }
  html {
    scroll-behavior: smooth;
  }
  body {
    @apply bg-background text-foreground;
    font-feature-settings: "rlig" 1, "calt" 1;
```

```
  }
  :focus-visible {
    @apply outline-2 outline-offset-2 outline-ring;
  }
}
```

1. **Add utility classes** — Add a new `@layer utilities` block after the `@layer base` block:

```
@layer utilities {
  .card-interactive {
    @apply transition-all duration-200 ease-out;
  }
  .card-interactive:hover {
    @apply shadow-md -translate-y-0.5;
  }
  .auth-bg {
    background-image: radial-gradient(
      circle at 50% 0%,
      var(--accent) 0%,
      transparent 50%
    );
  }
}
```

Final file structure (order of sections)

1. `@import "tailwindcss";`
2. `@import "tw-animate-css";`
3. `@theme inline { ... }` (with animation tokens added at end)
4. `:root { ... }` (with 3 updated color values)
5. `.dark { ... }` (with 3 updated color values)
6. `@keyframes` definitions (3 keyframes)
7. `@layer base { ... }` (expanded)
8. `@layer utilities { ... }` (new)

Acceptance Criteria

- [ ] `tw-animate-css` is imported and animation utilities like `animate-fade-up` are available
- [ ] `:root` has `--primary: oklch(0.21 0.034 270)`, `--ring: oklch(0.705 0.06 270)`, `--accent: oklch(0.96 0.012 270)`
- [ ] `.dark` has `--primary: oklch(0.92 0.02 270)`, `--ring: oklch(0.552 0.05 270)`, `--accent: oklch(0.28 0.018 270)`
- [ ] Three keyframe animations (`fade-in`, `fade-up`, `scale-in`) are defined
- [ ] `@theme inline` includes `--animate-fade-in`, `--animate-fade-up`, `--animate-scale-in`
- [ ] `.card-interactive` utility class is defined with hover shadow and translate
- [ ] `.auth-bg` utility class is defined with radial gradient
- [ ] `html` has `scroll-behavior: smooth`
- [ ] `:focus-visible` has global outline styling
- [ ] All other existing color tokens remain unchanged

Task 02: Layout Sticky Footer

Status

complete

Wave

1

Description

Update the root layout to use a flex column structure that pushes the footer to the bottom of the viewport on short-content pages. Currently, pages like login/register have minimal content and the footer floats up, leaving blank space below. Adding `min-h-screen flex flex-col` to the body and `flex-1` to the main element creates a proper sticky footer layout.

Dependencies

Depends on: None (Wave 1) **Blocks:** task-09-auth-pages.md (auth pages rely on the flex-1 main for proper vertical centering)

Context from dependencies: N/A — this is a foundation task.

Files to Modify

- `src/app/layout.tsx` — Add flex classes to body and main elements

Technical Details

Implementation Steps

1. **Add flex layout classes to the `<body>` element** (line 92). The current className is:

```
className={`${geistSans.variable} ${geistMono.variable} antialiased`}
```

Change it to:

```
className={`${geistSans.variable} ${geistMono.variable} antialiased min-h-screen fle
```

1. **Add `flex-1` to the `<main>` element** (line 101). The current element is:

```
<main id="main-content">{children}</main>
```

Change it to:

```
<main id="main-content" className="flex-1">{children}</main>
```

Why this works

The ThemeProvider component (`src/components/theme-provider.tsx`) is a pass-through wrapper around `next-themes` ThemeProvider — it renders children directly without adding a wrapping `<div>`. So the flex column chain from `<body>` flows directly through to `<SiteHeader>`, `<main>`, and `<SiteFooter>`.

The `min-h-screen` ensures the body fills at least the full viewport. `flex flex-col` creates a vertical flex container. `flex-1` on `<main>` makes it expand to fill all available space, pushing `<SiteFooter>` to the bottom.

Acceptance Criteria

- [ ] `<body>` has classes `min-h-screen flex flex-col` in addition to existing font variable classes
- [ ] `<main>` has `className="flex-1"`
- [ ] On short-content pages (e.g. `/login`), the footer sits at the bottom of the viewport
- [ ] On long-content pages (e.g. `/profile`), the footer sits below the content as usual
- [ ] No visual regressions on any page

Task 03: Site Header Responsive

Status

complete

Wave

1

Description

Update the site header component with responsive padding and sizing so it scales better on mobile devices. Currently the header uses fixed `px-4 py-4` padding and `text-2xl` logo text that could be tighter on small phones. The action buttons also use a fixed `gap-4` that is generous on narrow screens.

Dependencies

Depends on: None (Wave 1) **Blocks:** None

Context from dependencies: N/A

Files to Modify

- `src/components/site-header.tsx` — Update Tailwind classes for responsive sizing

Technical Details

Implementation Steps

1. **Update nav padding** (line 19). Change:

```
container mx-auto px-4 py-4 flex justify-between items-center
```

To:

```
container mx-auto px-3 sm:px-4 py-3 sm:py-4 flex justify-between items-center
```

1. **Update logo text size** (line 21). Change the `<h1>` className from:

```
text-2xl font-bold
```

To:

```
text-xl sm:text-2xl font-bold
```

1. **Update actions group gap** (line 38). Change:

```
flex items-center gap-4
```

To:

```
flex items-center gap-2 sm:gap-4
```

Current file reference

The full current file is at `src/components/site-header.tsx` (46 lines). The structure is:

- Skip-to-main-content link (accessibility)
- `<header className="border-b">` wrapper
- `<nav>` with logo on left, UserProfile + ModeToggle on right

Acceptance Criteria

- [ ] Nav padding is `px-3 sm:px-4 py-3 sm:py-4`
- [ ] Logo text is `text-xl sm:text-2xl`
- [ ] Actions gap is `gap-2 sm:gap-4`
- [ ] Header looks good at 320px viewport width (no overflow, no cramping)
- [ ] Header looks unchanged at desktop widths (1024px+)

Task 04: Site Footer Responsive

Status

complete

Wave

1

Description

Update the site footer with responsive padding to match the header's responsive scaling and maintain consistent container padding across the app. Currently the footer uses fixed `py-6` and `px-4` that could be tighter on mobile and more generous on large screens.

Dependencies

Depends on: None (Wave 1) **Blocks:** None

Context from dependencies: N/A

Files to Modify

- `src/components/site-footer.tsx` — Update Tailwind classes for responsive padding

Technical Details

Implementation Steps

1. **Update footer padding** (line 5). Change:

```
border-t py-6 text-center text-sm text-muted-foreground
```

To:

```
border-t py-4 sm:py-6 text-center text-sm text-muted-foreground
```

1. **Update container padding** (line 6). Change:

```
container mx-auto px-4
```

To:

```
container mx-auto px-4 sm:px-6 lg:px-8
```

Current file reference

The file is at `src/components/site-footer.tsx` (24 lines). It renders a `<footer>` with a GitHubStars component and an attribution link.

Acceptance Criteria

- [ ] Footer padding is `py-4 sm:py-6`
- [ ] Container padding is `px-4 sm:px-6 lg:px-8`
- [ ] Footer is slightly more compact on mobile (py-4 vs py-6)
- [ ] Footer has more generous side padding on large screens (lg:px-8)

Task 05: Home Page Overhaul

Status

pending

Wave

2

Description

Overhaul the home page with responsive typography, shadcn Card components replacing bare divs, hover effects on interactive cards, and better grid breakpoints. This is the highest-visibility page and sets the visual tone for the entire app. The feature cards and next steps cards are currently plain `<div>` elements with `border rounded-lg` — they need to be replaced with proper shadcn Card components with the `card-interactive` hover utility.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 defines the `card-interactive` utility class in globals.css which provides `transition-all duration-200 ease-out` with hover effects (`shadow-md -translate-y-0.5`). This task uses that class on all interactive cards. Task 01 also updates the color palette so `text-primary` on icons will show the new blue-tinted primary color.

Files to Modify

- `src/app/page.tsx` — Replace bare divs with shadcn Cards, update typography responsive classes, fix grid breakpoints

Technical Details

Implementation Steps

1. **Add Card imports** at the top of the file. Add this import:

```
import {
  Card,
  CardContent,
  CardHeader,
```

```
  CardTitle,
} from "@/components/ui/card";
```

1. **Update container spacing** (line 13). Change:

```
className="flex-1 container mx-auto px-4 py-12"
```

To:

```
className="flex-1 container mx-auto px-4 sm:px-6 lg:px-8 py-8 sm:py-12"
```

1. **Update hero h1 typography** (line 20). Change:

```
className="text-5xl font-bold tracking-tight bg-gradient-to-r from-primary via-prima
```

To:

```
className="text-3xl sm:text-4xl md:text-5xl font-bold tracking-tight bg-gradient-to-
```

1. **Update hero h2 subtitle** (line 24). Change:

```
className="text-2xl font-semibold text-muted-foreground"
```

To:

```
className="text-xl sm:text-2xl font-semibold text-muted-foreground"
```

1. **Update hero description paragraph** (line 27). Change:

```
className="text-xl text-muted-foreground"
```

To:

```
className="text-base sm:text-lg md:text-xl text-muted-foreground"
```

1. **Update "Video Tutorial" heading** (line 35). Change:

```
className="text-2xl font-semibold flex items-center justify-center gap-2"
```

To:

```
className="text-xl sm:text-2xl font-semibold flex items-center justify-center gap-2"
```

1. **Add shadow to video embed container** (line 43). Change:

```
className="relative pb-[56.25%] h-0 overflow-hidden rounded-lg border"
```

To:

```
className="relative pb-[56.25%] h-0 overflow-hidden rounded-lg border shadow-md"
```

1. **Update feature grid** (line 55). Change:

```
className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mt-12"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 sm:gap-6 mt-12"
```

1. **Replace all 4 feature card divs** (lines 56-92) with shadcn Card components. Each card currently looks like:

```
<div className="p-6 border rounded-lg">
  <h3 className="font-semibold mb-2 flex items-center gap-2">
    <Shield className="h-4 w-4" />
    Authentication
  </h3>
  <p className="text-sm text-muted-foreground">
    Better Auth with Google OAuth integration
  </p>
</div>
```

Replace each with:

```
<Card className="card-interactive">
  <CardHeader className="pb-3">
    <CardTitle className="text-base flex items-center gap-2">
      <Shield className="h-4 w-4 text-primary" />
      Authentication
    </CardTitle>
  </CardHeader>
  <CardContent>
    <p className="text-sm text-muted-foreground">
      Better Auth with Google OAuth integration
    </p>
  </CardContent>
</Card>
```

Do this for all 4 cards (Authentication/Shield, Database/Database, AI Ready/Bot, UI Components/Palette). Add `text-primary` to each icon.

1. **Update "Next Steps" heading** (line 97). Change:

```
className="text-2xl font-semibold"
```

To:

```
className="text-xl sm:text-2xl font-semibold"
```

1. **Update next steps grid** (line 98). Change:

```
className="grid grid-cols-1 md:grid-cols-2 gap-4 text-left"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 gap-4 text-left"
```

1. **Replace all 4 next steps card divs** (lines 99-169) with shadcn Card components. Each currently looks like:

```
<div className="p-4 border rounded-lg">
  <h4 className="font-medium mb-2">1. Set up environment variables</h4>
  ...content...
</div>
```

Replace each wrapper with:

```
<Card className="card-interactive">
  <CardContent className="pt-6">
    <h4 className="font-medium mb-2">1. Set up environment variables</h4>
    ...content stays the same...
  </CardContent>
</Card>
```

Use `pt-6` on CardContent since there's no CardHeader for these simpler cards.

Acceptance Criteria

- [ ] Hero h1 scales: `text-3xl` on mobile, `text-4xl` on sm, `text-5xl` on md+
- [ ] Hero h2 and description also scale responsively
- [ ] Feature grid uses `sm:grid-cols-2` breakpoint (2 columns at 640px)
- [ ] All 4 feature cards use shadcn `Card`/`CardHeader`/`CardContent` with `card-interactive`
- [ ] All feature card icons have `text-primary` class
- [ ] All 4 next steps cards use shadcn `Card`/`CardContent` with `card-interactive`
- [ ] Video embed has `shadow-md`
- [ ] Container has responsive padding: `px-4 sm:px-6 lg:px-8`
- [ ] No TypeScript errors (Card imports are correct)

Task 06: Dashboard Page

Status

pending

Wave

2

Description

Update the dashboard page to use shadcn Card components instead of bare divs, add responsive breakpoints, improve the loading state, and add visual polish. The dashboard currently has two plain `<div>` cards with `border border-border rounded-lg` and a simple "Loading..." text.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 defines the `card-interactive` utility class in globals.css and updates the primary color to a blue-tinted hue. This task uses `card-interactive` on cards and `text-primary` on new icon accents.

Files to Modify

- `src/app/dashboard/page.tsx` — Replace divs with Cards, add responsive classes, improve loading

Technical Details

Implementation Steps

1. **Add new imports**. The file currently imports: `Link`, `Lock`, `UserProfile`, `Button`, `useDiagnostics`, `useSession`. Add these:

```
import { Lock, Bot, User } from "lucide-react";
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
```

```
  CardTitle,
} from "@/components/ui/card";
import { Spinner } from "@/components/ui/spinner";
```

Remove the existing `Lock` from the lucide-react import (it's being re-imported in the combined import).

1. **Update loading state** (lines 15-19). Change:

```
return (
  <div className="flex justify-center items-center h-screen">
    Loading...
  </div>
);
```

To:

```
return (
  <div className="flex justify-center items-center h-screen">
    <Spinner size="lg" />
  </div>
);
```

1. **Update unauthenticated state container** (line 24). Change:

```
className="container mx-auto px-4 py-12"
```

To:

```
className="container mx-auto px-4 sm:px-6 lg:px-8 py-8 sm:py-12"
```

1. **Wrap unauthenticated content in Card** (lines 26-34). Change:

```
<div className="mb-8">
  <Lock className="w-16 h-16 mx-auto mb-4 text-muted-foreground" />
  <h1 className="text-2xl font-bold mb-2">Protected Page</h1>
  <p className="text-muted-foreground mb-6">
    You need to sign in to access the dashboard
  </p>
</div>
```

To:

```
<Card className="mb-8 text-center">
  <CardContent className="pt-6">
    <Lock className="w-16 h-16 mx-auto mb-4 text-muted-foreground" />
    <h1 className="text-2xl font-bold mb-2">Protected Page</h1>
    <p className="text-muted-foreground mb-6">
      You need to sign in to access the dashboard
    </p>
  </CardContent>
```

```
</Card>
```

1. **Update authenticated container** (line 40). Change:

```
className="container mx-auto p-6"
```

To:

```
className="container mx-auto p-4 sm:p-6 max-w-5xl"
```

1. **Update title section** (lines 41-43). Change:

```
<div className="flex justify-between items-center mb-8">
  <h1 className="text-3xl font-bold">Dashboard</h1>
</div>
```

To:

```
<div className="flex flex-wrap justify-between items-center gap-4 mb-6 sm:mb-8">
  <h1 className="text-2xl sm:text-3xl font-bold">Dashboard</h1>
</div>
```

1. **Update card grid** (line 45). Change:

```
className="grid grid-cols-1 md:grid-cols-2 gap-6"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 gap-6"
```

1. **Replace AI Chat card div** (lines 46-59). Change:

```
<div className="p-6 border border-border rounded-lg">
  <h2 className="text-xl font-semibold mb-2">AI Chat</h2>
  <p className="text-muted-foreground mb-4">
    Start a conversation with AI using the Vercel AI SDK
  </p>
  ...button...
</div>
```

To:

```
<Card className="card-interactive">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <Bot className="h-5 w-5 text-primary" />
      AI Chat
    </CardTitle>
```

```
    <CardDescription>
      Start a conversation with AI using the Vercel AI SDK
    </CardDescription>
  </CardHeader>
  <CardContent>
    ...button stays the same...
  </CardContent>
</Card>
```

1. **Replace Profile card div** (lines 62-76). Change:

```
<div className="p-6 border border-border rounded-lg">
  <h2 className="text-xl font-semibold mb-2">Profile</h2>
  <p className="text-muted-foreground mb-4">
    Manage your account settings and preferences
  </p>
  <div className="space-y-2">
    <p><strong>Name:</strong> {session.user.name}</p>
    <p><strong>Email:</strong> {session.user.email}</p>
  </div>
</div>
```

To:

```
<Card className="card-interactive">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <User className="h-5 w-5 text-primary" />
      Profile
    </CardTitle>
    <CardDescription>
      Manage your account settings and preferences
    </CardDescription>
  </CardHeader>
  <CardContent>
    <div className="space-y-2">
      <p><strong>Name:</strong> {session.user.name}</p>
      <p><strong>Email:</strong> {session.user.email}</p>
    </div>
  </CardContent>
</Card>
```

Acceptance Criteria

- [ ] Both dashboard cards use shadcn `Card`/`CardHeader`/`CardTitle`/`CardDescription`/`CardContent`
- [ ] Both cards have `card-interactive` class
- [ ] AI Chat card has Bot icon with `text-primary`, Profile card has User icon with `text-primary`
- [ ] Loading state shows `Spinner` component instead of "Loading..." text
- [ ] Container has `max-w-5xl` to prevent overly wide content
- [ ] Grid uses `sm:grid-cols-2` breakpoint

- [ ] Title scales: `text-2xl sm:text-3xl`
- [ ] Unauthenticated state wraps in Card
- [ ] No TypeScript errors

Task 07: Chat Page

Status

pending

Wave

2

Description

Update the chat page to use shadcn Input/Button components instead of native HTML elements, add responsive breakpoints to the header and message bubbles, make the input form sticky at the bottom, and improve the empty state. This page has the most interactive elements and benefits significantly from consistent component usage.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette (so `text-primary`, `bg-primary` references use the new blue-tinted primary). No utility classes from Task 01 are used on this page (chat doesn't use `card-interactive`).

Files to Modify

- `src/app/chat/page.tsx` — Replace native elements with shadcn components, add responsive classes, sticky input

Technical Details

Implementation Steps

1. **Add Input import**. Add to the existing imports:

```
import { Input } from "@/components/ui/input";
```

The file already imports `Button` from `@/components/ui/button`.

1. **Also import Bot icon** for the improved empty state. Add `Bot` to the existing lucide-react import:

```
import { Copy, Check, Loader2, Bot } from "lucide-react";
```

1. **Replace native `<button>` in CopyButton** (lines 165-177). Change the function body from:

```
return (
  <button
    onClick={handleCopy}
    className="p-1 hover:bg-muted rounded transition-colors"
    title="Copy to clipboard"
  >
    {copied ? (
      <Check className="h-3.5 w-3.5 text-green-500" />
    ) : (
      <Copy className="h-3.5 w-3.5 text-muted-foreground" />
    )}
  </button>
);
```

To:

```
return (
  <Button
    variant="ghost"
    size="icon"
    className="h-7 w-7"
    onClick={handleCopy}
    title="Copy to clipboard"
  >
    {copied ? (
      <Check className="h-3.5 w-3.5 text-green-500" />
    ) : (
      <Copy className="h-3.5 w-3.5 text-muted-foreground" />
    )}
  </Button>
);
```

1. **Update container** (line 247). Change:

```
className="container mx-auto px-4 py-8"
```

To:

```
className="container mx-auto px-4 sm:px-6 lg:px-8 py-6 sm:py-8"
```

1. **Update chat header** (lines 249-261). Change:

```
<div className="flex justify-between items-center mb-6 pb-4 border-b">
  <h1 className="text-2xl font-bold">AI Chat</h1>
  <div className="flex items-center gap-4">
    <span className="text-sm text-muted-foreground">
      Welcome, {session.user.name}!
```

```
    </span>
```

To:

```
<div className="flex flex-col sm:flex-row sm:justify-between sm:items-center gap-3 s
  <h1 className="text-xl sm:text-2xl font-bold">AI Chat</h1>
  <div className="flex items-center gap-2 sm:gap-4">
    <span className="text-sm text-muted-foreground hidden sm:inline">
      Welcome, {session.user.name}!
    </span>
```

Key changes: flex stacks on mobile, welcome text hidden on mobile (`hidden sm:inline`), smaller gaps.

1. **Update message bubble max-widths** (line ~289-290). For user messages, change:

```
bg-primary text-primary-foreground ml-auto max-w-[80%]
```

To:

```
bg-primary text-primary-foreground ml-auto max-w-[85%] sm:max-w-[80%] md:max-w-2xl s
```

For AI messages, change:

```
bg-muted max-w-[80%]
```

To:

```
bg-muted max-w-[85%] sm:max-w-[80%] md:max-w-2xl shadow-sm
```

1. **Update message area** (line 271). Change:

```
className="min-h-[50vh] overflow-y-auto space-y-4 mb-4"
```

To:

```
className="min-h-[50vh] overflow-y-auto space-y-4 mb-4 pb-4"
```

1. **Improve empty state** (lines 272-275). Change:

```
<div className="text-center text-muted-foreground py-12">
  Start a conversation with AI
</div>
```

To:

```
<div className="text-center text-muted-foreground py-16 space-y-3">
```

```
  <Bot className="h-12 w-12 mx-auto text-muted-foreground/50" />
  <p className="text-lg font-medium">Start a conversation with AI</p>
  <p className="text-sm">Type a message below to begin chatting</p>
</div>
```

1. **Make input form sticky and replace native input** (lines 317-344). Change the form from:

```
<form
  onSubmit={...}
  className="flex gap-2"
>
  <input
    value={input}
    onChange={(e) => setInput(e.target.value)}
    placeholder="Type your message..."
    className="flex-1 p-2 border border-border rounded-md focus:outline-none focus:r
    disabled={isStreaming}
  />
  <Button type="submit" disabled={!input.trim() || isStreaming}>
    ...
  </Button>
</form>
```

To:

```
<form
  onSubmit={...}
  className="sticky bottom-0 bg-background py-4 border-t z-10 flex gap-2 sm:gap-3"
>
  <Input
    value={input}
    onChange={(e) => setInput(e.target.value)}
    placeholder="Type your message..."
    className="flex-1 min-w-0"
    disabled={isStreaming}
  />
  <Button type="submit" disabled={!input.trim() || isStreaming}>
    ...
  </Button>
</form>
```

Key changes: `sticky bottom-0 bg-background py-4 border-t z-10` makes the form stick. `Input` replaces native `<input>` for consistent styling. `min-w-0` prevents flex overflow.

Acceptance Criteria

- [ ] Chat input uses shadcn `Input` component, not a native `<input>`
- [ ] CopyButton uses shadcn `Button variant="ghost" size="icon"`, not a native `<button>`
- [ ] Chat header stacks vertically on mobile (`flex-col`) and is horizontal on sm+
- [ ] Welcome text is hidden on mobile (`hidden sm:inline`)
- [ ] Message bubbles have `shadow-sm` for depth

- [ ] Message widths scale: `max-w-[85%]` on mobile, `max-w-[80%]` on sm, `max-w-2xl` on md+
- [ ] Input form is sticky at bottom with `bg-background` and `border-t`
- [ ] Empty state shows Bot icon, title text, and subtitle
- [ ] Container has responsive padding: `px-4 sm:px-6 lg:px-8`
- [ ] No TypeScript errors (Input import is correct)

Task 08: Profile Page

Status

pending

Wave

2

Description

Update the profile page with responsive stacking, improved grid breakpoints, and flex-wrap on tight rows. The avatar section needs to stack vertically on mobile, the quick actions grid needs an intermediate `sm:` breakpoint, and all the `flex items-center justify-between` rows in cards and dialogs need `flex-wrap` to prevent overflow on narrow screens.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette. No utility classes from Task 01 are specifically used on this page (profile cards are informational, not interactive).

Files to Modify

- `src/app/profile/page.tsx` — Update responsive classes throughout

Technical Details

Implementation Steps

1. **Update container** (line 67). Change:

```
className="container max-w-4xl mx-auto py-8 px-4"
```

To:

```
className="container max-w-4xl mx-auto py-6 sm:py-8 px-4 sm:px-6 lg:px-8"
```

1. **Update title row** (line 68). Change:

```
className="flex items-center gap-4 mb-8"
```

To:

```
className="flex items-center gap-2 sm:gap-4 mb-6 sm:mb-8"
```

1. **Update title text** (line 78). Change:

```
className="text-3xl font-bold"
```

To:

```
className="text-2xl sm:text-3xl font-bold"
```

1. **Update avatar section** (line 85). Change:

```
className="flex items-center space-x-4"
```

To:

```
className="flex flex-col items-center sm:flex-row sm:items-center gap-4"
```

Important: remove `space-x-4` entirely — it conflicts with `flex-col`. The `gap-4` handles spacing in both directions.

1. **Update avatar size** (line 86). Change:

```
className="h-20 w-20"
```

To:

```
className="h-16 w-16 sm:h-20 sm:w-20"
```

1. **Update text container** next to avatar (line 96). Change:

```
className="space-y-2"
```

To:

```
className="space-y-2 text-center sm:text-left"
```

1. **Update name heading** (line 97). Change:

```
className="text-2xl font-semibold"
```

To:

```
className="text-xl sm:text-2xl font-semibold"
```

1. **Update email row** (line 98). Change:

```
className="flex items-center gap-2 text-muted-foreground"
```

To:

```
className="flex flex-wrap items-center justify-center sm:justify-start gap-2 text-mu
```

1. **Update Account Status grid** (line 160). Change:

```
className="grid grid-cols-1 md:grid-cols-2 gap-4"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 gap-4"
```

1. **Update all status/activity flex rows** — add `flex-wrap gap-2` and responsive padding. These appear at multiple locations. For each row that has `flex items-center justify-between p-4 border rounded-lg`, change to:

```
flex flex-wrap items-center justify-between gap-2 p-3 sm:p-4 border rounded-lg
```

Apply this to the following locations:

- Line 161 (Email Verification status)
- Line 172 (Account Type status)
- Line 196 (Current Session activity)

1. **Update Quick Actions grid** (line 224). Change:

```
className="grid grid-cols-1 md:grid-cols-3 gap-4"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4"
```

1. **Update dialog inner rows** — For the Security Settings dialog (lines 325, 342, 357), each row with `flex items-center justify-between p-4 border rounded-lg`, change to:

```
flex flex-wrap items-center justify-between gap-2 p-3 sm:p-4 border rounded-lg
```

1. **Update Email Preferences dialog rows** (lines 388, 396) — same pattern, change:

```
flex items-center justify-between p-4 border rounded-lg
```

To:

```
flex flex-wrap items-center justify-between gap-2 p-3 sm:p-4 border rounded-lg
```

Acceptance Criteria

- [ ] Avatar section stacks vertically on mobile (flex-col) and is horizontal on sm+
- [ ] Avatar is smaller on mobile (h-16 w-16) and full size on sm+ (h-20 w-20)
- [ ] Text next to avatar is centered on mobile, left-aligned on sm+
- [ ] Email row wraps on narrow screens (flex-wrap)
- [ ] Quick actions grid has intermediate breakpoint: 1 col → 2 at sm → 3 at lg
- [ ] Account status grid uses sm:grid-cols-2 instead of md:grid-cols-2
- [ ] All flex justify-between rows in cards and dialogs have flex-wrap and gap-2
- [ ] Container and title have responsive padding and sizing
- [ ] No overflow or horizontal scroll at 320px viewport
- [ ] Profile page title scales: `text-2xl sm:text-3xl`

Task 09: Auth Pages

Status

pending

Wave

2

Description

Polish all 4 auth pages (login, register, forgot-password, reset-password) with entrance animations, card shadows, a subtle background gradient, and improved vertical centering that works with the new flex layout. These pages are the first thing new users see, so visual polish matters.

Dependencies

Depends on: task-01-globals-css.md, task-02-layout.md **Blocks:** None

Context from dependencies: Task 01 defines the `auth-bg` utility class (radial gradient from accent color) and the `animate-fade-up` animation in globals.css. Task 02 adds `flex-1` to the `<main>` element, so auth pages can use `flex-1` to fill available vertical space properly.

Files to Modify

- `src/app/(auth)/login/page.tsx` — Add animation, shadow, background, update min-height
- `src/app/(auth)/register/page.tsx` — Same changes
- `src/app/(auth)/forgot-password/page.tsx` — Same changes
- `src/app/(auth)/reset-password/page.tsx` — Same changes

Technical Details

Implementation Steps

All 4 files follow the exact same pattern. Each has an outer wrapper div with a Card inside.

For each file, make these 2 changes:

Change 1: Update the wrapper div. Each file has:

```
<div className="flex min-h-[calc(100vh-4rem)] items-center justify-center p-4">
```

Change to:

```
<div className="auth-bg flex flex-1 min-h-[calc(100vh-8rem)] items-center justify-ce
```

Changes:

- `auth-bg` — adds the subtle radial gradient background from globals.css
- `flex-1` — fills available space in the flex column layout (from Task 02)
- `min-h-[calc(100vh-8rem)]` — accounts for both header and footer height (was just 4rem for header)

Change 2: Add shadow and animation to the Card. Each file has:

```
<Card className="w-full max-w-md">
```

Change to:

```
<Card className="w-full max-w-md shadow-lg animate-fade-up">
```

Changes:

- `shadow-lg` — adds elevation/depth to the card
- `animate-fade-up` — entrance animation (defined in globals.css keyframes, registered in @theme inline)

File-specific locations:

login/page.tsx:

- Wrapper div: line 27
- Card: line 28

register/page.tsx:

- Wrapper div: line 20
- Card: line 21

forgot-password/page.tsx:

- Wrapper div: line 21
- Card: line 22

reset-password/page.tsx:

- Wrapper div: line 21
- Card: line 22

Acceptance Criteria

- [ ] All 4 auth pages have `auth-bg` class on the wrapper div
- [ ] All 4 auth pages have `flex-1 min-h-[calc(100vh-8rem)]` on the wrapper div
- [ ] All 4 auth cards have `shadow-lg` for depth
- [ ] All 4 auth cards have `animate-fade-up` for entrance animation
- [ ] Cards animate upward on page load (smooth fade + translate)
- [ ] Subtle radial gradient visible in the background (especially visible in dark mode)
- [ ] Footer stays at the bottom of the viewport on all auth pages
- [ ] No visual regressions to auth form functionality

Task 10: Setup Checklist Card Adoption

Status

pending

Wave

2

Description

Replace the bare `<div>` wrapper in the SetupChecklist component with a shadcn Card component for visual consistency with the rest of the app. The checklist is displayed on the home page and currently uses `p-6 border rounded-lg` which is the same pattern as the feature cards that are being upgraded to shadcn Cards.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette. The setup checklist does not use `card-interactive` since it's an informational display, not a clickable card.

Files to Modify

- `src/components/setup-checklist.tsx` — Replace wrapper div with shadcn Card

Technical Details

Implementation Steps

1. **Add Card imports.** Add at the top:

```
import {
  Card,
  CardContent,
  CardHeader,
  CardTitle,
} from "@/components/ui/card";
```

1. **Replace the outer wrapper** (line 124). The current structure is:

```
<div className="p-6 border rounded-lg text-left">
  <div className="flex items-center justify-between mb-4">
    <div>
      <h3 className="font-semibold">Setup checklist</h3>
      <p className="text-sm text-muted-foreground">
        {completed}/{steps.length} completed
      </p>
    </div>
    <Button size="sm" onClick={load} disabled={loading}>
      {loading ? "Checking..." : "Re-check"}
    </Button>
  </div>

  {error ? <div className="text-sm text-destructive">{error}</div> : null}

  <ul className="space-y-2">
    ...checklist items...
  </ul>

  {data ? (
    <div className="mt-4 text-xs text-muted-foreground">
      Last checked: {new Date(data.timestamp).toLocaleString()}
    </div>
  ) : null}
</div>
```

Replace with:

```
<Card className="text-left">
  <CardHeader className="pb-4">
    <div className="flex items-center justify-between">
      <div>
        <CardTitle className="text-base">Setup checklist</CardTitle>
        <p className="text-sm text-muted-foreground">
          {completed}/{steps.length} completed
        </p>
      </div>
      <Button size="sm" onClick={load} disabled={loading}>
        {loading ? "Checking..." : "Re-check"}
      </Button>
    </div>
  </CardHeader>
  <CardContent>
    {error ? <div className="text-sm text-destructive mb-4">{error}</div> : null}

    <ul className="space-y-2">
      ...checklist items stay the same...
    </ul>

    {data ? (
      <div className="mt-4 text-xs text-muted-foreground">
        Last checked: {new Date(data.timestamp).toLocaleString()}
      </div>
    ) : null}
  </CardContent>
```

```
</Card>
```

Key changes:

- Outer `<div>` → `<Card>`
- Title area wrapped in `<CardHeader>` with the flex row inside
- `h3` → `<CardTitle className="text-base">` (text-base to match current visual size)
- Content wrapped in `<CardContent>`
- Error div gets `mb-4` since CardContent doesn't have the same padding structure
- All checklist items and timestamp stay exactly the same

Acceptance Criteria

- [ ] Setup checklist uses shadcn `Card`/`CardHeader`/`CardTitle`/`CardContent`
- [ ] Visual appearance is consistent — no jarring size or spacing changes
- [ ] Title uses `CardTitle` with `text-base` to maintain current size
- [ ] Re-check button still works and is positioned to the right of the title
- [ ] Error state still displays correctly
- [ ] Checklist items render identically to before
- [ ] No TypeScript errors

Task 11: Starter Prompt Modal

Status

pending

Wave

2

Description

Replace the native `<textarea>` in the StarterPromptModal with the shadcn Textarea component for visual consistency, and make the button row responsive so buttons stack on very small screens.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette. No utility classes from Task 01 are used in this component.

Files to Modify

- `src/components/starter-prompt-modal.tsx` — Replace native textarea, update button layout

Technical Details

Implementation Steps

1. **Add Textarea import.** Add at the top:

```
import { Textarea } from "@/components/ui/textarea";
```

1. **Replace native `<textarea>`** (lines 161-167). Change:

```
<textarea
  id="project-description"
  placeholder="e.g., A task management app for teams with real-time collaboration, p
  value={projectDescription}
  onChange={(e) => setProjectDescription(e.target.value)}
```

```
  className="w-full h-24 px-3 py-2 border rounded-md resize-none text-sm"
/>
```

To:

```
<Textarea
  id="project-description"
  placeholder="e.g., A task management app for teams with real-time collaboration, p
  value={projectDescription}
  onChange={(e) => setProjectDescription(e.target.value)}
  className="h-24 resize-none"
/>
```

Key changes:

- `<textarea>` → `<Textarea>` (shadcn component)
- Remove explicit `w-full px-3 py-2 border rounded-md text-sm` — the shadcn Textarea handles these styles internally
- Keep `h-24 resize-none` as overrides

1. **Update button row** (line 174). Change:

```
className="flex gap-2"
```

To:

```
className="flex flex-col-reverse sm:flex-row gap-2"
```

This makes the buttons stack on mobile (with Cancel on top via `flex-col-reverse`) and sit side-by-side on sm+.

Acceptance Criteria

- [ ] Textarea uses shadcn `Textarea` component, not a native `<textarea>`
- [ ] Textarea has consistent focus ring styling matching other form elements
- [ ] Button row stacks on mobile (`flex-col-reverse`) and is horizontal on sm+ (`sm:flex-row`)
- [ ] "Copy Starter Prompt" button appears before "Cancel" button in both layouts
- [ ] Textarea placeholder text renders correctly
- [ ] Copy functionality still works (the `projectDescription` state binding is unchanged)
- [ ] No TypeScript errors